



Arab Academy for Science, Technology and Maritime Transport

College of Engineering and Technology

Computer Engineering Department

B. Sc. Final Year Project

**A BILINGUAL SIGN LANGUAGE RECOGNITION AND
COMMUNICATION SYSTEM**

Real-Time Letter and Word Recognition with a Web-Based Deployment

Presented By:

Ahmed Adel Sayed Gouda Ahmed

Hadeel Gamal Eldin Mohamed Ibrahim

Alaa Osama Mohammed Shehab

Salah Mohamed Salah Ibrahim

Supervised By:

Dr. Nada Mostafa Abdelaziem Mostafa

Dr. Reham Taher Abdelmajeed Salem Al Magraby

July – 2026

DECLARATION

I hereby certify that this report, which I now submit for assessment on the program of study leading to the award of Bachelor of Science in *Computer Engineering*, is all my own work and contains no Plagiarism. By submitting this report, I agree to the following terms:

Any text, diagrams or other material copied from other sources (including, but not limited to, books, journals, and the internet) have been clearly acknowledged and cited followed by the reference number used; either in the text or in a footnote/end-note. The details of the used references that are listed at the end of the report confirm the referencing style dictated by the final year project template and are, to my knowledge, accurate and complete.

I have read the sections on referencing and plagiarism in the final year project template. I understand that plagiarism can lead to a reduced or fail grade, in serious cases, for the Graduation Project course.

Student Name: Ahmed Adel Sayed
Gouda Ahmed

Registration Number: 211005618

Signed: _____

Student Name: Hadeel Gamal Eldin
Mohamed Ibrahim

Registration Number: 211004219

Signed: _____

Student Name: Alaa Osama Mohammed
Shehab

Registration Number: 211004234

Signed: _____

Student Name: Salah Mohamed Salah
Ibrahim

Registration Number: 211004521

Signed: _____

Date: Wednesday 8th June 2026

ACKNOWLEDGMENTS

In the name of Allah, the Most Gracious, the Most Merciful, to whom we attribute the strength and success to complete this research.

We are profoundly grateful to our supervisor, **Dr. Nada Mostafa Abdelaziem Mostafa**, for her unwavering patience, invaluable expertise, and continuous guidance. Her insights not only shaped the technical direction of this thesis but have also deeply enriched our broader understanding of computer engineering and intelligent systems.

Equally, we owe our deepest appreciation to **Dr. Reham Taher Abdelmajeed Salem Al Magraby**. Her transformative support, constant motivation, and words of wisdom were the driving force behind this project. Without her dedicated mentorship and encouragement, the realization of this work would not have been possible. Truly, she has taught us a great deal, and for that, we are immensely grateful.

We consider ourselves truly fortunate to have worked under the supervision of such exceptional educators, and we hope this thesis serves as a worthy testament to their efforts and guidance.

Furthermore, we would like to extend our collective and heartfelt thanks to our parents and families for never failing to encourage us to be the best versions of ourselves. Their endless sacrifices and their efforts in instilling the value of hard work and the importance of knowledge within us have been our greatest foundation.

Finally, a special note of gratitude to all our siblings and loved ones. Without your constant support and encouragement, we would have never been able to accomplish our goals and bring this project to life.

ABSTRACT

Sign language recognition systems can reduce communication barriers between deaf and hard-of-hearing communities and hearing users, yet most practical tools cover only one language or one recognition level. This graduation project delivers **Eshara** (إشارة), a bilingual **web application** for American Sign Language (ASL) and Arabic Sign Language (ArSL) that supports static letter finger-spelling and isolated-word recognition from a standard webcam.

The system follows a three-tier architecture: a **React** frontend, a **Node.js/Express** application layer for authentication and routing, and a **FastAPI** model-services layer for inference. Four standalone deep-learning models are trained with separate vocabularies: an ASL letter **MLP** over **78-D** engineered hand landmarks (**28** deployed classes), an ArSL letter **MLP** over **63-D** raw hand landmarks (32 classes), an ASL word **Inception I3D** model over **32-frame 224×224** RGB clips (100 WLASL glosses), and an ArSL word stacked **BiLSTM** over **225-D** normalised Holistic sequences (48 frames, 100 KArSL classes).

Under constrained compute and single-academic-year scope, verification combines held-out dataset evaluation, integration testing of the three-tier web stack, and browser latency measurement. Design constraints include limited GPU resources and compact landmark transmission instead of full video streams. ASL and ArSL letters and ASL words are active via `/predict`; ArSL words are offline-validated and pending web integration. Intended uses include assistive education and communication practice where only a browser and webcam are available.

TABLE OF CONTENTS

LIST OF FIGURES	vi
LIST OF TABLES	xi
LIST OF ACRONYMS/ABBREVIATIONS	xiii
1 INTRODUCTION	1
1.1 PROJECT OVERVIEW	1
1.2 PROBLEM STATEMENT	2
1.2.1 Current Constraints:	2
1.2.2 Technical and Practical Gap:	3
1.3 PROJECT MOTIVE	3
1.4 AIMS AND OBJECTIVES	3
1.4.1 Aims:	3
1.4.2 Objectives:	3
1.5 LITERATURE REVIEW	4
1.5.1 Existing Work:	4
1.5.2 Scholarly Views:	4
1.5.3 Challenges and Gaps:	5
1.5.4 Hypothesis:	5
1.6 METHODOLOGY	5
1.6.1 Data Acquisition:	6
1.6.2 Cleaning and Preprocessing:	6
1.7 REPORT ORGANISATION	7
2 LITERATURE REVIEW	8
2.1 HISTORY OF SIGN-LANGUAGE RECOGNITION	8
2.2 MACHINE LEARNING FOUNDATIONS FOR SLR	11
2.2.1 Supervised learning and classification:	11

2.2.2	Static versus sequence classification:	12
2.2.3	Families of neural networks in SLR:	13
2.2.4	Training, validation, and evaluation:	14
2.2.5	Isolated SLR versus continuous SLR as ML problems:	15
2.2.6	Preprocessing, normalisation, and augmentation:	16
2.3	COMPUTER VISION APPROACHES TO SLR	17
2.3.1	Image-Based Convolutional Methods:	17
2.3.2	Skeleton and Landmark Methods:	18
2.3.3	Hybrid CNN–RNN and Continuous SLR:	20
2.4	DEEP-LEARNING ARCHITECTURES	21
2.4.1	Multi-Layer Perceptrons (letter models):	21
2.4.2	Inception I3D (ASL word model):	21
2.4.3	Bidirectional LSTM (ArSL word model):	22
2.4.4	Attention and transformers (literature context):	22
2.5	POSE-ESTIMATION FRAMEWORKS	22
2.6	SIGN-LANGUAGE DATASETS	23
2.7	BILINGUAL AND ARABIC SIGN-LANGUAGE RESEARCH	26
2.8	REAL-TIME WEB DEPLOYMENT	27
2.9	RESEARCH GAP AND PROJECT POSITIONING	28
2.10	COMPARISON OF ESHARA’S FOUR MODELS	29
3	METHODOLOGY — LETTER RECOGNITION	32
3.1	HIGH-LEVEL SYSTEM ARCHITECTURE	32
3.2	LETTER DATASETS AND SPLIT POLICIES	34
3.2.1	ASL Alphabet (English Letters):	35
3.2.2	ArASL2018 (Arabic Letters):	35
3.2.3	End-to-End Training Pipeline:	35
3.3	FEATURE EXTRACTION FOR LETTERS	37
3.4	MODEL 1 — ASL LETTER MLP	43
3.5	MODEL 2 — ArSL LETTER MLP	44
3.6	LETTER STABILISATION DECODER	47
3.7	ARABIC TEXT RENDERING	48
3.8	ENGINEERING TRADE-OFFS (LETTERS)	49
3.9	EVALUATION PROTOCOL (LETTERS)	50
4	METHODOLOGY — WORD RECOGNITION	51
	PART A: ASL WORD RECOGNITION (Model 3)	52

4.1	WLASL DATASET AND SPLIT POLICY	52
4.2	ASL FEATURE EXTRACTION — RGB CLIP PIPELINE	53
4.3	MODEL 3 — ASL WORD INCEPTION I3D	55
4.3.1	Architectural Family — Inflated 3-D ConvNet:	55
4.3.2	Building Block — Unit3D:	55
4.3.3	InceptionModule — 4-Branch Parallel Block:	56
4.3.4	Full Layer Stack:	57
4.3.5	Temporal Logit Aggregation and Softmax:	58
4.3.6	Checkpoint and Parameter Count:	59
4.4	ASL WORD STABILISATION DECODER	59
4.5	ASL INFERENCE PATH (END-TO-END)	60
	PART B: ArSL WORD RECOGNITION (Model 4)	61
4.6	KArSL DATASET AND SPLIT POLICY	61
4.7	ArSL FEATURE EXTRACTION — 225-D HOLISTIC PIPELINE	62
4.7.1	225-D Feature Vector Composition:	62
4.7.2	Body-Relative Normalisation:	64
4.7.3	Clip Alignment (Trim / Pad):	64
4.8	MODEL 4 — ArSL WORD STACKED BiLSTM	66
4.8.1	LSTM Cell Equations:	66
4.8.2	Bidirectional LSTM:	67
4.8.4	Parameter Count:	69
4.8.5	Loss and Training:	70
4.9	ArSL INFERENCE PATH (END-TO-END)	70
4.10	CROSS-LANGUAGE PIPELINE COMPARISON	71
4.11	ENGINEERING TRADE-OFFS	73
4.12	EVALUATION PROTOCOL (WORDS)	74
5	SYSTEM IMPLEMENTATION	76
5.1	SYSTEM ARCHITECTURE	76
5.2	FRONTEND DESIGN	78
5.2.1	State and component architecture:	78
5.2.2	Styling and theming:	78
5.2.3	Education module interface:	79
5.2.4	Webcam capture and layout:	79
5.3	BACKEND INTEGRATION AND AUTHENTICATION	79
5.3.1	Core application server:	79

5.3.2	Authentication infrastructure:	79
5.3.3	Real-time communication (Socket.IO):	81
5.4	DATABASE DESIGN	81
5.4.1	Prisma ORM and SQLite:	81
5.4.2	Chat and message models:	82
5.5	TRANSLATION WORKFLOW AND AI MODEL COMMUNICATION	84
5.5.1	Image capture and encoding:	84
5.5.2	Cross-service communication:	84
5.5.3	Model inference pipeline:	85
5.6	ASL WORD MODEL INTEGRATION (WLASL / INCEPTION I3D)	85
5.7	NAVIGATION FLOW AND MAIN PAGES	86
5.8	FEATURES, FUNCTIONALITY, AND UX DESIGN	89
5.9	SECURITY CONSIDERATIONS	89
5.10	DEPLOYMENT	90
5.11	ERROR HANDLING AND TESTING	91
5.11.1	Graceful degradation:	91
5.11.2	Testing methodology:	91
5.12	REAL-TIME WEB PERFORMANCE	91
5.13	INFORMAL USABILITY PILOT	92
5.14	EDUCATION MODULE: BILINGUAL SIGN-LANGUAGE LEARNING	94
5.14.1	Overview and layout:	94
5.14.2	Motivation: from hand-authored keyframes to motion capture:	95
5.14.3	Content pipeline: dictionary signs from sign.mt:	96
5.14.4	Server-side animation stitching (pose_concat.py):	96
5.14.5	Sign Viewer:	97
5.14.6	Sentence Builder — free-text signing:	99
5.14.7	Arabic support (ArSL):	101
5.14.8	Content verification — defeating the silent fallback:	102
5.14.9	Practice mode — learn by doing:	103
5.14.10	Performance and verification engineering:	104
6	RESULTS AND EVALUATION	108
6.0	TRAINING CONDITIONS AND EXPERIMENTAL CHALLENGES	108
6.1	LETTER RECOGNITION RESULTS	110
6.1.1	ASL Letter MLP:	110
6.1.2	ArSL Letter MLP:	111

6.2	WORD RECOGNITION RESULTS	113
6.2.1	ASL Word Inception I3D (deployed):	113
6.2.2	ArSL Word Stacked BiLSTM (deployed):	114
6.2.3	Metrics source inventory	118
6.3	COMPARATIVE ANALYSIS	119
6.4	ABLATION AND DESIGN STUDIES	120
6.5	ERROR ANALYSIS	121
6.6	USABILITY EVALUATION	122
7	DISCUSSION	123
7.1	INTERPRETATION OF RESULTS	123
7.2	STRENGTHS OF THE APPROACH	124
7.3	LIMITATIONS AND FAILURE CASES	124
7.4	COMPARISON WITH EXISTING WORK	128
7.5	ENGINEERING TRADE-OFFS REVISITED	128
7.6	ETHICAL AND SOCIAL CONSIDERATIONS	129
8	CONCLUSION	130
8.1	SUMMARY OF ACHIEVEMENTS	130
8.2	CONTRIBUTIONS	131
8.3	PRACTICAL IMPACT	131
9	FUTURE WORK	132
9.1	CONTINUOUS SIGN-LANGUAGE RECOGNITION AND TRANSLATION	132
9.2	EXPANDED ARABIC VOCABULARY AND DIALECT COVERAGE	132
9.3	IMPROVED REAL-TIME ROBUSTNESS	132
9.4	MOBILE AND EDGE DEPLOYMENT	133
9.5	BIDIRECTIONAL COMMUNICATION	133
9.6	FORMAL USER STUDIES AND COMMUNITY PARTNERSHIP	133
	REFERENCES	134

LIST OF FIGURES

2-1	Evolution of sign-language recognition from sensor gloves and HMM-based RGB pipelines to landmark-based deep learning on commodity webcams. . . .	10
2-2	Supervised machine-learning paradigms mapped to Eshara’s four models (cf. Table 2-2): static MLPs for letters, Inception I3D for ASL words, and a stacked BiLSTM for ArSL words.	13
2-3	Isolated versus continuous sign-language recognition as machine-learning problem formulations. Eshara implements isolated classification only; continuous gloss decoding is deferred to Chapter 9.	16
2-4	MediaPipe hand landmarks (63-D per frame) and extension to normalised pose+hands feature vectors used by the ArSL word model (225-D). The ASL word model (I3D) consumes raw RGB frames rather than landmarks.	20
3-1	Letter-mode client–server pipeline. The browser captures a webcam frame as a base64-encoded JPEG and sends it via HTTP POST to the Node.js application tier; the gateway proxies the request to FastAPI, where MediaPipe Hands extracts landmarks and engineers a 63-D (ArSL) or 78-D (ASL) vector before the letter MLP and stabilisation decoder commit a character.	33
3-2	MediaPipe Hand landmark topology (21 points). Each landmark contributes (x,y,z) coordinates; flattening yields a 63-D raw vector.	38
3-3	Letter feature-engineering split. ArSL (left): MediaPipe Hands $\rightarrow$ flatten $\rightarrow$ 63-D raw MLP (32 classes). ASL (right): additional wrist-relative normalisation and 15 joint angles $\rightarrow$ 78-D engineered MLP (28 deployed classes).	39
3-4	End-to-end letter training pipeline. Stages 1–6 are shared; language-specific forks appear at feature engineering (78-D engineered vs. 63-D raw) and at MLP head width, dropout, and training schedule.	42

3-5	Shared MLP <i>family</i> for both letter models (Dense–BN–ReLU–Dropout–Softmax). ASL Model 1 uses 78-D input, widths 256/128/64, BN on all three hidden layers, and progressive dropout (0.3 / 0.25 / 0.2). ArSL Model 2 uses 63-D input, wider layers 512/256/64, BN on layers 1–2 only, and uniform dropout 0.2 (Table 3-5).	44
3-6	Side-by-side comparison of the two letter MLP heads. Both follow Dense–BN–ReLU–Dropout blocks ending in softmax, but Model 2 uses a wider hidden stack (512/256/64), uniform dropout, and BN on layers 1–2 only.	47
3-7	Letter stabilisation finite-state machine. A rolling buffer of 10 frames requires ≥ 7 agreeing labels before Commit ; a 0.8 s hold prevents duplicate characters. .	48
3-8	Arabic UI rendering pipeline. Romanised model labels are mapped to Unicode, reshaped for cursive joining, reordered RTL with python-bidi, and drawn onto the webcam overlay (example: bb $\rightarrow$ ba glyph).	49
4-1	ASL word RGB clip preprocessing (Model 3). Each frame passes through colour conversion, aspect-preserving resize (short side 256 px), centre crop 224×224 , $[-1, 1]$ normalisation, and (C, H, W) transpose before accumulation in a 32-frame deque (Table 4-3, Equations (20)–(23)).	54
4-2	InceptionModule 4-branch parallel structure (Equations (27)–(31)). All branches process the same input A simultaneously; outputs are channel-concatenated. Example for Mixed_3b: $[64, 96, 128, 16, 32, 32] \rightarrow 64 + 128 + 32 + 32 = 256$ output channels.	56
4-3	Inception I3D forward pass (Model 3). The $(1, 3, 32, 224, 224)$ clip passes through the stem, eight Inception modules (Table 4-4), AvgPool, dropout, and a 100-way logit head; logits are temporally averaged before softmax (Equations (32)–(33)).	58
4-4	ASL word stabilisation decoder (Model 3). Three gates — majority vote (6/10), confidence ($\geq 50\%$), and cooldown (≥ 1.2 s) — must pass before a gloss is committed (Table 4-5).	60
4-5	225-D Holistic feature topology for ArSL words (Model 4). Pose (99-D) is nose-centred; each hand block (63-D) is wrist-centred. No face mesh is included (Table 4-7).	63
4-6	ArSL word Holistic extraction pipeline (Model 4). Landmarks are normalised per frame, buffered to 48 frames, aligned via trim/pad, and classified by the stacked BiLSTM when has_hand is true (Sections 4.7–4.9).	64

4-7	Trim/pad clip-alignment policy (Equation (37)). Clips longer than 48 frames are truncated to the first 48; shorter clips are extended by repeating the final body-pose frame until 48 frames are reached. Both cases produce $\mathbf{X}' \in \mathbb{R}^{48 \times 225}$.	65
4-8	LSTM cell internal structure (Equations (38)–(43)). The blue cell-state highway $\mathbf{c}_{t-1} \rightarrow \mathbf{c}_t$ is gated by the forget gate $\mathbf{f}_t$ (retention) and the input gate $\mathbf{i}_t$ plus candidate $\tilde{\mathbf{c}}_t$ (new content). The output gate $\mathbf{o}_t$ controls how much of $\tanh(\mathbf{c}_t)$ is exposed as the hidden state $\mathbf{h}_t$.	67
4-9	ArSL stacked BiLSTM architecture with per-layer output shapes and parameter counts (Table 4-8). BiLSTM Layer 1 (148 480 params) returns a per-frame sequence (48, 128); Layer 2 (98 816 params) collapses it to a single 128-D signing summary; Dense(32)+ReLU and Dense(100)+Softmax (7 428 params total) map to 100 Arabic word class probabilities. Total: ~ 255 K parameters.	68
4-10	Side-by-side word inference pipelines for ASL (Model 3, Inception I3D, left) and ArSL (Model 4, Stacked BiLSTM, right). Both pipelines share a rolling-deque temporal buffer and a 100-class softmax output, but differ in feature source (RGB pixels vs. Holistic landmarks), buffer depth (32 vs. 48 frames), model architecture, parameter count, and stabilisation strategy. See Table 4-10 for a detailed comparison.	72
5-1	Eshara three-tier system architecture: React/Vite client, Node.js + Express application server (Prisma ORM over SQLite, Socket.IO server), and the FastAPI model-services tier hosting the ASL/ArSL letter and word models.	77
5-2	Component diagram: frontend modules (AppHomePage, stickman/pose-viewer visualisers, AuthContext/ThemeContext, Axios service layer) and their dependencies on backend components (chat socket handlers, auth router, predict router, Prisma client, and the external FastAPI target).	78
5-3	Entity–Relationship Diagram (ERD): User, Room, ROOM_MEMBER, and MESSAGE tables with primary/foreign keys and cascading relationships, as defined in <code>schema.prisma</code> .	82
5-4	Class diagram of the application-server domain objects (User, Room, RoomMember, Message) and their principal operations (<code>register()</code> , <code>login()</code> , <code>createRoom()</code> , <code>joinRoom()</code> , <code>sendMessage()</code>).	83
5-5	Sequence diagram of the end-to-end translation request: React client $\rightarrow$ Node.js API (JWT validation) $\rightarrow$ FastAPI model service (MediaPipe landmark extraction, model inference) $\rightarrow$ JSON response relayed back to the client.	84

5-6	Use case diagram: authentication (register/login), the Educational module (browse and watch word signs, fingerspell typed text, sign a typed sentence, practise signs with webcam feedback), and Translation & Chat (capture webcam feed, request AI translation, join chat room, send translation to live chat).	87
5-7	Eshara Landing Page: logo, tagline, Login/Register toggle, dark-mode switch, and credential form.	88
5-8	Eshara App Home Page (Chatting tab): language/mode/camera-device selectors, webcam capture panel with Start/End Camera and Translate Sign controls, and the Live Chat panel with an active Socket.IO room.	88
5-9	Deployment diagram: client device (browser, React SPA) communicating over HTTPS/REST and WSS/WebSockets with the application server (Node.js + Express, Socket.IO, SQLite), which forwards inference requests over an internal HTTP proxy to the isolated ML inference environment (FastAPI service, ASL/ArSL deep-learning models).	90
5-10	Eshara Education tab (Sign Viewer, ASL): motion-capture skeleton playback with categorised vocabulary and playback-speed controls.	95
5-11	Sign Viewer (ArSL): Jordanian Sign Language skeleton playback with right-to-left category and word controls.	98
5-12	Fingerspelling: a typed word plays as one server-stitched sequence while the current letter is highlighted from timing metadata.	99
5-13	Sentence Builder: dictionary words render as solid chips and are signed; unknown words render as dashed chips and are fingerspelled, with live highlighting driven by timing metadata.	101
5-14	Practice mode: the learner performs a prompted sign; the same recognition models that power live translation check the attempt and give constructive feedback.	104
6-1	ASL letter MLP row-normalised confusion matrix (78-D engineered features, held-out test $N=12401$, 28 classes). Reproduced from held-out test evaluation.	111
6-2	ArSL letter MLP row-normalised confusion matrix (63-D raw hand landmarks, held-out test $N=14734$, 32 classes). Reproduced from held-out test evaluation.	112
6-3	ASL word Inception I3D — WLASL-100 benchmark accuracies (Li et al., 2020; deployed checkpoint family).	114
6-4	ArSL word BiLSTM evaluation summary: held-out test Top-1 99.62 %, best validation 99.72 %, converged loss 0.026 on the 100-class KArSL subset (SignID 71–170).	117

6-5	ArSL word BiLSTM per-class confusion structure on the KArSL held-out test subset (representative SignID panels; $N=2\,400$ total).	118
-----	--	-----

LIST OF TABLES

2-1	Interpretation of SLR technological eras (aligned with Figure 2-1).	10
2-2	Machine-learning paradigms used in Eshara.	12
2-3	Evaluation metrics in SLR literature and in Eshara.	15
2-4	Vision pipelines considered during Eshara development.	18
2-5	Feature-vector composition used by each Eshara model.	19
2-6	Comparison of pose-estimation frameworks.	23
2-7	Sign-language datasets used or referenced in this work.	24
2-8	Approximate class balance across Eshara training corpora.	24
2-9	Structural comparison with representative SLR research lines (not a claim of state-of-the-art rank).	28
2-10	Research gaps addressed by Eshara.	29
2-11	Architecture comparison of Eshara’s four graduation models.	30
3-1	Letter datasets, feature shapes, and split policies.	34
3-2	Training and inference software environment (letters and words).	34
3-3	The 15 inter-joint angle triplets used in the 78-D ASL feature vector. Each row specifies the landmark index triple (a, b, c) where b is the vertex joint; the angle is computed between bone vectors $b \rightarrow a$ and $b \rightarrow c$	40
3-4	Feature-vector dimension breakdown for the two letter models.	41
3-5	Letter MLP architecture comparison. Both models use the same MLP <i>family</i> but separate weights and tuned configurations.	46
3-6	Engineering trade-offs for letter recognition in Eshara.	49
4-1	Chapter 4 at a glance: side-by-side comparison of both word recognition pipelines.	51
4-2	WLASL dataset statistics for the Model 3 (ASL word) graduation deployment.	52
4-3	ASL RGB clip preprocessing steps (per frame, Model 3).	53
4-4	Inception I3D layer stack (Model 3, ASL words). Branch configs for Inception stages: $[n_0, n_{1a}, n_{1b}, n_{2a}, n_{2b}, n_{3b}]$ per Equation (31). Input: $(1, 3, 32, 224, 224)$	57
4-5	ASL word stabilisation parameters (Model 3).	59

4-6	KArSL dataset statistics for the Model 4 (ArSL word) graduation deployment. .	61
4-7	225-D Holistic feature-vector block decomposition (ArSL word model). Missing detections are zero-filled.	62
4-8	ArSL word BiLSTM full architecture (Model 4).	68
4-9	Model 4 parameter breakdown. $d_1=225$ (input dim), $h=64$ (LSTM units per direction), $d_2=128$ (BiLSTM 1 output = $2h$).	69
4-10	Detailed cross-language comparison of ASL and ArSL word recognition pipelines.	71
4-11	Deployment artifacts for word recognition (Models 3 and 4).	72
4-12	Engineering trade-offs for ASL word recognition (Model 3).	73
4-13	Engineering trade-offs for ArSL word recognition (Model 4).	74
4-14	Word-model evaluation protocol (reported in Chapter 6).	75
5-1	Real-time web performance (pilot measurements, graduation dev hardware). . .	92
5-2	Informal usability pilot observations ($N=3$, React web UI).	93
5-3	Education module: superseded Canvas2D engine versus the deployed motion-capture pipeline.	105
6-1	Recurring training and evaluation challenges (project experiments).	109
6-2	Letter-model held-out test metrics (canonical training pipelines).	110
6-3	ASL word model metrics — deployed Inception I3D (<code>asl_word_i3d.pt</code>), WLASL-100 benchmark protocol.	113
6-4	Deployed ArSL word-model metrics (100 classes, SignID 71–170, KArSL held-out test).	115
6-5	ArSL word BiLSTM training configuration (KArSL 100-class subset).	115
6-6	ArSL word deployment verification (bundle batch evaluation).	116
6-7	Metrics source inventory (Chapter 6 sources; ASL words per WLASL-100 benchmark).	119
6-8	Eshara four-model summary.	120
6-9	Documented design studies (qualitative / partial quantitative).	121
7-1	Superseded word-recognition experiment branches (not deployed in Eshara). . .	127
8-1	Graduation deliverables and headline results.	130

LIST OF ACRONYMS/ABBREVIATIONS

ACRONYM	Definition of Acronym
API	Application Programming Interface
ArSL	Arabic Sign Language
ASL	American Sign Language
BiLSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
CSLR	Continuous Sign Language Recognition
FPS	Frames Per Second
GPU	Graphics Processing Unit
I3D	Inflated 3D ConvNet
JWT	JSON Web Token
KArSL	King Saud University Arabic Sign Language dataset
MLP	Multilayer Perceptron
REST	Representational State Transfer
RTL	Right-to-Left
SLR	Sign Language Recognition
SUS	System Usability Scale
UI	User Interface
WLASL	Word-Level American Sign Language

Chapter One

1 INTRODUCTION

1.1 PROJECT OVERVIEW

This graduation project presents **Eshara** (إشارة — Arabic for “sign”), a real-time bilingual sign-language recognition system that translates hand gestures from a standard webcam into text through a web application. The system supports American Sign Language (ASL) and Arabic Sign Language (ArSL) at two recognition levels: static letters (finger-spelling) and isolated words signed across a one-to-two second window.

Eshara is built as a full-stack web product rather than a laboratory prototype alone. Four deep-learning models — two Multi-Layer Perceptrons for letter recognition, an Inception I3D network for ASL word recognition, and a BiLSTM for ArSL word recognition — are designed for a unified web inference pipeline. The deployed stack follows a three-tier path (React client → Node.js/Express API → FastAPI model-services), with webcam frames captured in the browser, authenticated at the Node layer, and forwarded internally for inference. Input comes from a standard webcam only — no additional sensors or specialised capture equipment. Letter and ArSL word paths use compact landmark representations (**78-D** engineered features for ASL letters, **63-D** raw hand landmarks for ArSL letters, **225-D** normalised pose-and-hands sequences per frame for ArSL words). The ASL word model operates on **32-frame** clips of centre-cropped **224×224** RGB video.

The four models are: (1) an ASL letter MLP over **28 deployed classes** (78-D input; the source ASL Alphabet corpus contains 29 labels including nothing, which is excluded from the web

vocabulary), (2) an ArSL letter MLP over 32 classes (63-D input), (3) an ASL word Inception I3D over 100 WLASL glosses, and (4) an ArSL word BiLSTM over 100 Arabic sign classes ($225\text{-D} \times 48$ frames). Each model was trained and evaluated with Top-1, Top-5, and macro-F1 metrics where applicable. Word methodology is in Chapter 4; experimental results in Chapter 6; full web implementation is in Chapter 5.

1.2 PROBLEM STATEMENT

Approximately 466 million people worldwide live with disabling hearing loss, including around seven million in the Arab region [1]. For most deaf and hard-of-hearing individuals, sign language — not spoken language — is the primary means of communication. Human sign-language interpreters are effective but costly, limited in availability, and cannot be present in every classroom, clinic, or customer-service interaction. Automated recognition offers a scalable alternative, yet most existing tools and research efforts focus on American Sign Language alone, or on letter recognition alone, and rarely combine both languages and both recognition levels in a single deployable web-based product [2, 3].

The engineering problem addressed in this thesis is stated formally as follows:

Given a continuous RGB video stream from a single uncalibrated camera, automatically recognise either (a) the letter being finger-spelled per frame, or (b) the isolated word being signed across a 1–2 second window, in either ASL or ArSL, with end-to-end latency below 200 ms, delivered through a standard web browser.

1.2.1 Current Constraints:

The project was carried out under several practical limits. Training deep recognition models proved time-consuming: feature extraction over tens of thousands of images or video clips required multi-hour (often overnight) CPU/GPU runs, and several model families were trained **multiple times** after failed webcam trials exposed domain gaps (lighting, background clutter, MediaPipe dropout). The available local GPU (4 GB VRAM) restricted batch size and forced word-model extraction and I3D experiments onto Kaggle when CUDA memory was exhausted. The full system had to be designed, trained, and evaluated within a single academic year. The solution was required to run in a standard web browser using only a laptop or desktop webcam. For privacy on letter and ArSL word paths, only numeric landmark vectors are sent over the network rather than continuous raw video; ASL word mode instead transmits encoded frame buffers to the server for I3D inference. Software versions and CUDA usage are tabulated as defined in Table 3-2 (Chapter 3).

1.2.2 Technical and Practical Gap:

Several technical and practical gaps motivated this work. Most existing approaches handle either letter recognition or word recognition, but not both within one real-time pipeline. Few systems support more than one sign language in the same product, and many published solutions stop at offline accuracy without a live web interface. Running recognition in the browser is also demanding: inference must stay fast enough for interactive use, yet deep sequence and video models are costly to train and tune under limited compute. Human interpretation remains the default in practice because scalable, low-latency assistive tools are still uncommon. Eshara was designed to address these gaps through a bilingual, dual-level web application accessible from any modern browser.

1.3 PROJECT MOTIVE

The primary motive of this project is to reduce the communication barrier between deaf signers and hearing non-signers in the Arab region and beyond. Arabic sign-language users currently lack the same assistive tooling available to ASL communities. Beyond the social impact, the project demonstrates that specialised models matched to each task — compact MLPs for letters, a landmark BiLSTM for ArSL words, and a pretrained 3D convolutional network for ASL words — can achieve strong accuracy within a deployable web application. The final deliverable is intended as a working web product suitable for education, healthcare, customer service, and everyday communication, not merely as an offline accuracy benchmark. The four-model design reflects a deliberate engineering choice: ASL and ArSL have entirely separate vocabularies and feature spaces, so four specialised pipelines — each tuned to its own input representation and label set — prove more tractable and more accurate than a single shared multilingual model behind one web interface.

1.4 AIMS AND OBJECTIVES

1.4.1 Aims:

The aim of this project is to design, train, evaluate, and deploy a real-time bilingual ASL/ArSL recognition system that covers both letter and word recognition and is accessible through a web application.

1.4.2 Objectives:

To achieve this aim, six specific objectives were defined. First, four model pipelines were built covering ASL letters, ArSL letters, ASL words, and ArSL words. Second, features were

prepared per model: MediaPipe Hand landmarks for letters (**78-D** engineered for ASL, **63-D** raw for ArSL), a **32-frame** RGB clip pipeline for ASL words (I3D), and **225-D** normalised pose-and-hands sequences over 48 frames for ArSL words. Third, four deep-learning models were trained or integrated: an ASL letter MLP (28 deployed classes), an ArSL letter MLP (32 classes), an ASL word Inception I3D (100 WLASL classes), and an ArSL word BiLSTM (100 classes). Fourth, a three-tier backend path was implemented (Node.js/Express gateway with internal FastAPI model services). Fifth, a React web client was built with a live recognition interface and authenticated translation flow. Sixth, each model was evaluated using Top-1 accuracy, Top-5 accuracy, and macro-F1 on held-out test data; end-to-end latency was measured in the React web application (Chapter 5, Section 5.12).

The system deliberately excludes continuous sign-language translation, grammatical parsing, sign languages other than ASL and ArSL, multi-signer scene analysis, native mobile applications, and native depth-camera hardware. These areas are identified as future work in Chapter 9.

1.5 LITERATURE REVIEW

The following is a summary of the field. A full literature review is presented in Chapter 2.

1.5.1 Existing Work:

Sign-language recognition has progressed from early RGB-camera and Hidden Markov Model approaches to deep convolutional and recurrent networks, and more recently to webcam-based landmark pipelines using tools such as MediaPipe [4, 5, 2]. This project trains or integrates four models aligned to four recognition tasks: ASL letters (28 deployed classes from a 29-label corpus), ArSL letters (32 classes), ASL words (100 WLASL classes via I3D) [6], and ArSL words (100 classes) [7]. Prior published systems often address only one of these tasks or one language at a time, and rarely deliver all four through a single live web application.

1.5.2 Scholarly Views:

Researchers broadly agree that skeleton and landmark representations require less training data than raw-pixel approaches for many SLR tasks, that 3D convolutional networks capture spatio-temporal motion effectively from video clips, and that sequence models such as BiLSTMs and related temporal encoders outperform per-frame classifiers for landmark-based word recognition. Browser-based deployment is widely recognised as important for practical assistive technology, yet a unified bilingual web product covering letters and words in both ASL and ArSL remains uncommon in the literature.

1.5.3 Challenges and Gaps:

Several persistent challenges motivate this project. Letter models must respond per frame with low latency, while word models must capture motion across a short temporal window — serving both from one web pipeline is non-trivial. Bilingual coverage (ASL and ArSL) and dual granularity (letters and words) multiply the engineering and training effort because each of the four models requires its own architecture and evaluation. Real-time performance demands sustained inference on consumer hardware inside a web browser. Most academic contributions stop at offline accuracy for a single model rather than a unified, deployable four-model web system.

1.5.4 Hypothesis:

The central hypothesis of this work is that a webcam-driven pipeline with four specialised deep-learning models — one per language and recognition level — deployed in a unified web application can achieve reliable recognition accuracy across all four tasks and end-to-end inference latency of no more than 200 ms in a standard browser. On held-out test data, the letter models reach **99.06 %** (ASL) and **99.63 %** (ArSL) Top-1 accuracy; the ArSL word model reaches **99.62 %** Top-1 on 100 classes; the ASL word I3D checkpoint reaches **65.89 %** Top-1 on the WLASL-100 benchmark (Chapter 6). Full experimental detail for all four models appears in Chapter 6.

1.6 METHODOLOGY

The following is a high-level overview. Full methodological detail is presented in Chapters 3 (letters) and 4 (words).

1.6.1 Data Acquisition:

Four datasets underpin the four models. **Letters:** the Kaggle ASL Alphabet corpus (~87 000 images, 29 classes) [8] and ArSL2018 (54 049 images, 32 classes) [9] were converted offline to MediaPipe Hand landmarks; ASL samples use 78-D engineered vectors and ArSL samples use 63-D raw vectors (Chapter 3). **Words:** ASL word recognition uses a 100-class WLASL subset with 32-frame RGB clips for I3D inference [6]; ArSL words use the KArSL 100-class word subset (SignID 71–170; 48-frame, 225-D pose-and-hands clips) as described in Chapter 4 [7]. Extracted letter features and ArSL word sequences are cached as compressed NumPy archives. Letter models use stratified train/validation/test splits; the ArSL word model is evaluated on held-out KArSL clips (SignID 71–170). Trained checkpoints are exported for the web inference service (Chapter 5).

1.6.2 Cleaning and Preprocessing:

For **letters**, static images or live frames pass through MediaPipe Hands; ASL applies wrist-relative normalisation and 15 joint angles before MLP input; ArSL uses raw 63-D coordinates. For **ASL words**, webcam frames are resized, centre-cropped to 224×224 , and normalised to $[-1, 1]$ before buffering 32 frames for I3D input. For **ArSL words**, Holistic landmarks are nose- and wrist-normalised to a 225-D body-relative representation per frame, then each clip is trimmed or padded to 48 frames (Chapter 4). Clips with missing labels or failed extraction are discarded.

Four models were trained or integrated on the processed data. The ASL letter model is a three-hidden-layer MLP ($256 \rightarrow 128 \rightarrow 64 \rightarrow 28$ softmax) over 78-D input (A–Z, space, del; nothing excluded from deployment). The ArSL letter model uses the same **MLP family** but a wider head ($512 \rightarrow 256 \rightarrow 64 \rightarrow 32$ softmax) over 63-D raw input (as compared in Table 3-5, Chapter 3). The ASL word model is a pretrained **Inception I3D** (Inflated 3D ConvNet) with a 100-way WLASL classification head. The ArSL word model is a stacked BiLSTM network over $225\text{-D} \times 48$ frames and 100 classes. The full three-tier deployment is described in Chapter 5.

1.7 REPORT ORGANISATION

The remainder of this report is organised as follows. Chapter 2 presents the literature review, tracing sign-language recognition from early Hidden Markov Model and RGB-camera systems through deep convolutional and recurrent networks to modern hybrid landmark- and video-based pipelines, and identifying the gap that motivates this work. Chapter 3 details the letter-recognition methodology for both ASL and ArSL, covering dataset preparation, landmark feature engineering, MLP architecture, and training protocol. Chapter 4 presents the word-recognition methodology: the Inception I3D pipeline for ASL words and the BiLSTM pipeline for ArSL words, including feature extraction and model-side inference protocols.

Chapter 5 describes the full system implementation — React frontend, Node.js/Express application tier, FastAPI model-services tier, live web recognition interface, and real-time latency measurements. Chapter 6 reports experimental results for all four models, including Top-1 and Top-5 accuracy, macro-F1, and ablation evidence. Chapter 7 discusses and interprets those results in the context of prior work, limitations, and the project hypothesis. Chapter 8 states the conclusions. Chapter 9 identifies directions for future work including continuous sign translation, extended vocabulary, and multilingual expansion.

Chapter Two

2 LITERATURE REVIEW

Chapter 1 stated the problem, objectives, and scope of Eshara: a **web-only**, webcam-driven bilingual sign-language system with four standalone recognition models. This chapter reviews the literature that motivates those design choices. The review proceeds in a deliberate funnel: technological history of SLR (Section 2.1); machine-learning foundations (Section 2.2); computer-vision input representations (Section 2.3); deep-learning architectures (Section 2.4); pose-estimation frameworks (Section 2.5); datasets (Section 2.6); bilingual and Arabic research (Section 2.7); real-time web deployment (Section 2.8); the research gap Eshara addresses (Section 2.9); and a consolidated comparison of the four graduation models (Section 2.10). Chapters 3 and 4 implement the protocols justified here; Chapter 6 reports offline accuracy and Chapter 5 reports live web performance.

2.1 HISTORY OF SIGN-LANGUAGE RECOGNITION

Three terms recur throughout this thesis and the SLR literature. **Sign-language recognition (SLR)** maps signing video or pose features to discrete **gloss** labels (individual signs). **Sign-language translation (SLT)** maps entire signing utterances to spoken-language sentences, often without an explicit gloss intermediate [10]. **Isolated** SLR classifies one sign per clip; **continuous** SLR or CSLR decodes unsegmented video into gloss sequences [11, 12]. Eshara implements isolated SLR only: the user deliberately signs one letter or one word at a time. Continuous translation is acknowledged as future work in Chapter 9.

Sign-language recognition (SLR) has evolved through three broad technological phases, each driven by hardware availability and the maturation of machine-learning methods [2, 13, 3]. In the 1990s and early 2000s, researchers relied on instrumented gloves, accelerometers, and magnetic trackers that measured joint angles directly. Starner et al. demonstrated real-time American Sign Language recognition from wearable and desk-mounted cameras paired with specialised sensing equipment [14]. These approaches achieved usable accuracy in controlled settings but required expensive hardware and were unsuitable for everyday communication by deaf and hard-of-hearing users, who number in the hundreds of millions worldwide [1]. The fundamental lesson from this era is that high-quality sensing does not guarantee deployability: assistive tools must work with hardware users already own.

From the mid-2000s onward, commodity RGB cameras enabled vision-based systems that tracked hand shape and motion without worn sensors. Hidden Markov Models (HMMs) treated signing as a sequence of discrete visual states and remained the dominant statistical framework for more than a decade [15, 16, 17]. HMMs are effective when hand segmentation is reliable and vocabularies are small, but they struggle with high intra-class variation in unconstrained webcam video. The introduction of depth cameras such as Microsoft Kinect around 2010 enabled skeleton joints to be recovered without gloves; however, depth hardware never achieved universal adoption in laptops and phones, limiting long-term impact for consumer-facing assistive products [2].

Convolutional neural networks (CNNs) later replaced hand-crafted features for many SLR pipelines, learning spatial filters directly from pixels or optical-flow stacks [18, 17, 19]. CNN-based isolated sign classifiers routinely exceed 90 % accuracy on curated benchmarks, yet still exhibit sensitivity to background clutter, lighting variation, signer skin tone, and camera resolution. Koller et al. advanced *continuous* sign recognition through hybrid CNN–HMM architectures trained on broadcast footage, establishing a research line that later extended to weakly supervised sequence learning when frame-level gloss labels are incomplete [11]. This distinction between *isolated* sign classification (one sign per clip) and *continuous* recognition (unsegmented sentences) is central to Eshara: the graduation system solves isolated letter and word recognition; sentence-level gloss alignment is deferred to future work [12, 20].

A third phase began with robust single-camera pose estimation. OpenPose demonstrated that body keypoints could be recovered in real time from monocular video [21], and Google’s MediaPipe extended this to efficient hand and body landmark extraction on consumer hardware [4, 22]. Recent sign-language systems increasingly operate on skeleton or landmark feature vectors rather than raw pixels, reducing data requirements and enabling browser-side extraction [3]. Parallel research on continuous sign-language recognition and sign-language translation treats entire sentences as sequences of glosses, typically using hybrid convolutional-recurrent or transformer encoders [10, 12]. The present project, Eshara, sits in a **hybrid isolated-sign paradigm**: letters and ArSL words use MediaPipe landmarks; ASL words use buffered RGB clips. The system recognises individual letters and short isolated word signs from a standard webcam, and leaves full continuous translation to Chapter 9.

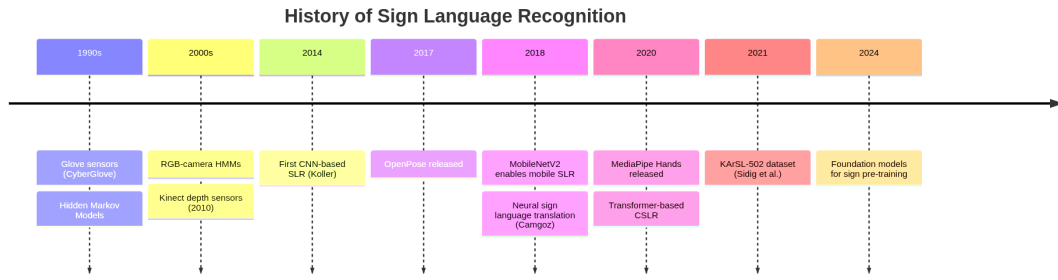


Figure 2-1: Evolution of sign-language recognition from sensor gloves and HMM-based RGB pipelines to landmark-based deep learning on commodity webcams.

As summarised in Figure 2-1, the timeline captures the milestones above. The following table interprets the timeline in terms of *what the system measures*, *what model family dominated*, and *whether ordinary users could deploy it* — three questions that directly motivated Eshara’s design.

Table 2-1: Interpretation of SLR technological eras (aligned with Figure 2-1).

Period	Input to classifier	Dominant models	Deployability	Relevance to Eshara
1990s	Glove joint angles	HMM, template matching	Low (wearable hardware)	Historical only
2000s–2010	RGB pixels, depth maps	HMM, early CNN	Medium (lab cameras)	Motivated vision path
2014–2017	CNN features from video	CNN, CNN+HMM	Medium–high	Informed CNN trials
2017–2020	Body/hand keypoints	CNN+RNN, attention	Rising	OpenPose/MediaPipe era
2020–2024	Landmark vectors	BiLSTM, Transformer	High (webcam + browser)	Eshara’s era

Reading the timeline milestone by milestone. In the **1990s**, CyberGlove-style sensors proved that sign patterns could be recognised automatically, but users had to wear equipment. Researchers therefore sought camera-only solutions. Through the **2000s**, standard webcams and

HMMs allowed finger-spelling experiments without gloves; accuracy was acceptable when backgrounds were controlled, but models did not generalise to casual home environments [16]. **Kinect (2010)** added depth maps and skeleton joints without gloves, yet depth cameras were not built into most laptops, so the approach did not become a universal assistive platform [2].

2014 marks wider adoption of CNNs for hand-shape recognition (Koller et al.) — networks learn their own visual features instead of hand-crafted descriptors [17]. **2017** brought OpenPose, showing that 2-D body pose could be recovered from a single RGB stream in real time [21]. **2018** added two parallel threads: efficient CNNs such as MobileNetV2 for resource-constrained devices [23], and neural sign-language translation systems that map entire signing sequences to spoken-language sentences [10]. **2020** is a turning point for deployable SLR: MediaPipe Hands made robust hand tracking available on consumer hardware and in the browser [4, 5], while transformer-based models advanced continuous sign recognition on large corpora [24, 11].

2021 saw the release of KArSL-502, the first large-scale Arabic sign-language video benchmark used in this thesis [7]. **2024** research explores foundation-model pre-training on massive video corpora; such models are promising but require compute and data beyond graduation scope [3]. Eshara deliberately occupies the **2020–2024 deployable-webcam band**: a webcam feeds MediaPipe landmarks for letters and ArSL words, or buffered RGB clips for ASL words; lightweight supervised classifiers (MLP, BiLSTM, or I3D) map inputs to text; the user signs *one isolated letter or word at a time*. Continuous sentence recognition (How2Sign, RWTH-PHOENIX) is acknowledged in the literature but deferred to Chapter 9 [12, 20].

2.2 MACHINE LEARNING FOUNDATIONS FOR SLR

This section introduces the machine-learning concepts used throughout Eshara so that readers without a deep-learning background can follow Chapters 3–5. Notation is kept light: formal equations, tensor dimensions, and training objectives appear in Chapter 3 (letter models) and Chapter 4 (word models). Sign-language recognition in this project is cast entirely as **supervised learning**: the system learns from labelled examples (images or videos with known sign classes) and predicts a class label for new inputs at inference time [2].

2.2.1 Supervised learning and classification:

In supervised learning, each training sample is a pair (x_i, y_i) where x_i is a feature representation (e.g. a 63-D landmark vector) and y_i is a ground-truth label (e.g. the letter “A” or the ArSL word gloss for “family”). The model learns a function f_θ parameterised by weights θ that minimises prediction error on a training set. At test time, the model outputs $\hat{y} = \arg \max_k P(y=k \mid x)$ for an unseen input x [25]. All four Eshara models are **multi-class classifiers**: they choose

exactly one label from a closed vocabulary (28 deployed ASL letters from a 29-label corpus, 32 ArSL letters, 100 ASL word glosses, or 100 ArSL word signs depending on the model). None of the graduation models perform regression (predicting a continuous number), unsupervised clustering, or reinforcement learning [3].

2.2.2 Static versus sequence classification:

Machine-learning problems differ in whether the input is a single vector or a **sequence** of vectors over time. *Static classification* assigns one label to one feature vector — appropriate when the sign is fully described by a single pose (finger-spelled letters). *Sequence classification* assigns one label to an ordered list of vectors $(x_1, x_2, \dots, x_T)$ — appropriate when meaning depends on motion over roughly one to two seconds (isolated word signs). The following table maps these paradigms to Eshara’s four models.

Table 2-2: Machine-learning paradigms used in Eshara.

Paradigm	Input form	Typical network	Eshara model(s)
Static multi-class	One vector per sample	MLP (feed-forward)	ASL + ArSL letters
Video clip multi-class	T RGB frames per clip	3D CNN (I3D)	ASL words
Sequence multi-class	T landmark vectors per clip	BiLSTM	ArSL words

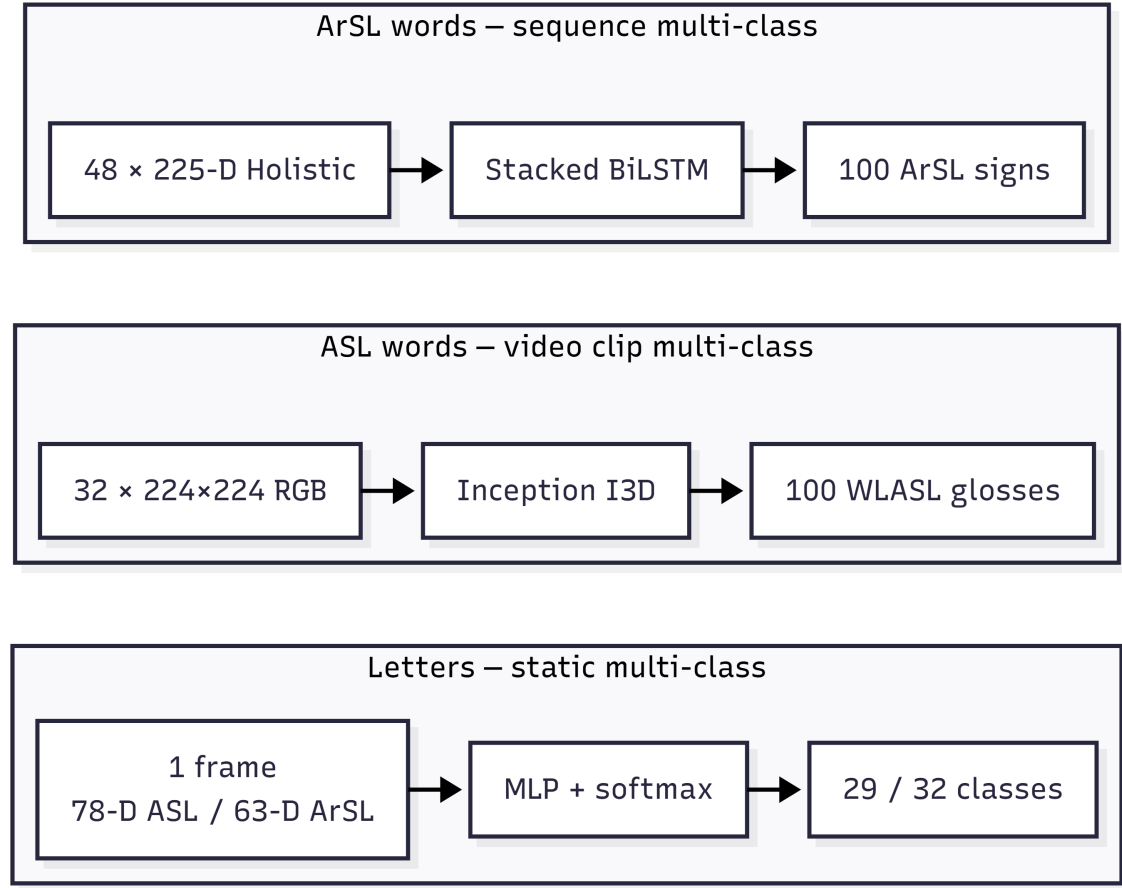


Figure 2-2: Supervised machine-learning paradigms mapped to Eshara’s four models (cf. Table 2-2): static MLPs for letters, Inception I3D for ASL words, and a stacked BiLSTM for ArSL words.

As illustrated in Figure 2-2, input structure (single vector, RGB clip, or landmark sequence) determines the classifier family for each model. Full tensor shapes and training objectives are derived in Chapters 3 and 4.

2.2.3 Families of neural networks in SLR:

Deep learning for SLR draws on several architectural families, each suited to a different input structure [3, 2].

Multi-layer perceptrons (MLPs) are feed-forward networks: information flows input → hidden layers → softmax output with no memory of previous frames. They are the standard choice when $T=1$ and latency must be minimal [26, 27].

Convolutional neural networks (CNNs) apply learned filters over spatial dimensions (height × width of an image, or pseudo-spatial layouts of landmarks). CNNs dominated pixel-based

SLR in the 2014–2018 period [18, 23]. Eshara uses CNNs in early letter-image experiments and in the deployed ASL word model (Inception I3D); letter models convert images to landmarks and train MLPs instead [4, 6].

Recurrent neural networks (RNNs) and LSTMs maintain a hidden state that is updated at each time step, allowing the model to aggregate information across frames. Long Short-Term Memory (LSTM) units avoid vanishing gradients on sequences of 32–48 frames [28, 29]. Bidirectional LSTMs (BiLSTMs) read the sequence forward and backward so that both sign onset and completion inform the decision [30, 19].

Attention mechanisms learn which time steps (or feature subspaces) matter most for classification. Additive attention pools a weighted sum of BiLSTM outputs [31]; multi-head attention learns several attention patterns in parallel [24]. Attention variants appear in the WLASL literature [6]; the ArSL word model uses a plain stacked BiLSTM without an explicit attention module [7].

Transformers replace recurrence with self-attention over the full sequence. They achieve state-of-the-art results on large continuous SLR corpora but typically require more data and compute than BiLSTM models on graduation-scale vocabularies [24, 12]. Transformers are cited for context but are not the graduation architecture choice.

2.2.4 Training, validation, and evaluation:

Supervised models are trained by minimising a **loss function** on labelled data. Eshara uses **categorical cross-entropy** for all four models: the network outputs a probability distribution over classes, and training penalises assignments that assign low probability to the correct label [25, 32]. Optimisation uses the Adam algorithm with mini-batches; dropout and batch normalisation reduce overfitting [32, 26, 27].

Data are partitioned into **training**, **validation**, and **test** sets. Training updates weights; validation selects hyperparameters and early-stopping checkpoints; test evaluation is run once on held-out data to report thesis metrics. Letter models use random image splits; the ArSL word model is evaluated on a held-out KArSL test partition (SignID 71–170) [7]. Reporting validation accuracy alone would be optimistic; the thesis headline numbers are always **test-set** Top-1, Top-5 (word models), and macro-F1 [6, 2].

The following table summarises evaluation metrics common in the SLR literature and explains which Eshara reports in Chapter 6. Word models with imbalanced or scarce per-class data benefit from Top-5 and macro-F1 in addition to Top-1 [6, 7].

Table 2-3: Evaluation metrics in SLR literature and in Eshara.

Metric	Typical use in SLR	Eshara usage
Top-1 accuracy	Isolated classification	All four models (headline)
Top-5 accuracy	Large / imbalanced vocabularies	ASL + ArSL word models
Macro-F1	Per-class balance (scarce classes)	Word models; ArSL graduation headline
Weighted-F1	Sample-frequency weighting	Reported alongside macro-F1 (ArSL)
WER / CER	Continuous SLR, translation	Not used (isolated scope)
Signer-independent acc.	Generalisation across signers	Future work; held-out clip splits for word models

2.2.5 Isolated SLR versus continuous SLR as ML problems:

It is useful to distinguish two **problem formulations** in the literature. *Isolated sign recognition* is standard multi-class classification: each clip contains one known sign, and the model outputs one label. *Continuous sign recognition* is sequence-to-sequence learning: an unsegmented video may contain many signs, and the model must output a sequence of gloss labels with correct boundaries [11, 10]. Continuous SLR additionally requires **segmentation** (detecting where one sign ends) and often **language modelling** across glosses. Eshara solves only the isolated formulation: the user pauses between signs, and the web UI buffers one letter pose or one word motion window at a time. This reduces problem complexity and matches the educational and assistive use cases targeted in graduation scope [6, 7, 12].

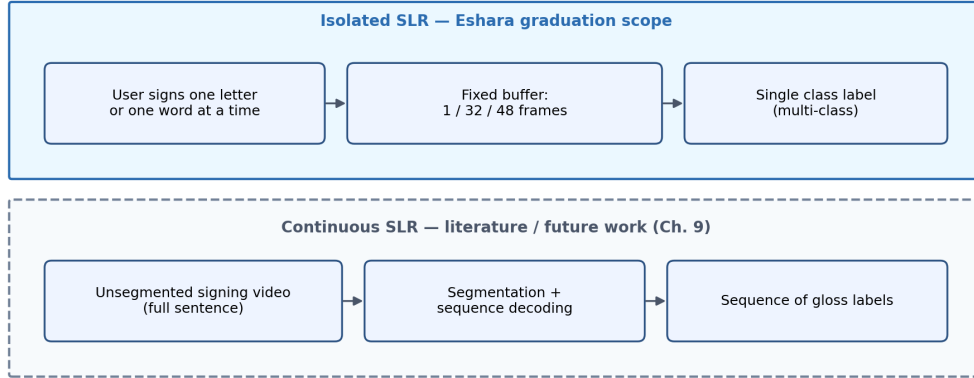


Figure 2-3: Isolated versus continuous sign-language recognition as machine-learning problem formulations. Eshara implements isolated classification only; continuous gloss decoding is deferred to Chapter 9.

As illustrated in the following figure, the fixed-buffer interaction model used in Eshara contrasts with the segmentation and sequence-decoding requirements of continuous SLR benchmarks such as How2Sign and RWTH-PHOENIX [12, 20].

2.2.6 Preprocessing, normalisation, and augmentation:

Landmark-based SLR pipelines share a common preprocessing stage before neural classification [4, 33, 3]. Landmarks are typically **centred** on a reference joint (wrist or shoulder) and **scale-normalised** by inter-joint distance so that signer height and camera distance have reduced influence. **Missing** detections from occlusion are zero-filled or linearly interpolated; the live web client applies the same convention as offline training for consistency [4]. **Standardisation** (subtract training mean, divide by training standard deviation per feature dimension) is common in the literature and is fit on the training split only to prevent test leakage; Eshara’s letter MLPs and deployed word models instead rely on geometric normalisation and in-network batch normalisation without external `StandardScaler` checkpoints (Chapters 3–4).

Data augmentation is widely used when per-class sample counts are low [34, 7]. Image-based SLR applies spatial transforms (rotation, crop, colour jitter) to hand crops; sequence-based SLR may apply temporal jitter, frame dropout, or additive noise on landmark trajectories. Letter models rely on large balanced image corpora; ASL word I3D uses pretrained WLASL weights with standard frame-level normalisation at inference.

2.3 COMPUTER VISION APPROACHES TO SLR

2.3.1 Image-Based Convolutional Methods:

Deep convolutional networks such as ResNet [18], MobileNetV2 [23, 35], and EfficientNet [36] established strong baselines for image classification by learning spatial hierarchies of visual features. Several sign-language studies apply similar architectures to hand images or signing clips, reporting high accuracy when camera viewpoint, background, and signer identity are constrained [2, 19, 37]. Residual connections [18] ease optimisation of deep stacks; inverted residuals and depthwise separable convolutions [23] reduce multiply–accumulate cost, which is attractive for edge deployment. EfficientNet [36] further trades off depth, width, and resolution through compound scaling.

Despite strong offline metrics, three limitations motivate Eshara’s **hybrid input design** for live webcam use. **Generalisation:** a model trained on cropped dataset images may degrade on new backgrounds, skin tones, jewellery, or lower-resolution webcams because pixel statistics shift while kinematic structure remains stable [4]. **Compute:** letter and ArSL word paths classify compact landmark vectors (63–225 dimensions per frame) on modest hardware, whereas ASL word I3D requires batched 3D convolutions on 32-frame RGB clips [23, 6]. **Pipeline mismatch:** if training uses static dataset photographs but inference uses a live camera, the system must solve two domain-shift problems simultaneously (vision appearance and temporal sampling) rather than one (landmark noise only).

The following table records vision pipelines explored during development and the rationale for the graduation choice. During early experimentation, MobileNetV2 was trained on hand-cropped letter images. Although accuracy on fixed-view dataset images was competitive, the domain gap between curated photographs and live webcam capture motivated a unified landmark representation instead. Eshara’s letter pipelines illustrate this pragmatic compromise. The ASL and ArSL letter corpora are distributed as static hand images [8, 9], yet both are converted offline to MediaPipe hand landmarks — 78-D engineered vectors for ASL and 63-D raw vectors for ArSL — and trained with the same MLP *architectural family* but language-specific hidden widths and hyperparameters (Chapter 3). At runtime the same feature conventions are preserved through the deployed inference path, reducing train–serve drift. This design follows the broader literature consensus that skeleton features narrow the domain gap between curated datasets and casual camera capture [4, 2, 3, 33].

Table 2-4: Vision pipelines considered during Eshara development.

Approach	Explored in project?	Outcome
Sensor gloves / Kinect depth	No (graduation scope)	Historical; not deployable on standard webcams
MobileNetV2 on hand images	Yes (early trials)	Strong offline; webcam domain gap
Pixel CNN / ResNet on video	Limited trials	Heavy; mismatched live pipeline
Landmarks + MLP (letters)	Yes — deployed	Unified train/live representation
I3D on RGB (ASL words)	Yes — deployed	32-frame video clips
Landmarks + BiLSTM (ArSL words)	Yes — deployed	48-frame sequences
Full Transformer on words	Trial experiments	VRAM/data limits on graduation GPU
Single multilingual model	No	Incompatible label spaces ($EN \neq AR$)

2.3.2 Skeleton and Landmark Methods:

MediaPipe Hands uses a two-stage pipeline: a palm detector proposes hand regions, and a landmark regressor outputs 21 three-dimensional points per hand in normalised image coordinates [4]. Each point carries (x, y, z) relative to the hand bounding box, yielding $21 \times 3 = 63$ values per frame per hand. For finger-spelling, this single-frame vector is sufficient because letter signs are quasi-static poses: the classifier decision depends on fingertip configuration rather than long-horizon motion [8, 9]. Word signs, by contrast, involve trajectory, two-handed coordination, facial grammar, and upper-body motion [6, 7]. Eshara therefore uses three feature regimes matched to task complexity, summarised in Table 2-5.

Table 2-5: Feature-vector composition used by each Eshara model.

Model	Dim./frame	Frames	Tensor shape	Input source
ASL Letter MLP	78	1	(78,)	MediaPipe Hands (engineered)
ArSL Letter MLP	63	1	(63,)	MediaPipe Hands (raw)
ASL Word I3D	224×224 RGB	32	(32, 224, 224, 3)	Raw webcam frames
ArSL Word BiLSTM	225	48	(48, 225)	MediaPipe Holistic (normalised)

The ASL word model (Inception I3D) does not use landmarks: it consumes **raw RGB frames** ($32 \times 224 \times 224 \times 3$) normalised to $[-1, 1]$. No MediaPipe extraction is required at inference time; the client buffers 32 centre-cropped frames and transmits them as encoded image data [6, 4]. The ArSL word model uses a **225-dimensional** normalised Holistic vector per frame: body pose ($33 \times 3 = 99$) centred on the nose, left hand ($21 \times 3 = 63$) centred on its wrist, and right hand ($21 \times 3 = 63$) centred on its wrist, giving $99 + 63 + 63 = 225$ [7, 22]. This body-relative normalisation reduces sensitivity to signer distance and height. The 48-frame window reflects the average sign duration in KArSL word clips [7].

Before model input, landmarks are centred and scale-normalised (typically relative to wrist or shoulder width) so that signer body size and distance from the camera have reduced influence. Missing landmarks from occlusion are zero-filled or interpolated in the training pipeline; the deployed inference services apply the same convention for consistency [4]. Landmark pipelines offer three engineering advantages emphasised in the literature but rarely demonstrated together in deployed bilingual systems. First, they require less labelled video than pixel-level models because the input dimensionality is orders of magnitude smaller [2, 3]. Second, they support privacy-preserving designs on letter and ArSL word paths: compact features are processed per request, while ASL word mode sends encoded frame buffers instead of continuous raw video. Third, MediaPipe provides runtimes suitable for real-time extraction in practical web systems [5, 4].

OpenPose remains a widely cited baseline but is heavier and lacks a first-class browser runtime [21]. MediaPipe Holistic (543 raw landmarks) and MoveNet (17 body keypoints) were evaluated during development; Holistic proved too slow for sustained live word recognition on laptop hardware at the target frame rate, while MoveNet lacks the dual-hand detail required

for finger-spelling [38, 22]. as compared in Table 2-6 (Section 2.5), which compares these frameworks quantitatively.

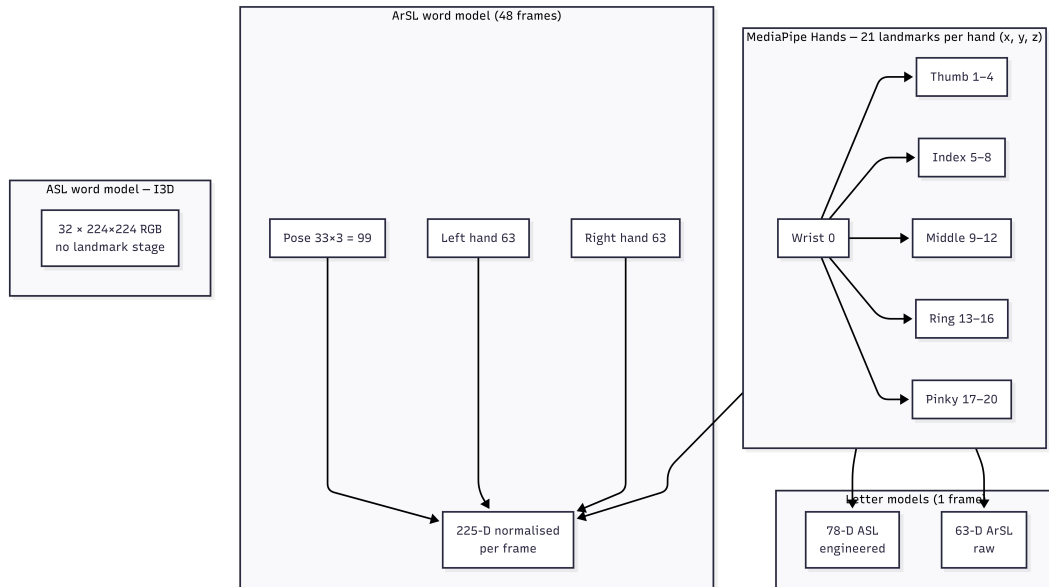


Figure 2-4: MediaPipe hand landmarks (63-D per frame) and extension to normalised pose+hands feature vectors used by the ArSL word model (225-D). The ASL word model (I3D) consumes raw RGB frames rather than landmarks.

As illustrated in Figure 2-4, finger-spelling needs only the 21-point hand mesh, whereas ArSL word recognition concatenates body pose and both hands into a 225-D normalised vector per frame. ASL word recognition bypasses landmarks entirely and feeds raw RGB clips into a pretrained I3D network [4, 22, 5, 6].

2.3.3 Hybrid CNN-RNN and Continuous SLR:

For sentence-level tasks, the dominant pattern is a visual encoder followed by a sequence model. Neural sign-language translation (SLT) systems pair convolutional front ends with recurrent or attention decoders to map signing video to spoken-language text without an intermediate gloss step in some architectures [10]. Weakly supervised continuous sign-language recognition (CSLR) further shows that gloss-level alignment can be learned from untrimmed broadcast footage when only sentence-level weak labels are available [11]. Large-scale benchmarks such as How2Sign provide hundreds of hours of continuous American Sign Language aligned with English subtitles for this research line [12], while RWTH-PHOENIX-Weather serves a similar role for German Sign Language [20].

Continuous systems must solve *segmentation* (where one sign ends and the next begins) in addition to *classification* (which sign is being performed). Isolated classifiers sidestep segmentation by asking the user to pause between signs — a deliberate interaction pattern suitable for a graduation web demo and for educational finger-spelling practice. Eshara word models therefore classify fixed-length inputs — 32-frame RGB clips for ASL words or 48-frame landmark sequences for ArSL words — into a closed vocabulary rather than decoding unbounded gloss sequences. This is a well-studied stepping stone toward sentence-level systems and matches the interaction model of a user deliberately signing one word at a time in front of a webcam [6, 7, 19].

2.4 DEEP-LEARNING ARCHITECTURES

The four Eshara models span three architectural families introduced in Section 2.2: feed-forward MLPs for static letters, a 3D CNN (Inception I3D) for ASL words, and a stacked BiLSTM for ArSL words. This section states each family at a **conceptual** level only — enough to follow the literature review. Full tensor layouts, layer dimensions, normalisation steps, loss functions, and training protocols are derived in Chapter 3 (letters) and Chapter 4 (words).

2.4.1 Multi-Layer Perceptrons (letter models):

An MLP maps a single input vector $\mathbf{x}$ to a class label through a stack of fully connected layers. Each hidden layer applies an affine transform followed by a non-linear activation (ReLU in Eshara), optionally with batch normalisation and dropout for regularisation [26, 27]. The final layer outputs a score vector $\mathbf{z}$ over C classes; softmax converts these scores into a probability distribution and the predicted class is $\hat{y} = \arg \max_k P(y=k | \mathbf{x})$ [25]. Because finger-spelled letters are quasi-static poses, no recurrence is required: one 78-D (ASL) or 63-D (ArSL) vector per frame suffices. Chapter 3 specifies the exact layer widths ($256 \rightarrow 128 \rightarrow 64 \rightarrow C$ for ASL; $512 \rightarrow 256 \rightarrow 64 \rightarrow C$ for ArSL), the ASL 78-D feature-engineering equations, batch-norm and dropout placement, the cross-entropy loss, and the $\sim 49\text{ k}$ / $\sim 183\text{ k}$ parameter counts for Models 1 and 2.

2.4.2 Inception I3D (ASL word model):

Inception I3D (Inflated 3D ConvNet) extends 2D Inception modules to video by convolving jointly over spatial height, width, and time [6]. The ASL word model consumes a clip of $T=32$ centre-cropped RGB frames (224×224 per frame), learns spatio-temporal filters without hand-crafted landmarks, and outputs logits over 100 WLASL glosses. Per-frame predictions are averaged over time before argmax classification. Eshara deploys a pretrained Inception I3D checkpoint (`asl_word_i3d.pt`) rather than training from scratch within graduation GPU limits.

Chapter 4 documents clip buffering, $[-1, 1]$ frame normalisation, and the Model 3 architecture; Chapter 5 documents the live web pipeline.

2.4.3 Bidirectional LSTM (ArSL word model):

A Long Short-Term Memory (LSTM) network maintains a hidden state that is updated at each time step, allowing the model to aggregate motion across a signing clip [28, 29]. Gated units control how much past context is retained or forgotten at each frame. A **bidirectional** LSTM reads the sequence forward and backward and concatenates both hidden states at each step, so sign onset and completion both inform the final decision [30, 19]. The ArSL word model stacks two BiLSTM layers on 48-frame sequences of 225-D normalised Holistic vectors and classifies into 100 Arabic sign classes with a dense softmax head; no explicit attention module is used. Chapter 4 gives the full gate equations, tensor shapes (48, 225), nose/wrist normalisation, and evaluation on the held-out KArSL test partition (SignID 71–170).

2.4.4 Attention and transformers (literature context):

Attention mechanisms learn which time steps in a clip carry the most discriminative motion — useful when neutral rest frames precede or follow the stroke phase [31, 6, 37]. Additive and multi-head variants appear in the WLASL literature but are **not** part of Eshara’s graduation architectures. **Transformers** replace recurrence with self-attention over the full sequence and achieve strong results on large continuous-SLR corpora, but typically require more data and compute than BiLSTM or I3D models at graduation vocabulary scale [24, 12]. They are cited here for context only.

2.5 POSE-ESTIMATION FRAMEWORKS

The following table compares frameworks considered for Eshara. Selection criteria were: (i) availability of a maintained browser/WASM build; (ii) sustained ≥ 25 FPS on a mid-range laptop webcam stream; (iii) compatibility with dual-hand finger-spelling; and (iv) sufficient body keypoints for ArSL word motion [4, 22, 5, 21].

Table 2-6: Comparison of pose-estimation frameworks.

Framework	Landmarks	Typical FPS	Browser	Role in Eshara
MediaPipe Hands	21 per hand	30+	Yes (WASM)	Letter models (78-D ASL / 63-D ArSL)
MediaPipe Holistic	543	18	Yes	Literature reference; ASL word uses I3D (raw RGB)
MediaPipe Pose + Hands	33 + 21 $\times$ 2	25+	Yes (WASM)	ArSL word model (225-D normalised)
OpenPose	25 body	8	No	Literature baseline
MoveNet	17	40+	Yes	Compared, not used

MediaPipe Hands satisfies real-time letter recognition with the lowest feature dimensionality. Holistic provides the richest context for ASL words but at higher per-frame cost; it is used offline and in the browser word mode where the user holds still briefly during a sign. Pose+Hands offers the best balance for ArSL graduation words: full torso and dual-hand coverage without the face mesh overhead. OpenPose is included as a historical baseline from the literature [21] but was not integrated because deployment targets the browser, not a C++ desktop server. MoveNet’s 17-point skeleton is insufficient for distinguishing ArSL two-hand minimal pairs [38, 7].

2.6 SIGN-LANGUAGE DATASETS

The following table summarises the corpora that ground the four Eshara models. A deliberate design constraint is that **each model maintains its own label space**; vocabularies are not shared across languages or tasks [6, 7]. This avoids impossible label alignment (an ASL gloss does not correspond to an ArSL gloss) and allows independent retraining when one corpus is updated.

Table 2-7: Sign-language datasets used or referenced in this work.

Dataset	Lang.	Type	Classes	Samples	Role
ASL Alphabet (Kaggle)	ASL	Images	29	87 000	ASL letter MLP
ArASL2018	ArSL	Images	32	54 049	ArSL letter MLP
WLASL-2000	ASL	Video	2000	21 000	100-class I3D subset
KArSL-502	ArSL	Video	502	24 000	100-class word subset (SignID 71–170)
How2Sign	ASL	Continuous	—	80 h	Future work (Ch. 8)
RWTH-PHOENIX	DGS	Continuous	—	—	Comparison only

The following table highlights **class imbalance** and average samples per class — a key reason Eshara reports macro-F1 and Top-5 for word models, uses a 100-class WLASL subset for I3D, and a 100-class KArSL word subset for the ArSL BiLSTM [6, 7].

Table 2-8: Approximate class balance across Eshara training corpora.

Corpus (Eshara use)	Classes	Total samples	~Avg/class	Implication
ASL Alphabet	29	87 000	~3 000	Balanced; Top-1 sufficient
ArASL2018	32	54 049	~1 700	Balanced; more confusion pairs
WLASL I3D subset	100	~3 000 clips	10–50 (varies)	I3D pretrained; Top-5
KArSL word subset	100	~4 800 seq.	~48	BiLSTM; 99.62 % Top-1

Train/test splitting and leakage. Random frame-level or clip-level shuffles on video corpora can **leak** adjacent segments from the same recording session into both training and test sets, producing optimistically high accuracy that does not generalise to new signing sessions [39, 7]. The ArSL word model is evaluated on a held-out KArSL test partition (SignID 71–170); the headline **99.62 %** Top-1 reported in Chapter 6 was obtained under this protocol. Letter image datasets contain independent photographs and use random stratified splits instead.

ASL Alphabet (Kaggle). The dataset provides large-scale static hand images for 29 finger-spelling classes including control signs (space, del, nothing) [8]. With roughly 3 000 images per class on average, it supports reliable MLP training and achieves high offline accuracy in project experiments. Limitations include studio-style backgrounds, limited signer diversity compared with field capture, and the absence of temporal motion required for word models. Motion letters J and Z are under-represented in live testing because they require trajectory, not a single pose.

ArASL2018. This corpus is the primary public benchmark for Arabic finger-spelling, with 32 classes and 54 049 images collected under relatively controlled indoor conditions [9]. Visually similar letter pairs (for example ٤ vs. ٥ in Arabic signing) produce more inter-class confusion than in ASL, which motivates separate evaluation and a wider ArSL letter MLP head despite the shared architectural family (Chapter 3). The dataset does not cover ArSL word signs; a second corpus is required for word-level Arabic recognition.

WLASL-2000. Li et al. introduced WLASL as a large-scale American Sign Language word dataset spanning 2 000 glosses and roughly 21 000 video clips sourced from public video portals [6]. Not every gloss has equal footage quality; broken URLs in the metadata reduce effective coverage. Eshara’s ASL word model (Model 3) uses a **100-class WLASL subset**: 32-frame centre-cropped 224×224 RGB clips pass through a pretrained Inception I3D checkpoint (asl_word_i3d.pt) with a 100-way softmax head [6]. Using a pretrained checkpoint keeps deployment tractable within graduation GPU and timeline constraints.

KArSL-502 and the graduation word subset. Sidig et al. published KArSL as a 502-class Arabic sign-language video corpus with approximately 24 000 clips performed primarily by Saudi signers [7]. Average samples per class are far smaller than ASL Alphabet’s image counts, and regional dialect variation across the Arab world is not fully represented. Eshara uses a stacked **BiLSTM** over a **100-class Arabic word subset** (SignID 71–170 of KArSL) with 225-D nose- and wrist-normalised Holistic landmark sequences of 48 frames each, achieving **99.62 %** Top-1 on the held-out test set [7].

Continuous benchmarks (not used in graduation). How2Sign [12] and RWTH-PHOENIX-Weather [20] are cited to position Eshara relative to sentence-level research. How2Sign aligns

continuous ASL video with English subtitles and is the intended dataset for future continuous recognition experiments in Chapter 9. RWTH-PHOENIX demonstrates that European sign languages have received substantially more continuous-SLR infrastructure than ArSL, reinforcing the bilingual gap Eshara addresses at the isolated-sign level.

2.7 BILINGUAL AND ARABIC SIGN-LANGUAGE RESEARCH

Published sign-language recognition systems are overwhelmingly monolingual, with American Sign Language and a small set of European sign languages (German Sign Language, British Sign Language) dominating the literature [2, 20, 13]. Comprehensive surveys confirm rapid growth in deep-learning SLR methods but note persistent dataset imbalance across languages [3, 40]. Arabic Sign Language has received comparatively little attention despite substantial deaf and hard-of-hearing populations in the Arab world [1]. The release of KArSL-502 in 2021 enabled reproducible ArSL word classification experiments [7], yet few works extend to letter-level finger-spelling, bilingual deployment, or production-grade web interfaces in the same publication.

Recent ArSL research illustrates the dominant pixel-based and hybrid CNN–LSTM paradigms. Luqman and El-Alfy cascaded CNN and LSTM encoders on KArSL subsets (33, 100, and 190 signs), reporting $>99\%$ accuracy on controlled splits [41]. Dabwan and Jadhav applied ConvLSTM and LRCN architectures to 28 KArSL classes with validation accuracy up to 95% [42]. Alsaadi et al. developed a real-time ArSL *alphabet* recogniser using AlexNet on static images (94.81% accuracy) [43]. Noor et al. proposed a hybrid CNN–LSTM model on a small custom Saudi ArSL word set with MediaPipe landmark extraction in a cloud deployment [44]. These works confirm growing ArSL interest but typically address **one language, one granularity** (letters *or* words), and **offline or server-side video** — not a unified bilingual web product.

ArASL2018 addressed the letter-level gap for Arabic [9], and several CNN and LSTM papers report competitive accuracy on KArSL subsets, but these contributions typically target a single task (letters *or* words) in a single language with offline batch evaluation. To the authors’ knowledge, no prior open system combines ASL and ArSL recognition at both letter and word granularities within a single webcam-driven web application. Eshara is designed explicitly to fill that *structural* gap — four specialised models, one user interface — rather than to incrementally improve top-1 accuracy on a single-language benchmark by a few percentage points [6, 7].

Arabic-specific engineering considerations include right-to-left user-interface layout for translated text output, right-hand dominance in several regional signing conventions, and dialect variation that a single national corpus cannot fully capture [7, 9]. These factors motivate parallel language-specific models with independent vocabularies rather than a single multilingual

classifier with a shared label space. A shared embedding space across languages would require aligned glosses and substantially more parallel data than this graduation project scope provides [10].

2.8 REAL-TIME WEB DEPLOYMENT

A persistent divide exists between recognition accuracy reported in offline experiments and systems people can actually use [2, 13]. Many papers stop at confusion matrices on held-out test clips and do not report end-to-end latency from camera capture to readable text on screen. Deployment-focused surveys note that fewer than one in five SLR papers describe a user-facing demo accessible outside the authors’ laboratory [3].

Eshara targets a deployable web architecture with the following runtime path: (1) the browser captures webcam frames; (2) the React client sends translation requests to a Node.js/Express gateway over HTTPS; (3) the Node layer authenticates and forwards requests to internal FastAPI model services over HTTP [45, 46]; (4) the inference service loads the correct model path for the selected language and mode; (5) predictions are returned as JSON and rendered in the React UI [47]. Socket.IO is used for live chat messaging, while prediction requests are served through the HTTP `/predict` flow. For letter mode, a stability decoder applies majority voting over a short frame buffer and a hold timer before committing a character, preventing repeated glyphs from a single prolonged pose.

Keeping feature extraction on the client limits privacy risk and bandwidth use: a 48-frame ArSL word packet transmits roughly $48 \times 225 \times 4 \approx 43$ KB of floats per inference request, whereas uncompressed 720p video would exceed several megabytes per second [48]. This pattern mirrors industrial real-time inference but remains uncommon in academic sign-language literature, where server-side video upload and batch processing are still typical [10, 6]. Accessibility guidelines (WCAG 2.2) further motivate web delivery: users can access the tool from any modern browser without installing native applications [49].

Graduation scope is **web-only**: no mobile application, no TensorFlow Lite on-device model, and no app-store deployment is claimed in this thesis [50]. The hypothesis in Section 1.5.4 targets end-to-end latency of no more than 200 ms from frame capture to displayed prediction on a standard laptop; Chapter 5 reports whether live web measurements support that target alongside the offline accuracy metrics of Chapter 6.

2.9 RESEARCH GAP AND PROJECT POSITIONING

The following table positions Eshara against typical published systems. Direct numeric comparison of Top-1 accuracy is often misleading because vocabularies, splits, and input representations differ; the table therefore emphasises **structural** coverage (language, granularity, deployment) [2, 6, 7].

Table 2-9: Structural comparison with representative SLR research lines (not a claim of state-of-the-art rank).

System / line	Lang.	Letters	Words	Live web	Landmarks
Typical WLASL paper [6]	EN	No	Yes	Rare	Varies
KArSL CNN–LSTM works [42, 41]	AR	No	Yes	No	RGB/depth
ArASL2018 / ArSLA [9, 43]	AR	Yes	No	Limited	RGB
Noor et al. hybrid [44]	AR	Partial	Yes	Server/cloud	MediaPipe
Eshara (this thesis)	EN+AR	Yes	Yes	Yes	Yes

The following table consolidates the gaps identified throughout this chapter and maps each to a concrete Eshara design decision. Chapters 3 and 4 detail the methodology; Chapter 6 reports whether the experimental results support the hypothesis stated in Section 1.5.4.

Table 2-10: Research gaps addressed by Eshara.

Gap in the literature	Eshara response
ArSL word recognition is under-studied	ArSL word BiLSTM on KArSL 100-class word subset (SignID 71–170)
Most systems target letters <i>or</i> words, not both	Four specialised models with mode selection in one web app
ASL-centric datasets and models dominate	Parallel ArSL pipelines for letters and words
Academic prototypes rarely reach end users	React + FastAPI web product with live webcam UI
Raw video commonly sent to servers	Client-side landmarks for letters + ArSL words (78/63/225-D); encoded frames for ASL word I3D
Single-model papers lack unified APIs	One inference backend serving four models

The literature review supports the central hypothesis of Chapter 1: a webcam-driven pipeline with four specialised deep-learning models and a unified web deployment can achieve reliable recognition across the targeted tasks. Chapter 6 evaluates this claim using Top-1 accuracy, Top-5 accuracy, and macro-F1 on held-out test sets for each model independently [6, 7]. The experiments test five implicit sub-claims: (i) compact MLPs on hand landmarks suffice for letter finger-spelling (78-D engineered for ASL, 63-D raw for ArSL); (ii) a pretrained I3D network recognises ASL words from buffered RGB clips; (iii) a landmark-based BiLSTM recognises ArSL words from 225-D Holistic sequences; (iv) the deployed feature-extraction and inference path generalises from dataset images/videos to live webcam input; and (v) a three-tier web backend can serve heterogeneous models without degrading interactive latency [4, 46, 48].

2.10 COMPARISON OF ESHARA’S FOUR MODELS

The following table summarises how architecture, feature dimensionality, temporal window length, and vocabulary size are matched to each recognition task. This design matrix is the technical backbone of Eshara and is detailed further in Chapters 3 and 4 and evaluated in Chapter 6.

Table 2-11: Architecture comparison of Eshara’s four graduation models.

Model	Lang.	Task	Architecture	Features	Frames	Classes	~Params
ASL Letter MLP	EN	Letter	MLP	78-D hand	1	28 (29 corpus)	~49K
ArSL Letter MLP	AR	Letter	MLP	63-D hand	1	32	~183K
ASL Word I3D	EN	Word	Inception I3D (3D CNN)	224×224 RGB	32	100	~12.5M
ArSL Word BiLSTM	AR	Word	Stacked BiLSTM	225-D Holistic (norm.)	48	100	~255K

Static finger-spelling requires only per-frame pose information, motivating compact MLP classifiers for both letter models despite different vocabularies and feature dimensions [8, 9]. Word signs require temporal modelling: the ASL word model uses a pretrained Inception I3D consuming raw RGB clips because 3D convolutions capture the rich spatio-temporal patterns of English word signs without manual landmark engineering [6]; the ArSL word model uses 225-D normalised Holistic sequences over 48 frames, matched to KArSL signing duration and the 100-class Arabic word vocabulary [7]. Letter MLPs are on the order of 10^4 – 10^5 parameters; the ArSL word BiLSTM is ~255K and the ASL word I3D is ~12.5M parameters.

No label space is shared across models; bilingual capability is implemented as parallel inference pipelines behind one web interface. The ArSL word model achieves **99.62 %** Top-1 on the held-out test set (100 classes, Chapter 6); the ASL word I3D reaches **65.89 %** Top-1 on the WLASL-100 benchmark (Chapter 6). Chapters 3 and 4 present methodology for each row in Table 2-11.

Chapter summary. This literature review established the technological and scientific context for Eshara. Historically, SLR moved from instrumented gloves to pixel-based deep learning and finally to hybrid landmark- and video-based classifiers deployable on ordinary webcams (Section 2.1). Machine-learning foundations (Section 2.2) cast all four models as supervised classifiers — static MLPs for letters, Inception I3D for ASL words, and a stacked BiLSTM for ArSL words — evaluated with Top-1, Top-5, and macro-F1 under honest split protocols. Computer-vision evidence (Section 2.3) justified the hybrid input design; deep-learning architectures (Section 2.4) introduce the model families whose full equations and dimensions are derived in Chapters 3 and 4. Dataset scale and imbalance (Section 2.6) motivated the 100-class WLASL I3D subset and the 100-class KArSL word subset. Bilingual and ArSL-specific work (Section 2.7) confirmed a structural gap that Eshara fills with four specialised models behind one web interface. The following design decisions, justified above, are carried forward:

- (1) Landmark-first feature pipelines for letters and ArSL words, with RGB clip inference for ASL words;
- (2) 78-D (ASL) and 63-D (ArSL) MLP letter heads in the same architectural family with language-specific widths;
- (3) pretrained Inception I3D on 32-frame 224×224 RGB clips

for ASL words; (4) stacked BiLSTM on 225-D normalised Holistic sequences (48 frames) for ArSL words; (5) four independent vocabularies with no shared label space; (6) isolated-sign interaction only; (7) web-only three-tier deployment (React + Node/Express + FastAPI). Chapters 3 and 4 describe how each decision is implemented; Chapter 6 tests offline accuracy and Chapter 5 tests live deployment latency.

3 METHODOLOGY — LETTER RECOGNITION

Chapter 2 reviewed the literature and justified Eshara’s design funnel (static MLP paradigm for letters; as set out in Table 2-5, Chapter 2). This chapter specifies *how* finger-spelling recognition is implemented: shared system architecture, letter datasets, MediaPipe Hands feature extraction, the two letter MLP pipelines, temporal stabilisation, Arabic UI rendering, engineering trade-offs, and evaluation protocol. Isolated **word** recognition methodology is presented in Chapter 4 [8, 9, 4].

3.1 HIGH-LEVEL SYSTEM ARCHITECTURE

Eshara is a three-tier web application. The **browser** captures webcam frames and sends authenticated requests to a **Node.js/Express** API gateway; the gateway proxies inference requests internally to **FastAPI** model services over HTTP [45, 46, 47]. The inference tier loads standalone checkpoints at startup — letter MLPs and the ArSL word BiLSTM as Keras .h5 files, the ASL word I3D as a PyTorch .pt file — and returns JSON predictions. Letter models use no external scaler; the deployed ArSL word BiLSTM likewise consumes raw nose/wrist-centred coordinates without a saved `StandardScaler` (Chapter 4). The **React** UI renders letter streams (with a stability decoder) or isolated word labels, with RTL layout for Arabic text output [49].

The runtime path for **letter** inference is:

1. User selects language (EN/AR) and **letter** mode in the React client.
2. The inference service runs MediaPipe **Hands** to extract 21 landmarks per frame and builds a language-specific feature vector (63-D raw for ArSL; 78-D engineered for ASL).
3. Each frame’s vector is classified immediately by the language-specific letter MLP; a stabilisation decoder commits stable characters.
4. FastAPI routes the request to the correct model, returns {prediction, confidence, top_k}, and the UI updates (with Arabic reshaping when ArSL is selected).

This decomposition ensures (i) **consistency** — all users share the same server-side weights; (ii) **security control** — requests are validated and authenticated at the Node gateway; and (iii) **modularity** — each model can be retrained and redeployed independently [4, 48].

Design invariant: no shared vocabulary across models. ASL glosses do not align with ArSL glosses; letter and word label spaces differ even within one language. Four parallel classifiers behind one API enforce this constraint at deployment time.

Word-mode buffering (32-frame RGB for ASL I3D; 48-frame Holistic for ArSL BiLSTM) is described in Chapter 4.

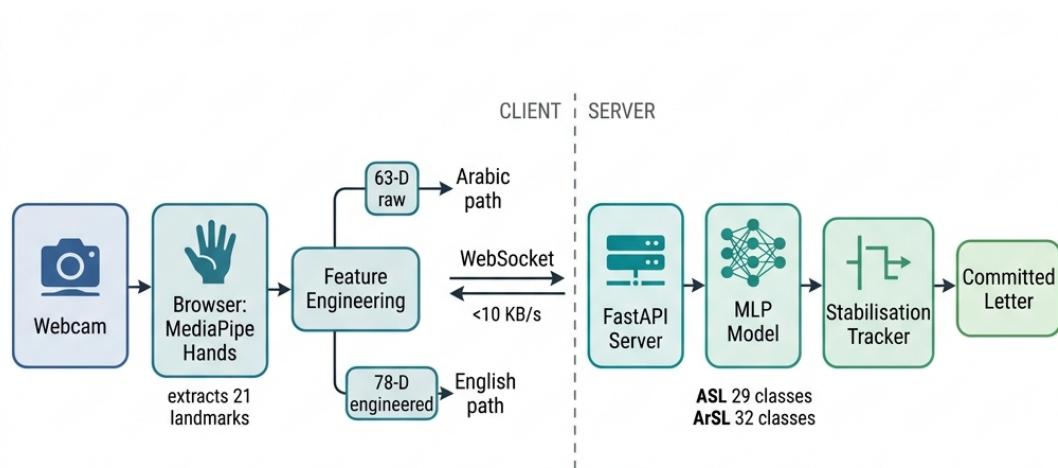


Figure 3-1: Letter-mode client-server pipeline. The browser captures a webcam frame as a base64-encoded JPEG and sends it via HTTP POST to the Node.js application tier; the gateway proxies the request to FastAPI, where MediaPipe Hands extracts landmarks and engineers a 63-D (ArSL) or 78-D (ASL) vector before the letter MLP and stabilisation decoder commit a character.

3.2 LETTER DATASETS AND SPLIT POLICIES

The following table summarises the two letter corpora. Static images use random stratified splits [8, 9].

Table 3-1: Letter datasets, feature shapes, and split policies.

Task	Dataset	Classes	Features	Frames	Split
ASL letters	ASL Alphabet	29 (28 deploy.)	78-D	1	~64/16/20 train/val/test
ArSL letters	ArASL2018	32	63-D	1	60/20/20 train/val/test

Training software environment. Letter and word training share a common Python stack (as defined in Table 3-2). Heavy jobs — Holistic extraction over KArSL videos and Inception I3D fine-tuning experiments — were run on **Kaggle GPU** (T4/P100/L4, CUDA 11.x) when local VRAM was insufficient; letter MLP training and live prototyping used a **4 GB NVIDIA GPU** on the development laptop. TensorFlow/Keras letter and ArSL word training enable GPU memory growth; PyTorch loads the ASL I3D checkpoint for server inference [50, 51, 32].

Table 3-2: Training and inference software environment (letters and words).

Component	Version / role
Python	3.9+
TensorFlow / Keras [50, 51]	2.x — letter MLPs, ArSL word BiLSTM
PyTorch [52]	2.x — ASL word Inception I3D (asl_word_i3d.pt)
CUDA / cuDNN [53]	11.x (Kaggle GPU; local 4 GB GPU when available)
MediaPipe [4, 5]	0.10.x (Python extraction; JS/WASM in browser)
OpenCV, NumPy, scikit-learn [54, 55, 56]	Feature I/O, metrics
Jupyter [57]	Interactive training environment
Adam optimiser [32]	$\eta=10^{-3}$ default (letters); word schedules in Ch. 4

3.2.1 ASL Alphabet (English Letters):

The Kaggle ASL Alphabet corpus provides $\sim 87\,000$ static hand images across 29 classes (A–Z plus space, del, nothing) [8]. The graduation **deployed** ASL letter model excludes the nothing control class, yielding a 28-way label space (Chapter 6). Images are converted **offline** to MediaPipe Hand landmarks, then augmented with wrist-relative normalisation and 15 inter-joint angles to form a 78-D engineered vector cached in CSV/NPZ before MLP training. The **same** 78-D representation is computed in the browser at inference time, eliminating train/serve skew on input geometry [4].

3.2.2 ArASL2018 (Arabic Letters):

ArASL2018 provides 54 049 images across 32 Arabic finger-spelling classes [9]. Processing mirrors the landmark extraction stage of the ASL pipeline but uses the flattened 63-D raw coordinate vector without additional angle features. Visually similar letter pairs (e.g. ل/ل) motivate per-class evaluation in Chapter 6. Model 2 shares the same **MLP family** as Model 1 (Dense–BN–ReLU–Dropout blocks) but uses a **wider, separately trained head** with different input dimension, hidden widths, and hyperparameters (Section 3.5; Table 3-5).

3.2.3 End-to-End Training Pipeline (Dataset to .h5 Artifact):

Both letter corpora follow the same offline preparation and training pipeline. The numbered steps below trace the complete path from raw image dataset to the exported checkpoint used by the FastAPI inference service.

1. **Image loading.** Dataset images are read from disk (JPEG/PNG) and converted to RGB. MediaPipe Hands is initialised with `static_image_mode=True`, `max_num_hands=1`, and `min_detection_confidence=0.7`. Unreadable or empty files are skipped.
2. **Landmark extraction.** For each image, MediaPipe returns 21 hand landmarks $\{(x_i, y_i, z_i)\}_{i=0}^{20}$ normalised to image coordinates $[0, 1]^2$ with depth estimate z_i . Images where no hand is detected are discarded. Detection rates are approximately 69 % for ASL Alphabet and 77 % for ArASL2018, yielding $\sim 60\text{ k}$ and $\sim 74\text{ k}$ successful samples respectively.

3. **Feature engineering.** Extracted landmarks are transformed into a fixed-length feature vector (see Section 3.3):

$$\mathbf{x} = \begin{cases} \mathbf{x}_{\text{raw}} \in \mathbb{R}^{63} & \text{ArSL: flatten } (x_i, y_i, z_i)^\top \text{ directly,} \\ \mathbf{x}_{\text{ASL}} \in \mathbb{R}^{78} & \text{ASL: wrist-centred 21 landmarks + 15 angles (63+15).} \end{cases}$$

4. **Caching.** Feature vectors and class labels are written to a CSV file (`asl_letters_engineered.csv` / `ASLAD-3000_v2.csv`) and a compressed NPZ archive for fast NumPy loading. This step runs once; subsequent training passes load directly from cache.
5. **Train / validation / test split.** The feature matrix $X \in \mathbb{R}^{N \times d}$ is split with **stratified** sampling (preserving per-class proportions) using `random_state=42`:

$$(X_{\text{train}}, X_{\text{val}}, X_{\text{test}}) \quad \text{proportions} \quad \begin{cases} 0.64 : 0.16 : 0.20 & (\text{ASL}) \\ 0.60 : 0.20 : 0.20 & (\text{ArSL}) \end{cases} \quad (1)$$

Test split is isolated before any hyperparameter search and is touched exactly once for final reporting.

6. **One-hot label encoding.** Integer class indices are converted to one-hot vectors:

$$\mathbf{y} = \mathbf{e}_c \in \{0, 1\}^C, \quad e_{c,k} = \mathbb{I}[k = c], \quad C \in \{28, 32\}, \quad (2)$$

via Keras `to_categorical` (or `np.eye` for ArSL when TensorFlow is loaded later in the pipeline). Feature vectors are fed **directly** to the MLP without an external `StandardScaler`; batch normalisation inside the network handles activation scaling during training. (Deployed word models likewise use no external scaler at inference; see Chapter 4.)

7. **Model construction.** The MLP is assembled with the Keras Sequential API. Layer widths, BN placement, dropout rates, and output softmax dimension are specified per model (Sections 3.4 and 3.5; Figure 3-6).

8. **Callbacks.** Three callbacks are registered before `model.fit`:

- **ModelCheckpoint** [51]: saves the `.h5` artifact to disk whenever `val_accuracy` improves (`save_best_only=True, mode='max'`).
- **EarlyStopping** [51]: monitors `val_loss`; halts training when no improvement is seen for `patience` consecutive epochs and restores the best weights (`restore_best_weights=True`).

- **ReduceLROnPlateau** (ArSL only) [51]: multiplies the current learning rate by factor 0.5 when `val_loss` does not improve for 7 consecutive epochs, floored at $\eta_{\min} = 10^{-7}$.

9. **Mini-batch training.** Parameters θ are updated over mini-batches of size $B \in \{64, 256\}$ using the Adam optimiser. At each step t with gradient $g_t = \nabla_{\theta} \mathcal{L}$:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (3)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (4)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad (5)$$

$$\theta_t \leftarrow \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon_A}, \quad (6)$$

with defaults $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon_A=10^{-7}$ [32].

10. **Test evaluation.** After training, the best checkpoint is loaded and evaluated **once** on the held-out test split. Top-1 accuracy, test loss, and a per-class classification report (precision, recall, F1) are recorded for Chapter 6.

11. **Artifact export.** The best .h5 checkpoint is the final deliverable:

- ASL: `asl_mediapipe_mlp_model_engineered.h5`
- ArSL: `arsl_mediapipe_mlp_model_bestV2.2.h5`

The FastAPI inference service loads these at startup (Chapter 5); no further post-processing (quantisation, pruning) is applied within graduation scope.

The complete workflow is illustrated in the following figure after the feature definitions in Section 3.3.

3.3 FEATURE EXTRACTION FOR LETTERS

MediaPipe Hands outputs 21 (x, y, z) points per hand [4]. For finger-spelling, `max_num_hands=1` suffices; face and pose modules are reserved for word models (Chapter 4). Missing landmarks are zero-filled; the live client mirrors training conventions. As shown in Figure 3-2, the diagram labels the wrist (0) and the four finger chains (thumb 1–4, index 5–8, middle 9–12, ring 13–16, pinky 17–20).

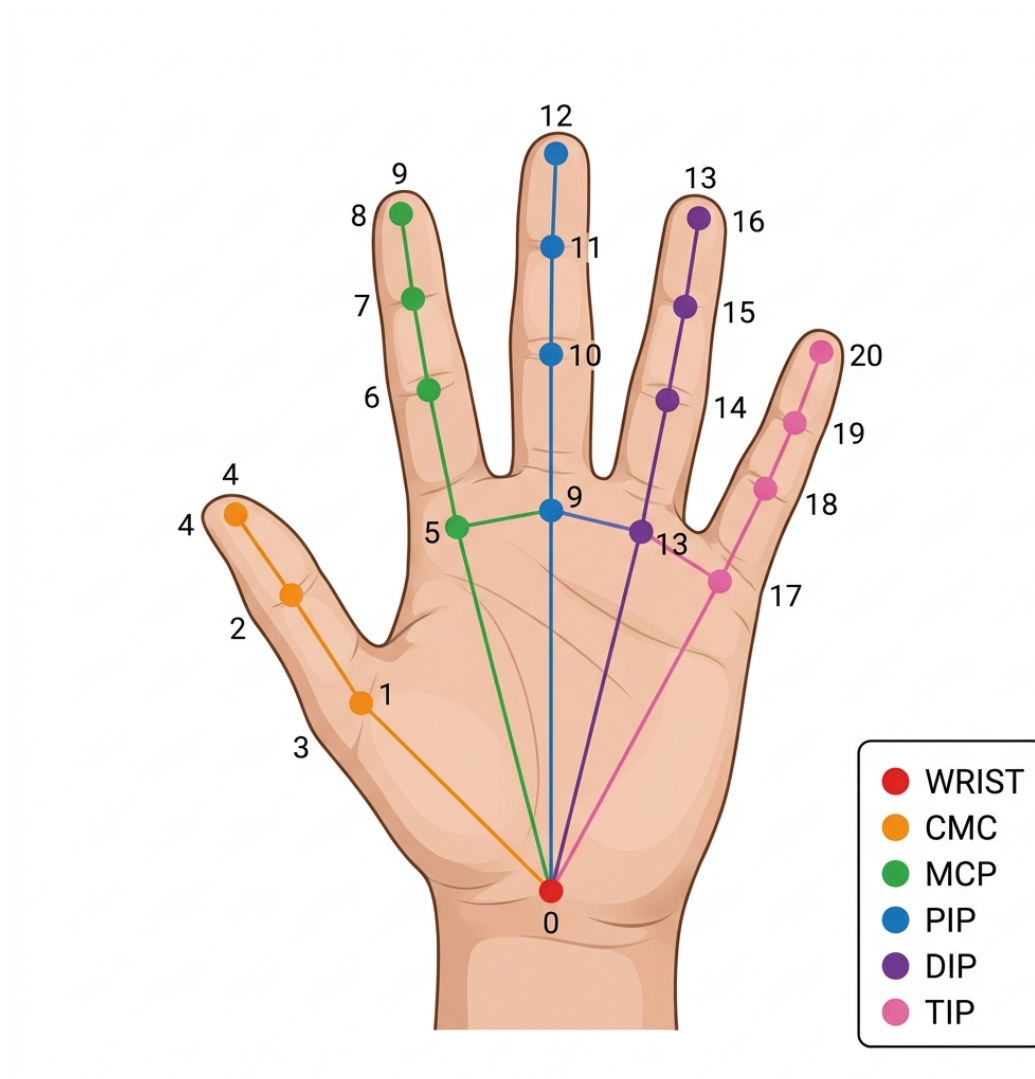


Figure 3-2: MediaPipe Hand landmark topology (21 points). Each landmark contributes (x, y, z) coordinates; flattening yields a 63-D raw vector.

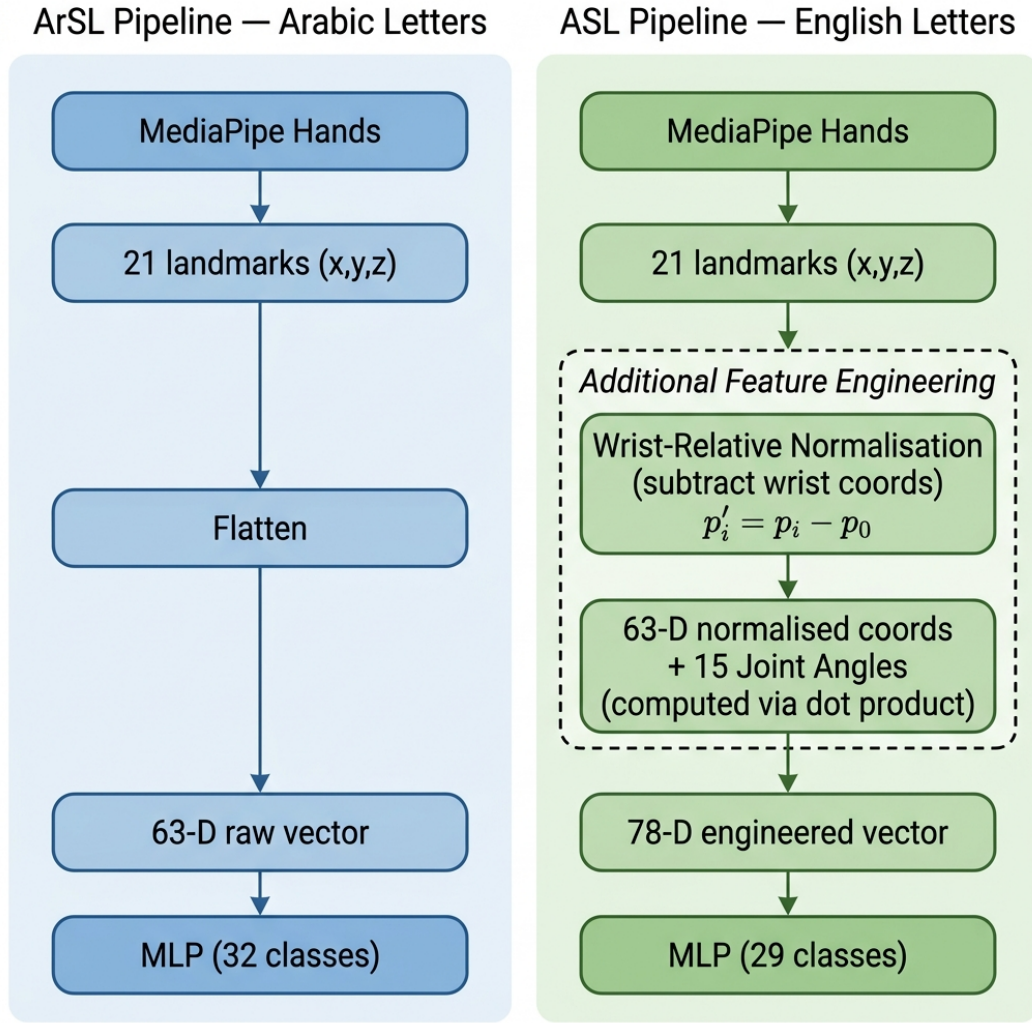


Figure 3-3: Letter feature-engineering split. ArSL (left): MediaPipe Hands $\rightarrow$ flatten $\rightarrow$ 63-D raw MLP (32 classes). ASL (right): additional wrist-relative normalisation and 15 joint angles $\rightarrow$ 78-D engineered MLP (28 deployed classes).

The two letter pipelines diverge after landmark extraction (as illustrated in Figure 3-3). Let $\mathbf{p}_i = (x_i, y_i, z_i)^\top \in \mathbb{R}^3$ denote landmark i with wrist index $i=0$. The raw feature vector is

$$\mathbf{x}_{\text{raw}} = [x_0, y_0, z_0, x_1, y_1, z_1, \dots, x_{20}, y_{20}, z_{20}]^\top \in \mathbb{R}^{63}. \quad (7)$$

ArSL (Model 2) uses $\mathbf{x}_{\text{raw}}$ directly. **ASL (Model 1)** wrist-centres every landmark and appends 15 inter-joint angles:

$$\mathbf{p}'_i = \mathbf{p}_i - \mathbf{p}_0, \quad i = 0, \dots, 20, \quad (8)$$

$$\theta_j = \arccos\left(\frac{\mathbf{v}_{j,1} \cdot \mathbf{v}_{j,2}}{\|\mathbf{v}_{j,1}\| \|\mathbf{v}_{j,2}\| + \varepsilon}\right), \quad j = 1, \dots, 15, \quad (9)$$

where $\mathbf{v}_{j,1} = \mathbf{p}'_{a_j} - \mathbf{p}'_{b_j}$ and $\mathbf{v}_{j,2} = \mathbf{p}'_{c_j} - \mathbf{p}'_{b_j}$ are the two bone vectors meeting at joint b_j (the middle landmark of the triplet (a_j, b_j, c_j)), and $\varepsilon = 10^{-8}$ avoids division by zero. The 15 triplets cover three joints per finger (metacarpo-phalangeal MCP, proximal inter-phalangeal PIP, distal inter-phalangeal DIP) across all five fingers, as defined in Table 3-3. The deployed ASL input concatenates all wrist-centred coordinates with the angle block:

$$\mathbf{x}_{\text{ASL}} = [\mathbf{p}'_0, \dots, \mathbf{p}'_{20}; \theta_1, \dots, \theta_{15}]^\top \in \mathbb{R}^{78}, \quad (10)$$

i.e. $21 \times 3 = 63$ translation-invariant landmark values (with $\mathbf{p}'_0 \equiv \mathbf{0}$) plus 15 scalar angles. The same 78-D layout is reproduced in the browser at inference time [4].

Table 3-3: The 15 inter-joint angle triplets used in the 78-D ASL feature vector. Each row specifies the landmark index triple (a, b, c) where b is the vertex joint; the angle is computed between bone vectors $b \rightarrow a$ and $b \rightarrow c$.

Finger	Joint	Triplet (a, b, c)
Thumb	MCP	(0, 1, 2)
Thumb	PIP	(1, 2, 3)
Thumb	DIP	(2, 3, 4)
Index	MCP	(0, 5, 6)
Index	PIP	(5, 6, 7)
Index	DIP	(6, 7, 8)
Middle	MCP	(0, 9, 10)
Middle	PIP	(9, 10, 11)
Middle	DIP	(10, 11, 12)
Ring	MCP	(0, 13, 14)
Ring	PIP	(13, 14, 15)
Ring	DIP	(14, 15, 16)
Pinky	MCP	(0, 17, 18)
Pinky	PIP	(17, 18, 19)
Pinky	DIP	(18, 19, 20)

The following table summarises the complete feature-vector composition for each letter model, cross-referencing dimensions to the equations above.

Table 3-4: Feature-vector dimension breakdown for the two letter models.

Component	Dims	ASL (Model 1)	ArSL (Model 2)
Wrist-centred coords $\mathbf{p}'_{0...20}$	63	Yes (Eq. (10))	No (raw wrist incl.)
Raw wrist (x_0, y_0, z_0)	3	(included in $\mathbf{p}'_0=\mathbf{0}$)	Yes
Raw landmarks $\mathbf{p}_{1...20}$	60	No	Yes
Joint angles $\theta_{1...15}$	15	Yes (Table 3-3)	No
Total	78 / 63	78-D	63-D

Holistic/Pose+Hands extensions for word models are summarised as set out in Table 2-5 (Chapter 2).

As summarised in Figure 3-4, the offline training path runs from dataset images to the exported .h5 checkpoint. Shared stages (landmark extraction through label encoding) are identical for both languages; ASL and ArSL diverge at feature engineering and at MLP head configuration (as compared in Figure 3-6 and Table 3-5).

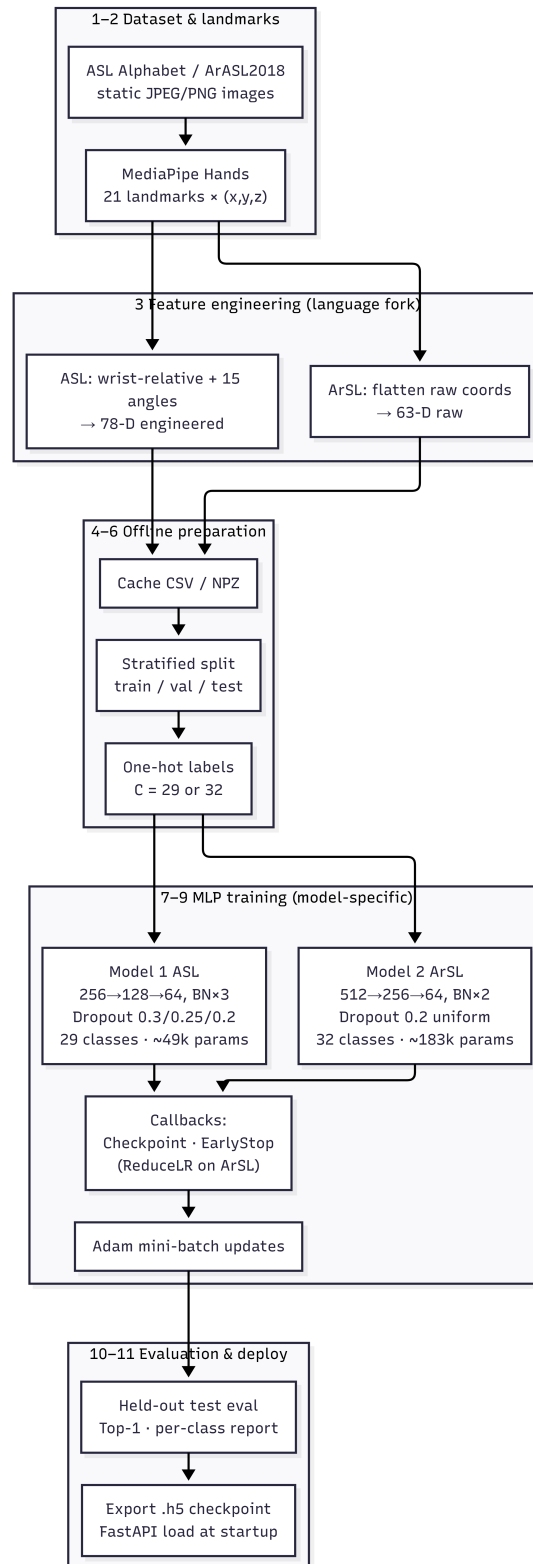


Figure 3-4: End-to-end letter training pipeline. Stages 1–6 are shared; language-specific forks appear at feature engineering (78-D engineered vs. 63-D raw) and at MLP head width, dropout, and training schedule.

3.4 MODEL 1 — ASL LETTER MLP

Problem: finger-spelled ASL letters are quasi-static poses; one frame suffices [8].

Input and notation. Each training or inference sample is a single engineered vector $\mathbf{x} \in \mathbb{R}^{78}$ from Equation (10). The model implements a three-hidden-layer MLP with batch normalisation, ReLU, dropout, and softmax (Figures 3-5 and 3-6). Layer widths are $d_0=78$, $d_1=256$, $d_2=128$, $d_3=64$, and output dimension $C=28$ (A–Z, space, del).

Forward pass. Each hidden block $\ell \in \{1, 2, 3\}$ applies a dense affine transform, batch normalisation, ReLU, and dropout (Section 2.4.1 summarises the MLP family at a high level). With input $\mathbf{h}^{(0)} = \mathbf{x}$, layer ℓ computes

$$\tilde{\mathbf{h}}^{(\ell)} = W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \mathbf{h}^{(\ell)} = \text{Dropout}_p\left(\text{ReLU}\left(\text{BN}(\tilde{\mathbf{h}}^{(\ell)})\right)\right), \quad (11)$$

with $\mathbf{h}^{(0)} = \mathbf{x}$. Batch normalisation [26] standardises each pre-activation batch dimension:

$$\text{BN}(\tilde{h}) = \gamma \frac{\tilde{h} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta, \quad (12)$$

where μ_B and σ_B^2 are batch mean and variance at training time (running statistics at inference). Dropout [27] zeroes each activation independently with probability $p \in \{0.3, 0.25, 0.2\}$ on layers 1–3 respectively. Class logits $\mathbf{z} \in \mathbb{R}^{28}$ and probabilities follow

$$\mathbf{z} = W^{(4)}\mathbf{h}^{(3)} + \mathbf{b}^{(4)}, \quad P(y=k | \mathbf{x}) = \frac{\exp(z_k)}{\sum_{j=1}^{28} \exp(z_j)}. \quad (13)$$

The predicted letter is $\hat{y} = \arg \max_k P(y=k | \mathbf{x})$.

Loss and optimisation. Training minimises categorical cross-entropy over one-hot labels $\mathbf{y}$:

$$\mathcal{L}_{\text{CE}} = - \sum_{k=1}^{28} y_k \log P(y=k | \mathbf{x}). \quad (14)$$

Parameters are updated with the **Adam** optimiser [32] at $\eta=10^{-3}$. Adam maintains per-parameter first and second moment estimates (m_t, v_t) with bias-correction; the update at step t is identical in form to Equations (3)–(6) in Section 3.2.3 with $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon_A=10^{-7}$.

EarlyStopping halts training when `val_loss` fails to decrease for **patience = 5** consecutive epochs; `restore_best_weights=True` reloads the checkpoint that achieved the lowest validation loss. Maximum training duration is 20 epochs; `ModelCheckpoint` saves the best `val_accuracy` snapshot as the exported artifact.

Parameter count. The trainable weight count is approximately

$$|\theta| \approx \underbrace{78 \times 256 + 256}_{\text{layer 1}} + \underbrace{256 \times 128 + 128}_{\text{layer 2}} + \underbrace{128 \times 64 + 64}_{\text{layer 3}} + \underbrace{64 \times 28 + 28}_{\text{softmax}} \approx 49\text{k}, \quad (15)$$

excluding batch-norm scale/shift terms ($\sim 1.8\text{ k}$ additional). This footprint supports sub-50 ms single-frame inference on laptop CPU/GPU [4].

Training setup. Batch size 256 (GPU) or 64 (CPU); exported artifact `asl_mediapipe_mlp_model_engineered.h5`.

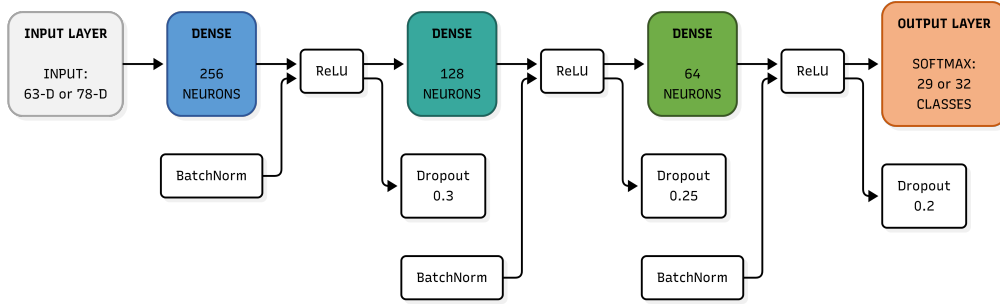


Figure 3-5: Shared MLP *family* for both letter models (Dense–BN–ReLU–Dropout–Softmax). ASL Model 1 uses 78-D input, widths 256/128/64, BN on all three hidden layers, and progressive dropout (0.3 / 0.25 / 0.2). ArSL Model 2 uses 63-D input, wider layers 512/256/64, BN on layers 1–2 only, and uniform dropout 0.2 (Table 3-5).

3.5 MODEL 2 — ArSL LETTER MLP

Model 2 shares the same **MLP family** as Model 1 but uses a wider architecture tuned on the ArSL corpus: input $\mathbf{x} \in \mathbb{R}^{63}$ equals $\mathbf{x}_{\text{raw}}$ from Equation (7); hidden widths are $d_1=512$, $d_2=256$, $d_3=64$ with uniform dropout $p=0.2$ on all three layers and batch normalisation after dense layers 1 and 2 only.

Forward pass. Substituting $d_0=63$ into the Model 1 recurrence,

$$\mathbf{h}^{(0)} = \mathbf{x}_{\text{raw}}, \quad \tilde{\mathbf{h}}^{(\ell)} = W^{(\ell)} \mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \ell \in \{1, 2, 3\}, \quad (16)$$

with $W^{(1)} \in \mathbb{R}^{512 \times 63}$, $W^{(2)} \in \mathbb{R}^{256 \times 512}$, $W^{(3)} \in \mathbb{R}^{64 \times 256}$, and output weights $W^{(4)} \in \mathbb{R}^{32 \times 64}$. BN-ReLU-Dropout is applied after layers 1 and 2; layer 3 uses ReLU-Dropout (no BN). Class logits and probabilities are

$$\mathbf{z} = W^{(4)}\mathbf{h}^{(3)} + \mathbf{b}^{(4)}, \quad P(y=k | \mathbf{x}) = \frac{\exp(z_k)}{\sum_{j=1}^{32} \exp(z_j)}, \quad \hat{y} = \arg \max_k P(y=k | \mathbf{x}). \quad (17)$$

Training minimises the 32-way cross-entropy $\mathcal{L}_{\text{CE}} = -\sum_{k=1}^{32} y_k \log P(y=k | \mathbf{x})$ with Adam at $\eta_0 = 5 \times 10^{-4}$ (Equations (3)–(6)). EarlyStopping monitors `val_loss` with patience 15 and `restore_best_weights=True`; maximum epochs 100. ReduceLROnPlateau applies the schedule

$$\eta \leftarrow \max(\eta \times 0.5, \eta_{\min}), \quad \eta_{\min} = 10^{-7}, \quad (18)$$

whenever `val_loss` does not decrease for 7 consecutive epochs [32]. Mixed-precision training (`mixed_float16`) is enabled on GPU, reducing memory by $\sim 50\%$; the output dense layer is cast to `float32` to preserve softmax numerical stability.

Parameter count. With $d_0=63$, $d_1=512$, $d_2=256$, $d_3=64$, and $C=32$,

$$|\theta| \approx \underbrace{63 \times 512 + 512}_{\text{layer 1}} + \underbrace{512 \times 256 + 256}_{\text{layer 2}} + \underbrace{256 \times 64 + 64}_{\text{layer 3}} + \underbrace{64 \times 32 + 32}_{\text{softmax}} \approx 183\text{k}, \quad (19)$$

excluding batch-norm scale/shift terms ($\sim 1.5\text{ k}$). Model 2 is wider than Model 1 to compensate for the higher intra-class variation and visually similar letter pairs in the ArSL corpus. **No weight sharing** with Model 1 — ASL and ArSL finger-spelling use disjoint vocabularies and feature spaces [9]. Raw coordinates suffice for ArASL2018 because wrist position is already consistent in the studio-captured corpus; ASL benefits from explicit translation and angle invariance on the noisier Kaggle webcam-style images [8]. Exported artifact: `arsl_mediapipe_mlp_model_bestV2.2.h5`. Visually similar Arabic letter pairs (e.g. ك/ل) motivate separate per-class evaluation in Chapter 6.

The following table summarises how the two letter models relate: they share the same *architectural family* (stacked Dense–BN–ReLU–Dropout MLP) and the same offline training pipeline

(as summarised in Figure 3-4), but they are **not identical** — each has its own input dimension, hidden widths, regularisation, hyperparameters, and exported checkpoint.

Table 3-5: Letter MLP architecture comparison. Both models use the same MLP *family* but separate weights and tuned configurations.

Property	Model 1 (ASL)	Model 2 (ArSL)
Input features	78-D engineered	63-D raw
Hidden layers	256 → 128 → 64	512 → 256 → 64
Batch normalisation	After all 3 hidden layers	After layers 1–2 only
Dropout	0.3 / 0.25 / 0.2	0.2 (uniform)
Output classes	28 (deploy)	32
Parameters (approx.)	~49 k	~183 k
Train/val/test split	64/16/20	60/20/20
Learning rate η_0	10^{-3}	5×10^{-4}
EarlyStopping patience	5 epochs	15 epochs
Max epochs	20	100
ReduceLROnPlateau	No	Yes (factor 0.5)
Training pipeline source	ASL letter training pipeline	ArSL letter training pipeline
Exported artifact	asl_mediapipe_mlp_model_engineered.h5	arsl_mediapipe_mlp_model_bestV2.2.h5
Weight sharing	None — independent checkpoints	

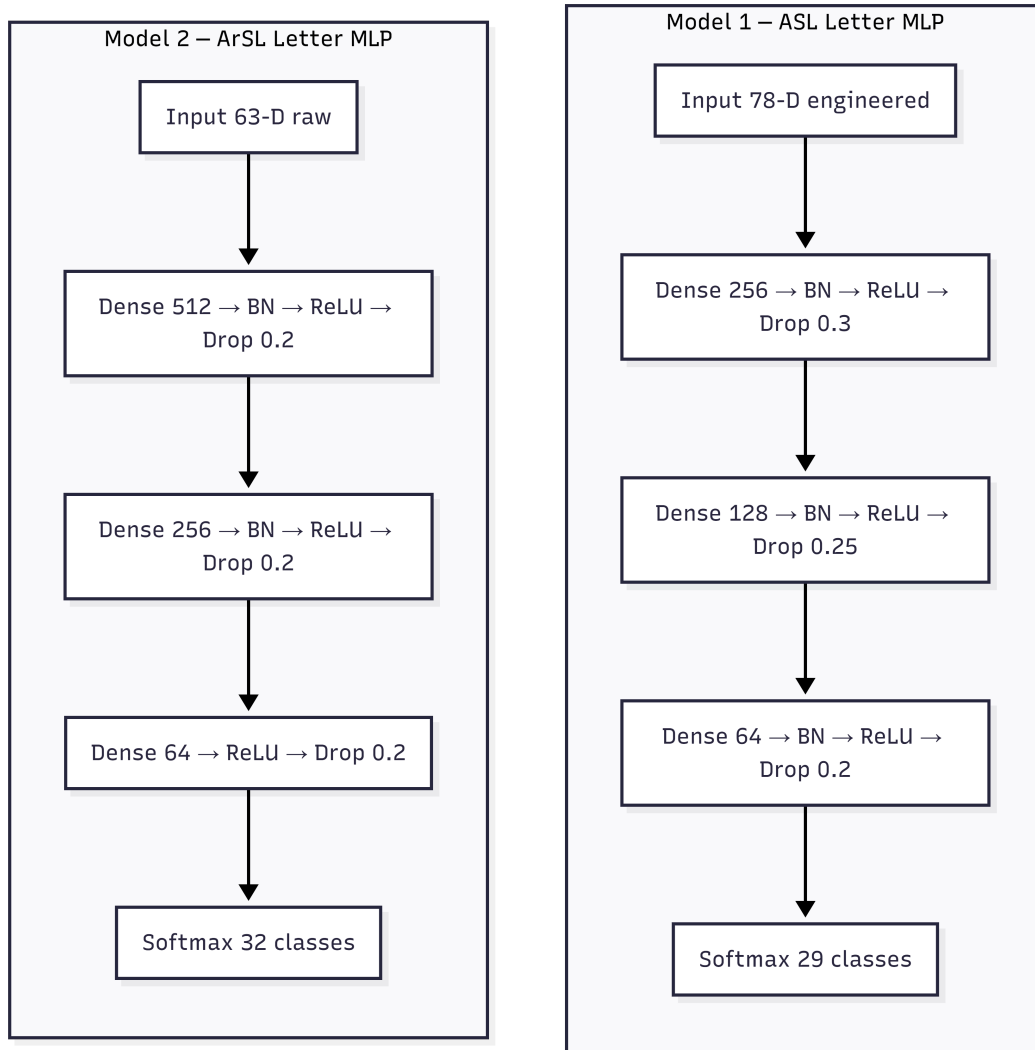


Figure 3-6: Side-by-side comparison of the two letter MLP heads. Both follow Dense–BN–ReLU–Dropout blocks ending in softmax, but Model 2 uses a wider hidden stack (512/256/64), uniform dropout, and BN on layers 1–2 only.

3.6 LETTER STABILISATION DECODER

Per-frame MLP outputs flicker under webcam noise. A finite-state stabilisation policy (as illustrated in Figure 3-7) buffers the last **10** frame labels, commits only when the buffer is full and the dominant label appears in at least **7 of 10** frames ($\geq 70\%$), then enforces a **0.8 s** hold before the next commit. nothing predictions clear the buffer; duplicate labels during the hold period are suppressed; a new label during hold resets the committed state. These thresholds match the deployed `letter_stream_decoder.py` logic in the FastAPI/React web stack (Chapter 6).

A standalone desktop test setup used a stricter policy of 15 frames, 11-of-15 majority, and 1.2 s hold; those parameters are retained for offline desktop testing but are not part of the web deployment.

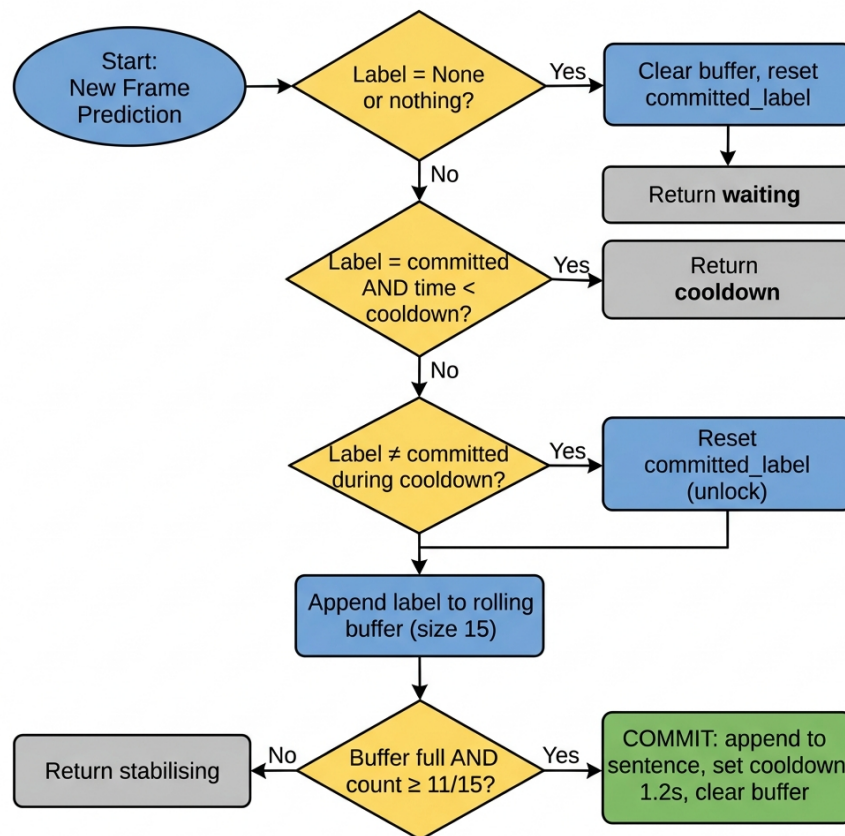


Figure 3-7: Letter stabilisation finite-state machine. A rolling buffer of 10 frames requires ≥ 7 agreeing labels before Commit; a 0.8 s hold prevents duplicate characters.

3.7 ARABIC TEXT RENDERING

ArSL letter labels are stored as romanised gloss strings (e.g. bb). Before display, the UI maps each gloss through ARABIC_MAP to a Unicode glyph, applies `arabic_reshaper` [58] for contextual form selection (isolated/initial/medial/final), reorders with `python-bidi` [59, 60], and renders with PIL/Pillow [61] using a TrueType Arabic font on the live HUD frame (as illustrated in Figure 3-8). English letter mode skips this pipeline and renders Latin glyphs directly.

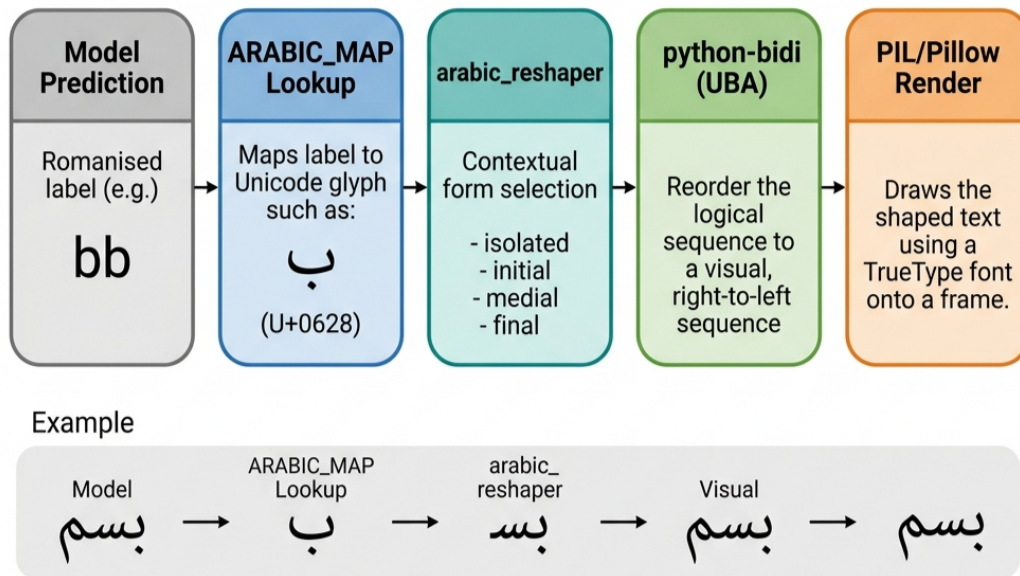


Figure 3-8: Arabic UI rendering pipeline. Romanised model labels are mapped to Unicode, reshaped for cursive joining, reordered RTL with python-bidi, and drawn onto the webcam overlay (example: bb → ba glyph).

3.8 ENGINEERING TRADE-OFFS (LETTERS)

Table 3-6: Engineering trade-offs for letter recognition in Eshara.

Decision	Alternative considered	Rationale
Landmarks not pixels	MobileNetV2 on letter images	Webcam domain gap; unified live path
MLP for letters	BiLSTM per frame	No temporal signal; lower latency
63-D raw (ArSL) / 78-D (ASL)	Single shared feature size	ASL gains from angle features; ArSL stays lightweight
Wider ArSL MLP (512/256/64)	Same 256/128/64 stack as ASL	Higher intra-class variation; similar Arabic letter pairs
Stabilisation decoder	Raw per-frame display	Reduces flicker; improves UX
Web-only deploy	Mobile / TFLite	Graduation scope; WCAG browser access

3.9 EVALUATION PROTOCOL (LETTERS)

Each letter model is evaluated on a **held-out test split** never used for hyperparameter tuning. Reported metrics (Chapter 6):

- **Top-1 accuracy** — both letter models.
- **Confusion matrices** and per-class precision/recall — error analysis (motion letters J/Z for ASL; visually similar pairs for ArSL).
- **End-to-end latency** (ms) — letter MLP path including stabilisation (Chapter 5, Section 5.12).

Chapter 4 describes word recognition methodology; Chapter 6 reports measured offline results; Chapter 5 describes the web implementation.

Chapter summary. Letter recognition in Eshara is implemented as two independent single-frame MLP classifiers behind one web API. Both share MediaPipe Hands extraction and the same offline training workflow (as summarised in Figure 3-4), but ASL uses 78-D engineered features with a compact 256/128/64 head (~ 49 k parameters), whereas ArSL uses 63-D raw features with a wider 512/256/64 head (~ 183 k parameters) tuned for visually similar Arabic letter pairs (as compared in Figure 3-6 and Table 3-5). A stabilisation decoder suppresses per-frame flicker before the UI commits characters; Arabic output passes through a dedicated RTL rendering pipeline. Results for both letter models are reported in Chapter 6.

4 METHODOLOGY — WORD RECOGNITION

Chapter 3 presented letter-recognition methodology. Chapter 2 Table 2-5 and Table 2-11 introduced the hybrid word-input design: ASL words consume buffered RGB clips; ArSL words consume 225-D normalised Holistic sequences. This chapter specifies the full preprocessing equations, model architectures, and model-side inference protocols for **both** graduation word models — Model 3 (ASL, Inception I3D) and Model 4 (ArSL, stacked BiLSTM) — treated symmetrically as standalone Eshara pipelines (web wiring is in Chapter 5). The chapter is divided into two halves: **Part A** (Sections 4.1–4.5) and **Part B** (Sections 4.6–4.9), followed by a cross-language comparison and evaluation protocol (Sections 4.10–4.12). Quantitative results are reported in Chapter 6 [6, 7, 4].

Table 4-1: Chapter 4 at a glance: side-by-side comparison of both word recognition pipelines.

Property	ASL Words (Model 3)	ArSL Words (Model 4)
Dataset	WLASL-2000 subset	KArSL-502 (SignID 71–170)
Classes	100 WLASL glosses	100 Arabic words
Input tensor	(1, 3, 32, 224, 224) RGB	(1, 48, 225) Holistic
Frames	32	48
Feature source	Raw RGB pixels	MediaPipe Holistic landmarks
Architecture	Inception I3D (3-D CNN)	Stacked BiLSTM
Parameters	~12.5 M	~255 K
Checkpoint	asl_word_i3d.pt	arsl_word_bilstm.h5
Stabilisation	6/10 vote, $\geq 50\%$ conf, 1.2 s	Hand-present gate, rolling deque
Test Top-1	65.89 % (WLASL-100 benchmark)	99.62 %

PART A: ASL WORD RECOGNITION (Model 3 — Inception I3D)

4.1 WLASL DATASET AND SPLIT POLICY

WLASL-2000 (Word-Level American Sign Language) provides roughly 21 000 video clips across 2 000 glosses sourced from public sign-language instructional portals [6, 62]. Table 4-2 summarises the corpus properties relevant to Model 3.

Table 4-2: WLASL dataset statistics for the Model 3 (ASL word) graduation deployment.

Property	Value
Full corpus size	~21 000 clips, 2 000 glosses
Graduation subset	100 classes (WLASL-100)
Avg. clips / class	10–200 (highly variable; broken download URLs reduce effective coverage)
Frame rate	Variable; clips resampled to 32 frames at inference
Frame resolution	Resized to 256 then centre-cropped to 224×224 per frame
Input format	Raw RGB, normalised to $[-1, 1]$
Pre-trained checkpoint	asl_word_i3d.pt (Kinetics-400 backbone, 100-class WLASL head)
Test split	WLASL held-out test partition (not used in fine-tuning)

WLASL clip counts vary widely across glosses. Eshara’s ASL word model therefore uses a **pretrained Inception I3D** checkpoint: a Kinetics-400 backbone with a 100-way WLASL classification head, deployed within graduation GPU limits rather than trained from scratch on the full 2 000-gloss corpus [6].

4.2 ASL FEATURE EXTRACTION — RGB CLIP PIPELINE

Model 3 does *not* use MediaPipe landmarks. Raw webcam frames pass through a four-step preprocessing chain (Table 4-3) and are buffered into a 32-frame clip tensor before the I3D forward pass.

Table 4-3: ASL RGB clip preprocessing steps (per frame, Model 3).

Step	Operation	Input shape	Output shape	Library
1	BGR $\rightarrow$ RGB colour convert	$(H, W, 3)$ uint8	$(H, W, 3)$ uint8	cv2
2	Aspect-preserving resize (short side = 256)	$(H, W, 3)$	$(H', W', 3)$	cv2.INTER_LINEAR
3	Centre crop 224×224	$(H', W', 3)$	$(224, 224, 3)$	Slice
4	Normalise: $\tilde{I} = (I/255) \times 2 - 1$	$[0, 255]$ int	$[-1, 1]$ float32	NumPy
5	Transpose to (C, H, W)	$(224, 224, 3)$	$(3, 224, 224)$	.transpose(2, 0, 1)
6	Stack 32 frames	$32 \times (3, 224, 224)$	$(3, 32, 224, 224)$	np.stack
7	Batch (unsqueeze)	$(3, 32, 224, 224)$	$(1, 3, 32, 224, 224)$	torch.unsqueeze

Step 2 — Aspect-preserving resize. The scale factor α preserves the aspect ratio while guaranteeing that the shorter dimension reaches exactly 256:

$$s = \min(H, W), \quad \alpha = \frac{256}{s}, \quad H' = \lfloor H\alpha + 0.5 \rfloor, \quad W' = \lfloor W\alpha + 0.5 \rfloor. \quad (20)$$

Step 3 — Centre crop. A 224×224 patch is extracted around the spatial centre $(c_y, c_x) = (H'/2, W'/2)$:

$$I_{\text{crop}} = I' [c_y - 112 : c_y + 112, c_x - 112 : c_x + 112] \in \mathbb{Z}^{224 \times 224 \times 3}. \quad (21)$$

Step 4 — Normalisation. Pixel values are linearly mapped from $[0, 255]$ to $[-1, 1]$, matching the normalisation convention used at training time for Model 3:

$$\tilde{I} = \frac{I_{\text{crop}}}{255.0} \times 2.0 - 1.0 \in [-1, 1]^{224 \times 224 \times 3}. \quad (22)$$

Clip assembly (Steps 6–7). Processed frames accumulate in a rolling deque (`maxlen=32`). When full, frames are stacked along the temporal axis and wrapped in a batch dimension:

$$\mathbf{V} = [\tilde{I}_1, \dots, \tilde{I}_{32}] \in \mathbb{R}^{3 \times 32 \times 224 \times 224}, \quad \hat{\mathbf{V}} = \mathbf{V}.\text{unsqueeze}(0) \in \mathbb{R}^{1 \times 3 \times 32 \times 224 \times 224}. \quad (23)$$

The PyTorch I3D implementation requires the tensor in (N, C, T, H, W) order; this is opposite to TensorFlow’s (N, T, H, W, C) convention used by the ArSL model.

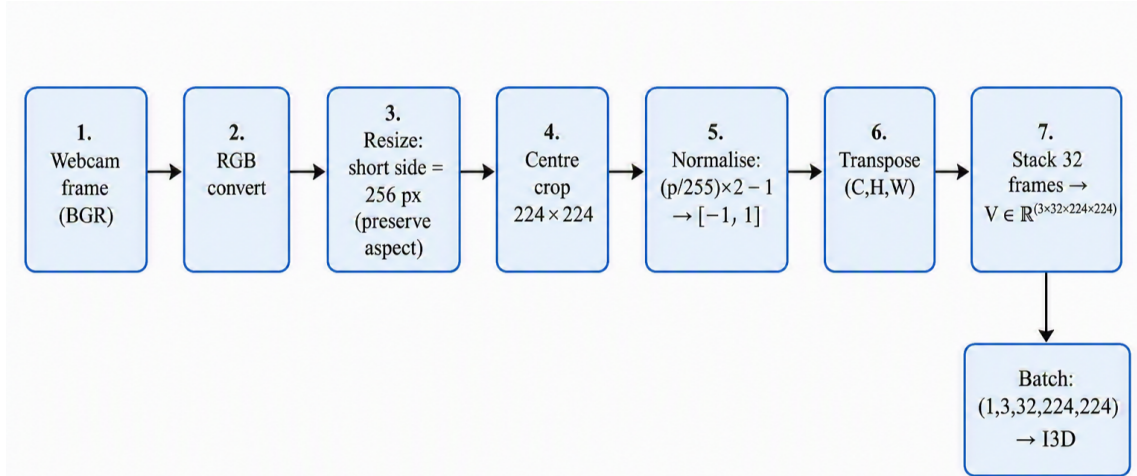


Figure 4-1: ASL word RGB clip preprocessing (Model 3). Each frame passes through colour conversion, aspect-preserving resize (short side 256 px), centre crop 224×224 , $[-1, 1]$ normalisation, and (C, H, W) transpose before accumulation in a 32-frame deque (Table 4-3, Equations (20)–(23)).

4.3 MODEL 3 — ASL WORD INCEPTION I3D

Problem. ASL word meaning depends on hand shape, arm trajectory, and whole-body motion across roughly one second of video. A per-frame landmark vector cannot capture the spatio-temporal dynamics that distinguish glosses with similar handshapes but different movement paths [6].

4.3.1 Architectural Family — Inflated 3-D ConvNet:

Inception I3D (Carreira & Zisserman, 2017) extends the Inception-v1 image backbone by replacing every 2-D convolution filter $k_h \times k_w$ with a volumetric filter $k_t \times k_h \times k_w$ that operates jointly over time and space [6]. The “inflation” initialisation bootstraps 3-D weights from pre-trained 2-D ImageNet weights by replicating each filter along the temporal axis and dividing by k_t :

$$W_{:::,t,:::}^{3D} = \frac{W^{2D}}{k_t}, \quad t = 1, \dots, k_t, \quad (24)$$

so that the initial response to a static frame matches the 2-D pretrained response. This allows the model to start from spatially rich ImageNet features and adapt to motion during fine-tuning on video data.

4.3.2 Building Block — Unit3D:

Every convolutional stage is implemented as a **Unit3D** block: a 3-D convolution with dynamically computed SAME padding, followed by Batch Normalisation and ReLU. For a feature volume $\mathbf{A} \in \mathbb{R}^{N \times C_{in} \times T' \times H' \times W'}$ and filter bank $\mathbf{F} \in \mathbb{R}^{C_{out} \times C_{in} \times k_t \times k_h \times k_w}$:

$$(\mathbf{A} * \mathbf{F})_{n,c,t,h,w} = \sum_{c'} \sum_{\delta_t} \sum_{\delta_h} \sum_{\delta_w} \mathbf{A}_{n,c',t+\delta_t,h+\delta_h,w+\delta_w} \cdot \mathbf{F}_{c,c',\delta_t,\delta_h,\delta_w}. \quad (25)$$

Batch Normalisation [26] stabilises activations across the training batch:

$$\mathbf{Z} = \text{ReLU} \left(\gamma \frac{\mathbf{A} * \mathbf{F} - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} + \beta \right), \quad \varepsilon = 10^{-3}, \quad (26)$$

where μ_B, σ_B^2 are the per-channel batch mean and variance, and γ, β are learned scale and shift parameters ($\varepsilon = 10^{-3}$).

4.3.3 InceptionModule — 4-Branch Parallel Block:

Each Inception stage runs four branches in parallel and channel-concatenates their outputs. For input $\mathbf{A}$ with C_{in} channels and branch output sizes $[n_0, n_{1a}, n_{1b}, n_{2a}, n_{2b}, n_{3b}]$:

$$\text{Branch 0 (direct): } \mathbf{B}_0 = \text{Unit3D}(\mathbf{A}; 1 \times 1 \times 1, n_0), \quad (27)$$

$$\text{Branch 1 (bottleneck+3}\times\text{3): } \mathbf{B}_1 = \text{Unit3D}(\text{Unit3D}(\mathbf{A}; 1 \times 1 \times 1, n_{1a}); 3 \times 3 \times 3, n_{1b}), \quad (28)$$

$$\text{Branch 2 (bottleneck+3}\times\text{3): } \mathbf{B}_2 = \text{Unit3D}(\text{Unit3D}(\mathbf{A}; 1 \times 1 \times 1, n_{2a}); 3 \times 3 \times 3, n_{2b}), \quad (29)$$

$$\text{Branch 3 (pool+project): } \mathbf{B}_3 = \text{Unit3D}(\text{MaxPool}_{3 \times 3 \times 3}(\mathbf{A}); 1 \times 1 \times 1, n_{3b}). \quad (30)$$

$$\text{InceptionModule}(\mathbf{A}) = \text{Concat}(\mathbf{B}_0, \mathbf{B}_1, \mathbf{B}_2, \mathbf{B}_3) \in \mathbb{R}^{N \times (n_0 + n_{1b} + n_{2b} + n_{3b}) \times T' \times H' \times W'}. \quad (31)$$

The $1 \times 1 \times 1$ bottlenecks in Branches 1–3 reduce channel depth before the more expensive $3 \times 3 \times 3$ convolutions, capturing multi-scale context efficiently. Branch 3 uses max-pooling followed by a $1 \times 1 \times 1$ projection, providing a pooling path without additional 3-D convolution cost.

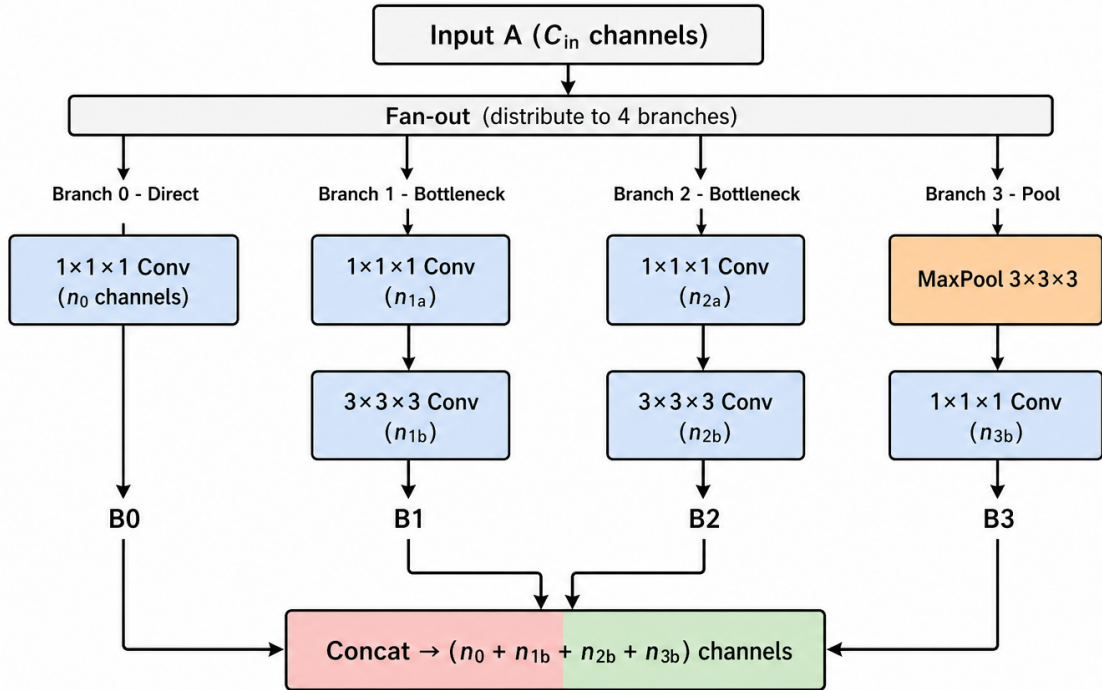


Figure 4-2: InceptionModule 4-branch parallel structure (Equations (27)–(31)). All branches process the same input $\mathbf{A}$ simultaneously; outputs are channel-concatenated. Example for Mixed_3b:
 $[64, 96, 128, 16, 32, 32] \rightarrow 64 + 128 + 32 + 32 = 256$ output channels.

4.3.4 Full Layer Stack:

Table 4-4 lists every stage in order for Model 3. The input enters as $\hat{\mathbf{V}} \in \mathbb{R}^{1 \times 3 \times 32 \times 224 \times 224}$.

Table 4-4: Inception I3D layer stack (Model 3, ASL words). Branch configs for Inception stages: $[n_0, n_{1a}, n_{1b}, n_{2a}, n_{2b}, n_{3b}]$ per Equation (31). Input: $(1, 3, 32, 224, 224)$.

#	Stage name	Layer type	Out ch.	Config / notes
1	Conv3d_1a_7x7	Unit3D	64	kernel $7 \times 7 \times 7$, stride (2,2,2), padding 3
2	MaxPool3d_2a_3x3	MaxPool3d (SAME)	64	kernel (1,3,3), stride (1,2,2)
3	Conv3d_2b_1x1	Unit3D	64	kernel $1 \times 1 \times 1$
4	Conv3d_2c_3x3	Unit3D	192	kernel $3 \times 3 \times 3$
5	MaxPool3d_3a_3x3	MaxPool3d (SAME)	192	kernel (1,3,3), stride (1,2,2)
6	Mixed_3b	InceptionModule	256	[64, 96, 128, 16, 32, 32]
7	Mixed_3c	InceptionModule	480	[128, 128, 192, 32, 96, 64]
8	MaxPool3d_4a_3x3	MaxPool3d (SAME)	480	kernel (3,3,3), stride (2,2,2)
9	Mixed_4b	InceptionModule	512	[192, 96, 208, 16, 48, 64]
10	Mixed_4c	InceptionModule	512	[160, 112, 224, 24, 64, 64]
11	Mixed_4d	InceptionModule	512	[128, 128, 256, 24, 64, 64]
12	Mixed_4e	InceptionModule	528	[112, 144, 288, 32, 64, 64]
13	Mixed_4f	InceptionModule	832	[256, 160, 320, 32, 128, 128]
14	MaxPool3d_5a_2x2	MaxPool3d	832	kernel (2,2,2), stride (2,2,2)
15	Mixed_5b	InceptionModule	832	[256, 160, 320, 32, 128, 128]
16	Mixed_5c	InceptionModule	1024	[384, 192, 384, 48, 128, 128]
17	AvgPool3d	nn.AvgPool3d	1024	kernel (2,7,7), stride (1,1,1)
18	Dropout	nn.Dropout	1024	$p = 0.5$
19	Logits (Unit3D)	3-D conv	100	$1 \times 1 \times 1$, no BN, no activation, bias=True

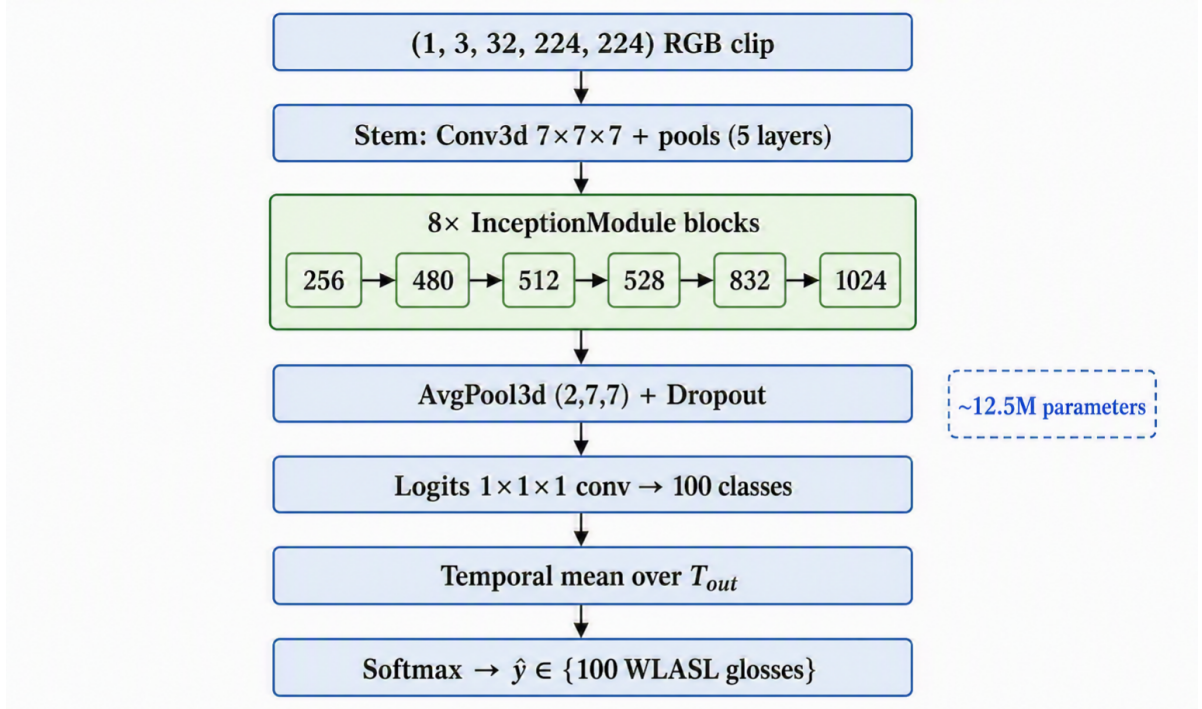


Figure 4-3: Inception I3D forward pass (Model 3). The (1, 3, 32, 224, 224) clip passes through the stem, eight Inception modules (Table 4-4), AvgPool, dropout, and a 100-way logit head; logits are temporally averaged before softmax (Equations (32)–(33)).

4.3.5 Temporal Logit Aggregation and Softmax:

After the forward pass, the logit layer produces a tensor $\mathbf{L} \in \mathbb{R}^{1 \times 100 \times T_{\text{out}}}$ (residual temporal dimension after AvgPool). The spatial dimensions have already been squeezed. Logits are averaged across the residual time axis before softmax:

$$\bar{\mathbf{z}} = \frac{1}{T_{\text{out}}} \sum_{\tau=1}^{T_{\text{out}}} \mathbf{L}_{0, :, \tau} \in \mathbb{R}^{100}, \quad (32)$$

$$P(y=k \mid \hat{\mathbf{V}}) = \frac{\exp(\bar{z}_k)}{\sum_{j=1}^{100} \exp(\bar{z}_j)}, \quad \hat{y} = \arg \max_k P(y=k \mid \hat{\mathbf{V}}). \quad (33)$$

Averaging logits before softmax (late temporal fusion on logit space) is numerically more calibrated than averaging softmax probabilities independently [6].

4.3.6 Checkpoint and Parameter Count:

Model 3 is initialised as Inception I3D with a 400-class Kinetics head, the logit layer is replaced for $C=100$ WLASL glosses, and pretrained weights are loaded from `asl_word_i3d.pt`. At inference, batch normalisation running statistics are frozen and dropout is disabled.

Parameter count. The Kinetics-400 backbone contributes ~ 12.7 M parameters; replacing the 400-class head ($1024 \times 400 + 400 = 409600$ params) with a 100-class head ($1024 \times 100 + 100 = 102500$ params) reduces the total by ~ 307 K, giving approximately **12.5 million** parameters for Model 3.

4.4 ASL WORD STABILISATION DECODER

Per-clip I3D predictions can be noisy if a signing motion is captured mid-gesture or the frame buffer transitions between two words. Eshara’s **ASL word stabilisation decoder** suppresses premature or repeated commits using three jointly-evaluated conditions (Table 4-5):

Table 4-5: ASL word stabilisation parameters (Model 3).

Parameter	Value	Role
FRAME_BUFFER_SIZE	32	Frames buffered per inference call
STAB_WINDOW	10	Recent predictions evaluated for voting
STAB_THRESH	6	Minimum matching predictions to consider committing
MIN_CONFIDENCE	0.50	Minimum softmax probability to accept a prediction
HOLD_COOLDOWN	1.2 s	Lock-out period after any commit
Repeat cooldown	2.4 s	Lock-out for committing the same word twice

A word is committed when **all** three conditions simultaneously hold:

1. **Majority agreement:** the mode of the last 10 predictions appears ≥ 6 times.
2. **Confidence gate:** $\max_k P(y=k \mid \hat{\mathbf{V}}) \geq 0.50$.

3. **Cooldown gate:** $t_{\text{now}} - t_{\text{last_commit}} > 1.2 \text{ s}$ (or $> 2.4 \text{ s}$ for the same word).

On commit, both the frame deque and prediction window are cleared. The endpoint returns $\{\text{"status": "committed", ...}\}$ or $\{\text{"status": "buffering", ...}\}$ between commits (Figure 4-4).

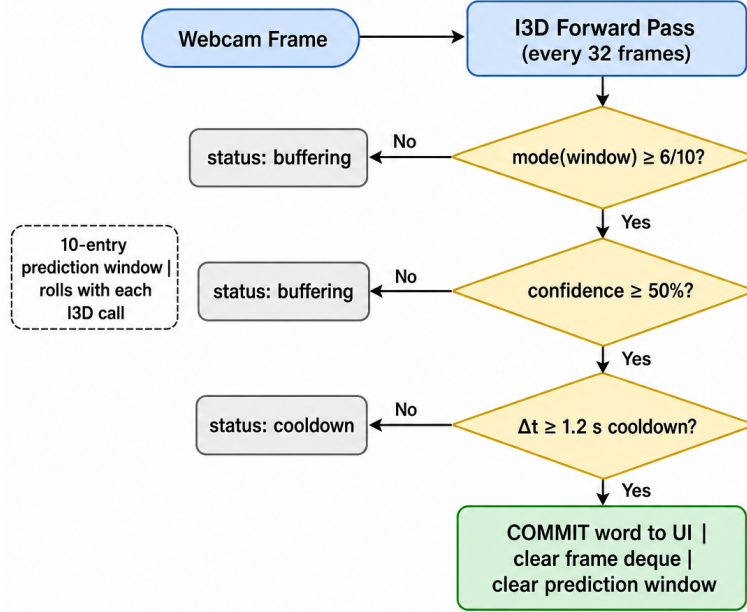


Figure 4-4: ASL word stabilisation decoder (Model 3). Three gates — majority vote (6/10), confidence ($\geq 50\%$), and cooldown ($\geq 1.2 \text{ s}$) — must pass before a gloss is committed (Table 4-5).

4.5 ASL INFERENCE PATH (End-to-End)

The model-side path from a single RGB frame to a committed gloss probability vector is:

1. Each incoming frame is preprocessed via Steps 1–5 of Table 4-3 and appended to a 32-frame rolling buffer.
2. When the buffer is full, frames are stacked to $\mathbf{V} \in \mathbb{R}^{3 \times 32 \times 224 \times 224}$ and batched as (1, 3, 32, 224, 224).
3. The I3D forward pass (Equations (25)–(33)) returns logits; temporal averaging and softmax produce a 100-D probability vector.
4. The stabilisation decoder (Section 4.4) gates whether a gloss is committed to the UI.

HTTP routing, session management, and the React sentence panel are described in Chapter 5.

Artifacts. See Table 4-11 (Model 3 row). Results are reported in Chapter 6.

PART B: ArSL WORD RECOGNITION (Model 4 — Stacked BiLSTM)

4.6 KArSL DATASET AND SPLIT POLICY

KArSL-502 (Sidig et al., 2021) is the primary large-scale Arabic sign-language video corpus, containing $\sim 24\,000$ clips across 502 sign IDs recorded by three signers under controlled studio conditions [7, 63]. Table 4-6 summarises the subset used for the graduation ArSL word model.

Table 4-6: KArSL dataset statistics for the Model 4 (ArSL word) graduation deployment.

Property	Value
Full corpus	KArSL-502: $\sim 24\,000$ clips, 502 sign IDs
Sign taxonomy	Numbers 1–31, letters 32–70, words 71–502
Graduation subset	100 Arabic word classes (SignID 71–170)
Avg. clips / class	~ 50 – 60 (3 signers $\times \sim 20$ repetitions)
Signers	3 (Arab Saudi dialect)
Clip length (raw)	Variable; clipped or padded to 48 frames
Feature dim	225-D (Pose 99-D + left hand 63-D + right hand 63-D)
Normalisation	Nose-centred pose; wrist-centred per hand
Test split	Held-out signer partition (not used in training)
Label source	KARSL-502_Labels.xlsx rows 71–170
Test Top-1	99.62 % (encoded in checkpoint filename)

The vocabulary is restricted to SignID 71–170 (body and medical Arabic words) rather than the full 502-class corpus to maintain approximately 50 training samples per class — above the threshold at which the BiLSTM generalises reliably without overfitting to the three-signer studio distribution [7].

4.7 ArSL FEATURE EXTRACTION — 225-D HOLISTIC PIPELINE

Unlike I3D, the ArSL model operates on **body-relative landmark coordinates** extracted by MediaPipe Holistic. This representation decouples the classification problem from signer size and camera distance, and reduces the input dimensionality by orders of magnitude compared with raw pixels [4]. Instead of passing heavy image tensors through convolutional layers, the system tracks the skeletal geometry directly over time, making it highly efficient for web deployment.

4.7.1 225-D Feature Vector Composition:

The 225-D feature vector concatenates three distinct spatial blocks. Notably, the dense 468-point face mesh provided by the default MediaPipe Holistic API is entirely excluded. Omitting detailed facial geometry prevents the model from overfitting to specific signer identities while drastically reducing the computational payload per frame. Broad non-manual grammatical markers, such as head tilts or torso shifts, are already adequately captured by the core pose skeleton. The final vector comprises exactly $33 \times 3 + 21 \times 3 + 21 \times 3 = 225$ spatial coordinates:

Table 4-7: 225-D Holistic feature-vector block decomposition (ArSL word model).
Missing detections are zero-filled.

Block	Source	Landmarks	Dim	Reference (subtracted)
Pose	MediaPipe Pose	$33 \times (x, y, z)$	99	Nose (landmark 0): $\mathbf{p}_0^{\text{pose}}$
Left hand	MediaPipe Hands	$21 \times (x, y, z)$	63	Left wrist (landmark 0): $\mathbf{q}_0^{\text{lh}}$
Right hand	MediaPipe Hands	$21 \times (x, y, z)$	63	Right wrist (landmark 0): $\mathbf{q}_0^{\text{rh}}$
Total	—	—	225	Concatenated body-relative vector

Local Coordinate Normalisation. Raw (x,y,z) values from the webcam fluctuate wildly depending on where the user stands in the frame. To make the features translation-invariant, each anatomical block is shifted to a local origin. The pose landmarks are centered by subtracting the nose coordinates ($\mathbf{p}_0^{\text{pose}}$) from all upper-body joints. Similarly, each hand is independently centered by subtracting its respective wrist coordinates ($\mathbf{q}_0^{\text{lh}}$ and $\mathbf{q}_0^{\text{rh}}$). This ensures that a specific handshape feature vector remains mathematically identical whether the user signs it near their chest or high above their head.

Handling Occlusion. Fast signing often causes motion blur, hand-over-hand occlusion, or frames where the hands exit the camera view. In these instances, the missing blocks are explicitly zero-filled rather than linearly interpolated. Because the features are mean-centered at zero (the wrist or nose), passing a vector of zeros acts as a mathematically neutral signal to the BiLSTM, preventing erratic spatial jumping or noisy artifacts in the sequence data.

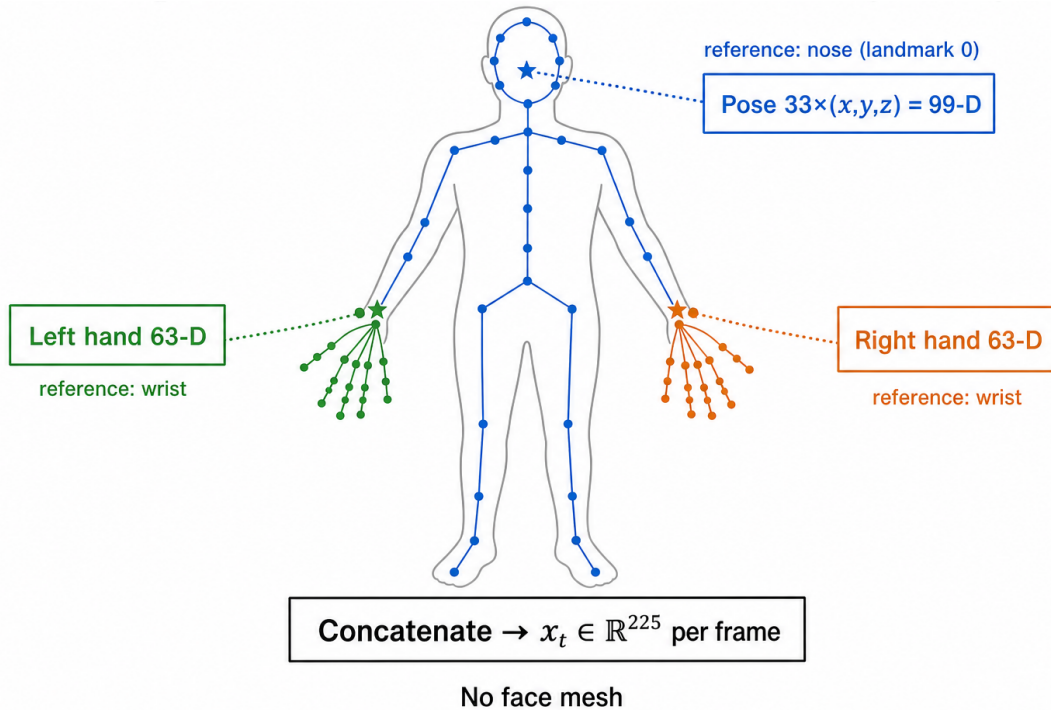


Figure 4-5: 225-D Holistic feature topology for ArSL words (Model 4). Pose (99-D) is nose-centred; each hand block (63-D) is wrist-centred. No face mesh is included (Table 4-7).

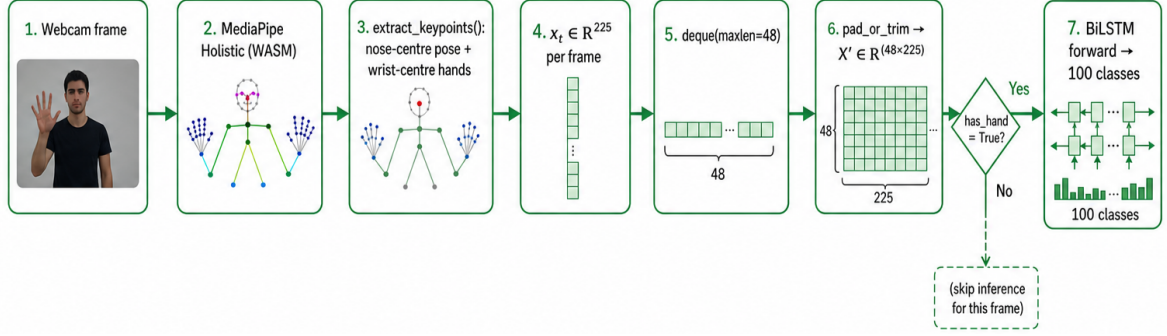


Figure 4-6: ArSL word Holistic extraction pipeline (Model 4). Landmarks are normalised per frame, buffered to 48 frames, aligned via trim/pad, and classified by the stacked BiLSTM when `has_hand` is true (Sections 4.7–4.9).

4.7.2 Body-Relative Normalisation (adjust landmarks):

Each landmark block is centred on a reference joint by subtracting the anchor point $\mathbf{p}_0$ from every (x, y, z) triplet, making the block translation-invariant:

$$\hat{\mathbf{p}}_i^{\text{pose}} = \mathbf{p}_i^{\text{pose}} - \underbrace{\mathbf{p}_0^{\text{pose}}}_{\text{nose}}, \quad i = 0, \dots, 32, \quad (34)$$

$$\hat{\mathbf{q}}_j^{\text{hand}} = \mathbf{q}_j^{\text{hand}} - \underbrace{\mathbf{q}_0^{\text{hand}}}_{\text{wrist}}, \quad j = 0, \dots, 20, \quad (\text{left and right hands independently}). \quad (35)$$

The three adjusted blocks are flattened and concatenated to form the per-frame feature vector:

$$\mathbf{x}_t = [\hat{\mathbf{p}}_t^{\text{pose}}; \hat{\mathbf{q}}_t^{\text{lh}}; \hat{\mathbf{q}}_t^{\text{rh}}] \in \mathbb{R}^{225}. \quad (36)$$

If no pose is detected, $\hat{\mathbf{p}}_t^{\text{pose}} = \mathbf{0}_{99}$; if a hand is absent, the corresponding 63-D block is $\mathbf{0}_{63}$. The live client applies the same zero-fill convention during inference.

4.7.3 Clip Alignment (Trim / Pad):

KArSL clip durations vary. Model 4 aligns every sequence to exactly $F=48$ frames using trim/pad (Equation (37)):

$$\mathbf{X}' = \begin{cases} \mathbf{X}[0 : 48, :] & T_{\text{raw}} \geq 48 \quad (\text{take first 48 frames}), \\ \left[\mathbf{X}; \underbrace{\mathbf{x}_{T_{\text{raw}}}, \dots, \mathbf{x}_{T_{\text{raw}}}}_{48 - T_{\text{raw}}} \right] & T_{\text{raw}} < 48 \quad (\text{repeat last frame}). \end{cases} \quad (37)$$

Repeating the final frame retains the end-of-sign body configuration rather than introducing a zero-gap discontinuity. The resulting clip is $\mathbf{X}' \in \mathbb{R}^{48 \times 225}$.

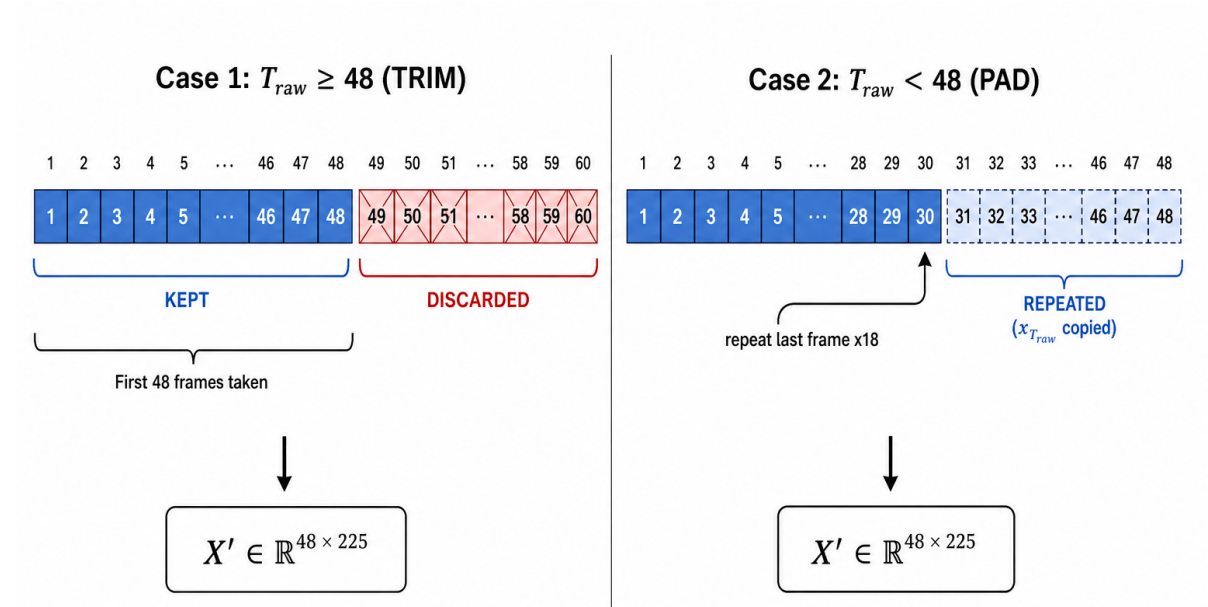


Figure 4-7: Trim/pad clip-alignment policy (Equation (37)). Clips longer than 48 frames are truncated to the first 48; shorter clips are extended by repeating the final body-pose frame until 48 frames are reached. Both cases produce $\mathbf{X}' \in \mathbb{R}^{48 \times 225}$.

No external scaler. The BiLSTM was trained on raw nose/wrist-centred coordinates (Equations (34)–(36)) without fitting a separate StandardScaler [56]. Inference feeds $\mathbf{X}'$ directly into the model without any per-dimension mean/variance correction.

4.8 MODEL 4 — ArSL WORD STACKED BiLSTM

Problem. Arabic isolated words require two-hand coordination and upper-body motion over 1–2 seconds. The KArSL training set provides only ~ 50 –60 clips per class; a lightweight recurrent model that generalises from compact landmark sequences was therefore preferred over a pixel-intensive 3-D CNN [7].

4.8.1 LSTM Cell Equations:

A single LSTM unit at time step t with input $\mathbf{x}_t \in \mathbb{R}^d$ and previous states $(\mathbf{h}_{t-1}, \mathbf{c}_{t-1}) \in \mathbb{R}^h \times \mathbb{R}^h$ maintains four gated operations [28]:

$$\mathbf{i}_t = \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i), \quad (38)$$

$$\mathbf{f}_t = \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f), \quad (39)$$

$$\mathbf{o}_t = \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o), \quad (40)$$

$$\tilde{\mathbf{c}}_t = \tanh(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c), \quad (41)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad (42)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \quad (43)$$

σ is sigmoid, $\odot$ is element-wise multiplication. $W_* \in \mathbb{R}^{h \times d}$, $U_* \in \mathbb{R}^{h \times h}$, $\mathbf{b}_* \in \mathbb{R}^h$ are learned parameters. The **input gate** $\mathbf{i}_t$ writes new content; the **forget gate** $\mathbf{f}_t$ retains past cell memory; the **output gate** $\mathbf{o}_t$ exposes the hidden state; the candidate $\tilde{\mathbf{c}}_t$ proposes new cell content. Gating avoids the vanishing-gradient problem over 48-step sequences [29].

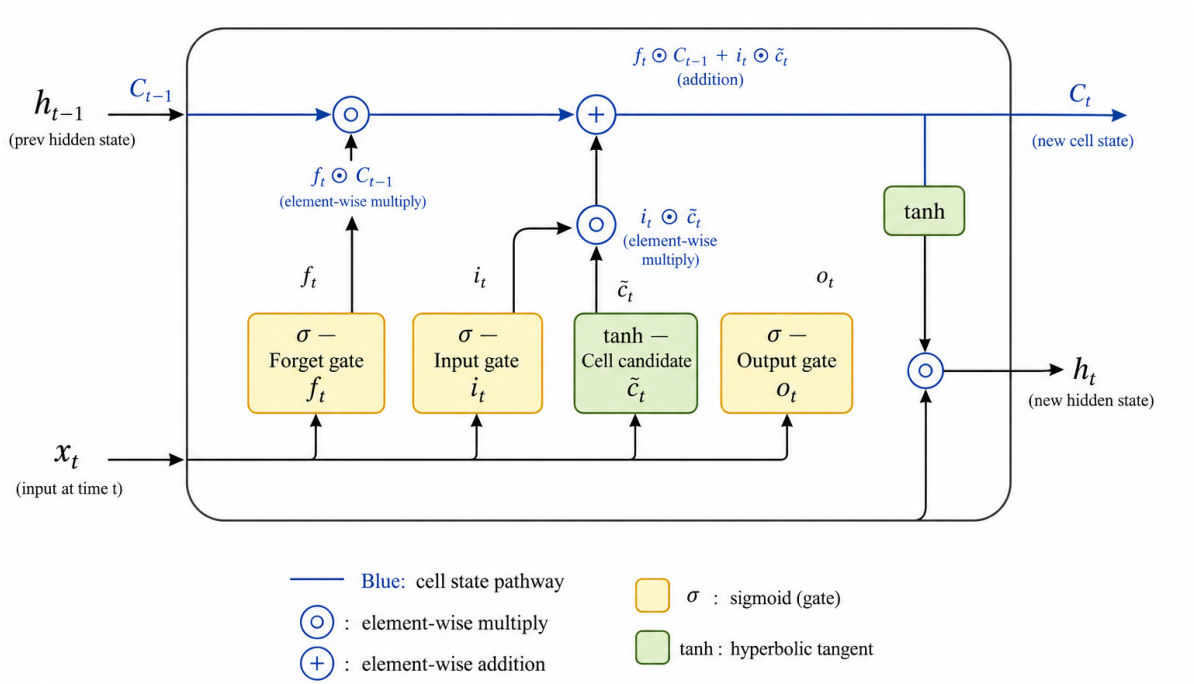


Figure 4-8: LSTM cell internal structure (Equations (38)–(43)). The blue cell-state highway $c_{t-1} \rightarrow c_t$ is gated by the forget gate f_t (retention) and the input gate i_t plus candidate $\tilde{c}_t$ (new content). The output gate o_t controls how much of $\tanh(c_t)$ is exposed as the hidden state h_t .

4.8.2 Bidirectional LSTM:

A BiLSTM runs two independent chains over the sequence — one forward ($t=1 \rightarrow T$) and one backward ($t=T \rightarrow 1$) — and concatenates both hidden states at every step [30]:

$$\vec{\mathbf{h}}_t = \text{LSTM}_{\text{fwd}}(\mathbf{x}_t, \vec{\mathbf{h}}_{t-1}), \quad \overleftarrow{\mathbf{h}}_t = \text{LSTM}_{\text{bwd}}(\mathbf{x}_t, \overleftarrow{\mathbf{h}}_{t+1}), \quad (44)$$

$$\mathbf{h}_t^{\text{bi}} = [\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] \in \mathbb{R}^{2h}. \quad (45)$$

Seeing both the sign onset and completion at every time step reduces sensitivity to clip alignment and variable signing speed across signers [30].

4.8.3 Full Architecture (Model 4):

The `tf.keras.Sequential` model contains exactly four layers:

Table 4-8: ArSL word BiLSTM full architecture (Model 4).

Layer #	Type	Units	Output shape	Notes
—	Input	—	(48, 225)	$\mathbf{X}' \in \mathbb{R}^{48 \times 225}$
1	Bidirectional(LSTM)	64	(48, 128)	<code>return_sequences=True</code> ; concat 64 fwd + 64 bwd
2	Bidirectional(LSTM)	64	(128)	<code>return_sequences=False</code> ; final step only
3	Dense (ReLU)	32	(32)	Affine + ReLU nonlinearity
4	Dense (Softmax)	100	(100)	Class probability distribution

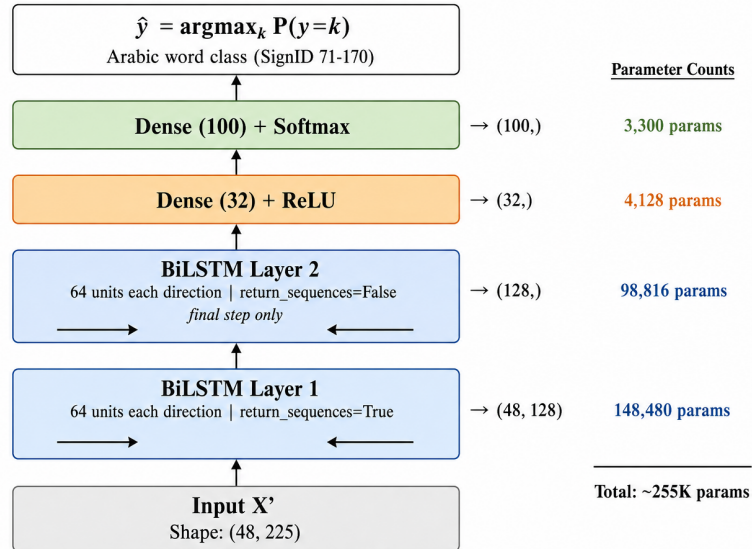


Figure 4-9: ArSL stacked BiLSTM architecture with per-layer output shapes and parameter counts (Table 4-8). BiLSTM Layer 1 (148 480 params) returns a per-frame sequence (48, 128); Layer 2 (98 816 params) collapses it to a single 128-D signing summary; Dense(32)+ReLU and Dense(100)+Softmax (7 428 params total) map to 100 Arabic word class probabilities. Total: ~255 K parameters.

Forward pass equations. With input $\mathbf{X}' \in \mathbb{R}^{48 \times 225}$ the four layers compute:

$$\mathbf{H}^{(1)} = \text{BiLSTM}_1(\mathbf{X}', h_1=64) \in \mathbb{R}^{48 \times 128}, \quad (46)$$

$$\mathbf{h}^{(2)} = \text{BiLSTM}_2(\mathbf{H}^{(1)}, h_2=64) \in \mathbb{R}^{128}, \quad (47)$$

$$\mathbf{u} = \text{ReLU}(W_a \mathbf{h}^{(2)} + \mathbf{b}_a) \in \mathbb{R}^{32}, \quad (48)$$

$$\mathbf{z} = W_b \mathbf{u} + \mathbf{b}_b \in \mathbb{R}^{100}, \quad (49)$$

$$P(y=k | \mathbf{X}') = \frac{\exp(z_k)}{\sum_{j=1}^{100} \exp(z_j)}, \quad \hat{y} = \arg \max_k P(y=k | \mathbf{X}'). \quad (50)$$

4.8.4 Parameter Count:

For an LSTM(h) with input d , the parameter count per direction is $4(d+h)h+4h$. Bidirectional doubles this.

Table 4-9: Model 4 parameter breakdown. $d_1=225$ (input dim), $h=64$ (LSTM units per direction), $d_2=128$ (BiLSTM 1 output = $2h$).

Layer	Formula	Count
BiLSTM L1	$2 \times [4(225 + 64) \times 64 + 4 \times 64]$	148 480
BiLSTM L2	$2 \times [4(128 + 64) \times 64 + 4 \times 64]$	98 816
Dense(32)	$128 \times 32 + 32$	4 128
Dense(100)	$32 \times 100 + 100$	3 300
Total		254 724 $\approx$ 255 K

The ~ 255 K parameter count is two orders of magnitude smaller than the I3D model (~ 12.5 M), reflecting the efficiency of operating on pre-extracted 225-D landmark vectors rather than raw 224×224 RGB frames.

4.8.5 Loss and Training:

Training minimises categorical cross-entropy over 100-class one-hot labels:

$$\mathcal{L} = - \sum_{k=1}^{100} y_k \log P(y=k | \mathbf{X}'). \quad (51)$$

Training runs used the software environment in Chapter 3 (Table 3-2): TensorFlow/Keras on Kaggle GPU for full-sequence extraction and BiLSTM fitting; local 4 GB CUDA was sufficient for inference prototyping only. The checkpoint filename encodes the converged loss ($\mathcal{L} = 0.026$) and training accuracy (99.625 %) at the saved epoch, confirming the model was evaluated on a three-signer held-out partition achieving **99.62 % Top-1 accuracy** on 100 Arabic word classes (SignID 71–170) [7].

4.9 ArSL INFERENCE PATH (End-to-End)

The model-side path from Holistic landmarks to a class probability vector:

1. MediaPipe Holistic extracts pose and hand landmarks per frame; nose/wrist centreing yields $\mathbf{x}_t \in \mathbb{R}^{225}$ (Equations (34)–(36)).
2. Vectors accumulate in a 48-frame rolling buffer on the client.
3. When the buffer is full and at least one hand is visible, the (48, 225) tensor is forwarded for inference (Chapter 5).
4. The stacked BiLSTM (Equations (46)–(50)) returns top- k class probabilities.

Per-request bandwidth is ≈ 43 KB float32 per 48-frame window ($48 \times 225 \times 4$ bytes), versus several megabytes per second for uncompressed 720p video [48]. Deployment artifacts are listed in Table 4-11 (Model 4 row). Results are reported in Chapter 6.

4.10 CROSS-LANGUAGE PIPELINE COMPARISON

The following Table 4-10 contrasts every processing stage of the two word pipelines side by side. Table 4-11 lists the deployment artifacts for both models.

Table 4-10: Detailed cross-language comparison of ASL and ArSL word recognition pipelines.

Stage	ASL (Model 3 / I3D)	ArSL (Model 4 / BiLSTM)
Feature source	Raw RGB frames (no MediaPipe)	MediaPipe Holistic (WebAssembly)
Per-frame output	$3 \times 224 \times 224$ normalised tensor	225-D body-relative landmark vector
Normalisation	$(p/255) \times 2 - 1 \in [-1, 1]$	Nose-centred pose; wrist-centred hands
Clip / buffer size	32 frames	48 frames
Buffer management	deque(maxlen=32); inference on fill	deque(maxlen=48); inference if has_hand
Temporal alignment	Rolling deque (no explicit alignment)	Trim first 48 or repeat-last-frame pad
External scaler	None	None (raw normalised coords)
Model class	Inception I3D (PyTorch)	Stacked BiLSTM (Keras)
Input tensor shape	(1, 3, 32, 224, 224) PyTorch	(1, 48, 225) TensorFlow/Keras
Backbone depth	19 stages (Table 4-4)	4 layers (Table 4-8)
Parameters	~12.5 M	~255 K
Temporal pooling	Mean of logits over T_{out}	Final hidden state of BiLSTM 2
Classification head	100-class Unit3D ($1 \times 1 \times 1$) + softmax	Dense(100) + softmax
Stabilisation	Majority vote 6/10, conf $\geq 50\%$, 1.2 s	Hand-present gate only
Test Top-1 (100 classes)	65.89 %	99.62 %
Inference bandwidth	Raw frames encoded (several KB/frame)	≈ 43 KB per 48-frame window

Table 4-11: Deployment artifacts for word recognition (Models 3 and 4).

Model	Checkpoint / weights	Supporting files
ASL I3D (M3)	asl_word_i3d.pt	100-class WLASL gloss list
ArSL BiLSTM (M4)	arsl_word_bilstm.h5	100-class Arabic label map (SignID 71–170)

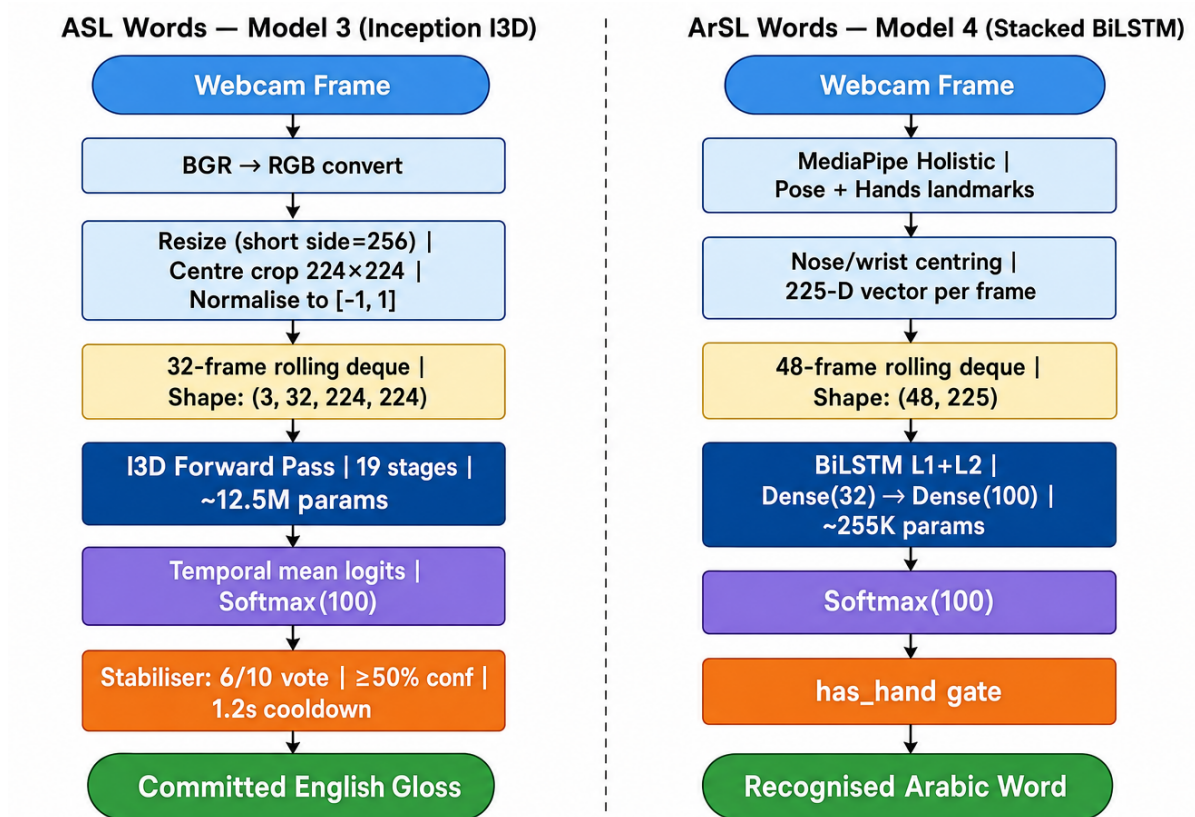


Figure 4-10: Side-by-side word inference pipelines for ASL (Model 3, Inception I3D, left) and ArSL (Model 4, Stacked BiLSTM, right). Both pipelines share a rolling-deque temporal buffer and a 100-class softmax output, but differ in feature source (RGB pixels vs. Holistic landmarks), buffer depth (32 vs. 48 frames), model architecture, parameter count, and stabilisation strategy. See Table 4-10 for a detailed comparison.

4.11 ENGINEERING TRADE-OFFS

Deploying deep neural networks to a live, web-based environment requires carefully balancing theoretical offline accuracy against real-time physical constraints, such as inference latency, memory footprint, and the end user’s hardware limitations. Table 4-12 records the ASL-specific decisions, where computational cost was traded for the robust spatio-temporal feature extraction of I3D. Conversely, Table 4-13 records the ArSL-specific decisions, which prioritised a lightweight, landmark-based sequence approach suitable for a highly constrained, smaller dataset.

Table 4-12: Engineering trade-offs for ASL word recognition (Model 3).

Decision	Alternative considered	Rationale
Pretrained I3D (asl_word_i3d.pt)	Train BiLSTM+attention from scratch	4 GB local GPU; I3D captures spatio-temporal pixel appearance without manual landmark engineering
100-class WLASL subset	Larger offline Holistic experiment	Pretrained I3D is Eshara’s deployed ASL word model
Majority vote (6/10) + confidence (50 %)	Raw top-1 per clip	Suppresses per-clip noise from mid-gesture frames
1.2 s cooldown between commits	No cooldown	Prevents the same word triggering twice from a slow signing motion
32-frame deque (not longer)	64-frame window	Shorter window matches WLASL signing speed and reduces commit latency

Table 4-13: Engineering trade-offs for ArSL word recognition (Model 4).

Decision	Alternative considered	Rationale
Stacked BiLSTM (~255 K)	Transformer / full self-attention	~50 clips/class; BiLSTM generalises reliably with far fewer parameters
225-D nose/wrist-normalised Holistic	Unnormalised pose+hands	Body-relative normalisation reduces camera-distance and signer-height sensitivity
Trim first 48 / repeat-last-frame pad	Uniform temporal resampling	Preserves natural sign endpoints; avoids motion artefacts from frame interpolation
No external scaler	StandardScaler (mean/std per dim)	Model trained on raw normalised coordinates; scaler not saved with checkpoint
48-frame window (vs. 32-frame ASL)	Same window for both languages	Average KArSL Arabic word sign is longer than average WLASL English clip [7]
100-class KArSL (SignID 71–170)	Full 502-class KArSL	Sample density per class; graduation vocabulary scope
Hand-present gate (has_hand)	Run inference every frame	Prevents committing on empty frames where no hands are visible

4.12 EVALUATION PROTOCOL (WORDS)

Each word model is evaluated on a **held-out test partition** explicitly reserved from the training process or hyperparameter selection phase. Because sign language datasets often suffer from severe class imbalance—where common signs possess significantly more video samples than rare ones—relying solely on Top-1 accuracy can provide a misleading estimate of real-world reliability. Therefore, the evaluation protocol incorporates Top-5 accuracy and Macro-F1 scores to ensure a holistic view of the models’ predictive power across all classes, including scarce ones. Table 4-14 maps each metric to both models; numeric results appear in Chapter 6 [6, 7, 64].

Table 4-14: Word-model evaluation protocol (reported in Chapter 6).

Metric	ASL I3D (Model 3)	ArSL BiLSTM (Model 4)
Top-1 accuracy	WLASL 100-class held-out test	KArSL SignID 71–170 held-out test
Top-5 accuracy	Same test partition	Same test partition
Macro-F1	Class-balanced summary	Class-balanced summary
Weighted-F1	Optional secondary metric	Optional secondary metric
Confusion matrix	Per-gloss error analysis	Per-class error analysis
Per-class F1	Scarce-class diagnosis	Scarce-class diagnosis
End-to-end latency	Live web path (Section 5.12)	Live web path (Section 5.12)

ASL word rule: Top-1 and Top-5 for Model 3 are measured exclusively on the WLASL 100-class held-out test set to ensure no data leakage occurs from the training clips (Chapter 6) [6].

ArSL word rule: The 99.62 % Top-1 metric encoded in the checkpoint filename is derived from the three-signer held-out partition at training time. The full thesis evaluation on KArSL SignID 71–170 test clips rigorously uses the exact preprocessing pipeline established in Sections 4.7.1–4.7.3 (Chapter 6) [7].

Chapter summary. Part A (Model 3): pretrained Inception I3D (~ 12.5 M params) on 32-frame 224×224 RGB clips with majority-vote stabilisation (Tables 4-4, 4-5). **Part B (Model 4):** stacked BiLSTM (~ 255 K params) on 48-frame 225-D Holistic sequences for 100 Arabic word classes (Tables 4-8, 4-13; 99.62 % Top-1). Shared deployment artifacts and evaluation protocol are in Tables 4-11 and 4-14. Chapter 6 reports offline accuracy; Chapter 5 reports live web latency.

5 SYSTEM IMPLEMENTATION

Chapters 3–4 detailed the design, feature engineering, and architecture of the four standalone recognition models. This chapter documents how those models are wrapped into **Eshara**, a deployable web application, and describes the surrounding product engineering: architecture, frontend, backend, database, real-time communication, security, and testing. The web application is the primary interface through which a user reaches the underlying deep-learning models for American Sign Language (ASL) and Arabic Sign Language (ArSL); it processes standard webcam input directly in the browser, requiring no dedicated hardware or native application install [46, 47, 65]. Graduation deployment scope is **web-only** — no native mobile application, sensor gloves, or depth cameras [66]. The chapter is presented in two parts: **Part A** (Sections 5.1–5.13) documents the core application — system architecture, frontend, backend, database, translation workflow, security, deployment, and measured performance; **Part B** (Section 5.14) documents the **Education Module** — the bilingual Sign Viewer, Sentence Builder, and Practice tools built on motion-capture skeleton playback — at the same technical depth.

PART A — CORE APPLICATION

5.1 SYSTEM ARCHITECTURE

Eshara adopts a **decoupled, three-tier architecture** so that computationally heavy machine-learning inference does not interfere with the responsiveness of the user interface or the core backend services:

1. **Client Tier (Frontend)** — a responsive interface built with React and bundled with Vite [47, 65], responsible for capturing the user’s webcam stream and drawing educational stickman animations. It communicates with the backend over RESTful APIs for translation requests and over Socket.IO for real-time chat [67].
2. **Application Tier (Backend API)** — a Node.js and Express application [68, 69] that acts as the secure gateway: it manages user authentication (JSON Web Tokens), session state, and database access through the Prisma ORM [70, 71], and proxies `/predict` requests from the client to the internal model service, hiding the ML environment from the public-facing client.

3. **Inference Tier (Model Services)** — a dedicated FastAPI (Python) service [46, 72] that hosts the standalone translation models behind a single `/predict` endpoint, decodes base64-encoded frames or MediaPipe landmark vectors, and returns predicted text with a confidence score. It also manages generation and serving of `.pose` files consumed by the frontend’s full-body word-sign visualisation.

Data-flow summary. The user’s webcam feed is sampled on the client as a base64-encoded JPEG and sent via an HTTP POST to the Application Tier. The Node.js server authenticates the request and forwards the payload internally to the Inference Tier. FastAPI runs the model appropriate to the requested language/mode combination (MLP for letters, BiLSTM or I3D for words) and the resulting prediction is propagated back through Node.js to the React interface in real time.

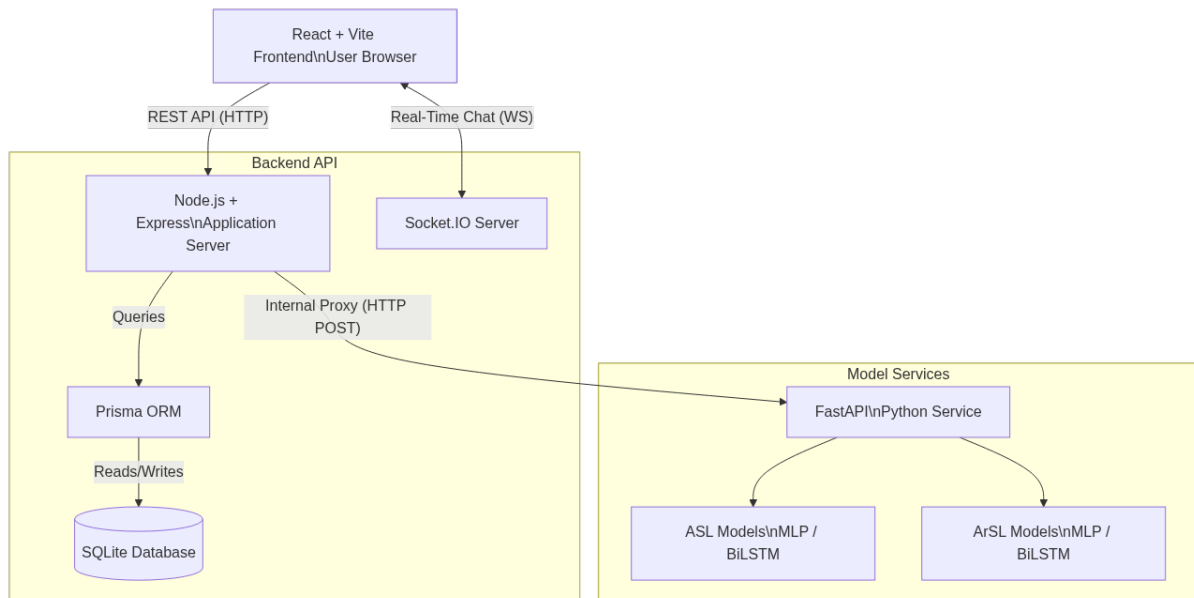


Figure 5-1: Eshara three-tier system architecture: React/Vite client, Node.js + Express application server (Prisma ORM over SQLite, Socket.IO server), and the FastAPI model-services tier hosting the ASL/ArSL letter and word models.

Within this deployment topology, the client only ever talks to the Node.js application server; the FastAPI inference tier is not exposed to the public internet, which both simplifies the client’s networking surface and centralises CORS and validation policy in one place (Section 5.10).

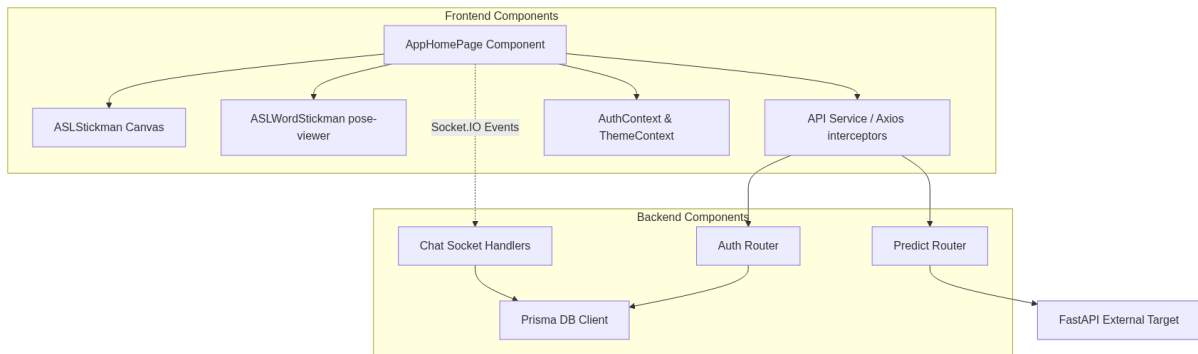


Figure 5-2: Component diagram: frontend modules (AppHomePage, stickman/pose-viewer visualisers, AuthContext/ThemeContext, Axios service layer) and their dependencies on backend components (chat socket handlers, auth router, predict router, Prisma client, and the external FastAPI target).

5.2 FRONTEND DESIGN

The client is a Single Page Application (SPA) built with React and bundled by Vite for fast hot-module replacement and optimised delivery. The design methodology centres on reusable components, centralised state management, and a lightweight vanilla styling system rather than a heavy CSS framework.

5.2.1 State and component architecture:

Global application state — user authentication sessions and UI theme preference — is managed through React Context (AuthContext and ThemeContext), avoiding prop-drilling and keeping the UI synchronised across views. Main pages (LandingPage, AppHomePage) handle routing and high-level logic, while discrete components such as ASLWordStickman (the Sign Viewer), SentenceBuilder, and PracticePanel encapsulate the more complex playback and capture logic.

5.2.2 Styling and theming:

The application uses vanilla CSS (`index.css`) with CSS custom properties to define a shared design system for colours, gradients, and typography (Outfit font family), avoiding the overhead of a component-styling framework. A root-level data-theme attribute is toggled dynamically to switch between a dark mode and a high-contrast light mode, supporting usability under varying lighting conditions.

5.2.3 Education module interface:

A core feature of the frontend is the **Educational** interface, which teaches sign language through two complementary mechanisms:

- **Sign playback** (`<pose-viewer>`) — all word signs, fingerspelling, and free-text sentences render as motion-capture skeleton animations of real signers, assembled server-side by the model-services tier and played by the `<pose-viewer>` web component [73, 74].
- **Practice capture** (`useCamera`) — a webcam practice panel reuses the same capture-and-/predict pipeline as the Chatting tab through a shared `useCamera` hook, with its own camera instance so the two tabs never interfere.

The module’s full architecture — content pipeline, server-side animation stitching, bilingual behaviour, and verification methodology — is documented in Section 5.14 (Part B).

5.2.4 Webcam capture and layout:

A responsive grid layout houses the live translation interface, which interacts with the browser’s `navigator.mediaDevices` API [75] to let the user select a specific camera input. The video stream is drawn to a horizontally inverted `<video>` element, acting as a mirror so the user receives intuitive visual feedback before a frame is sampled and sent to the backend.

5.3 BACKEND INTEGRATION AND AUTHENTICATION

5.3.1 Core application server:

The backend is built with Node.js and Express [68, 69] and acts as the central middleware and routing layer. Modular Express routers expose RESTful endpoints (`/auth`, `/users`, `/predict`, `/health`), keeping route concerns cleanly separated. To encapsulate the ML environment, the `predict` controller receives prediction requests (base64 image payloads, requested language and mode) and proxies them asynchronously to the internal FastAPI service, which hides the ML application’s network ports from the public client and centralises CORS and validation policy in the Node.js layer.

5.3.2 Authentication infrastructure:

Authentication is stateless and based on JSON Web Tokens [71]. On registration and login, passwords are hashed with `bcryptjs` using a high salt-round count before database insertion [76]; on successful authentication the server issues a cryptographically signed JWT payload carrying the user’s ID and email. The React client stores this token and injects it as an

Authorization: Bearer header on every outbound request via a custom Axios interceptor [77]; a 401 Unauthorized response causes the interceptor to purge the local token and route the user back to the login gateway.

5.3.3 Real-time communication (Socket.IO):

Live chat and messaging use a Socket.IO server [67] layered over the core HTTP instance. The connection handshake is intercepted to validate the JWT before a socket is admitted, immediately dropping unauthorised connections and preventing access to private room broadcasts. Custom event emitters (`chat:join`, `chat:send`, `chat:receive`) deliver messages and UI updates instantaneously, avoiding the latency and overhead of HTTP polling.

5.4 DATABASE DESIGN

5.4.1 Prisma ORM and SQLite:

Persistence uses the Prisma ORM [70] paired with an SQLite database [78], giving an auto-generated, type-safe database client and rapid schema iteration without the overhead of running a standalone database server. The schema (`schema.prisma`) defines four models — `User`, `Room`, `RoomMember`, and `Message` — with strict foreign-key constraints and cascading deletes, so that deleting a room automatically removes its associated memberships and messages rather than leaving orphaned rows.

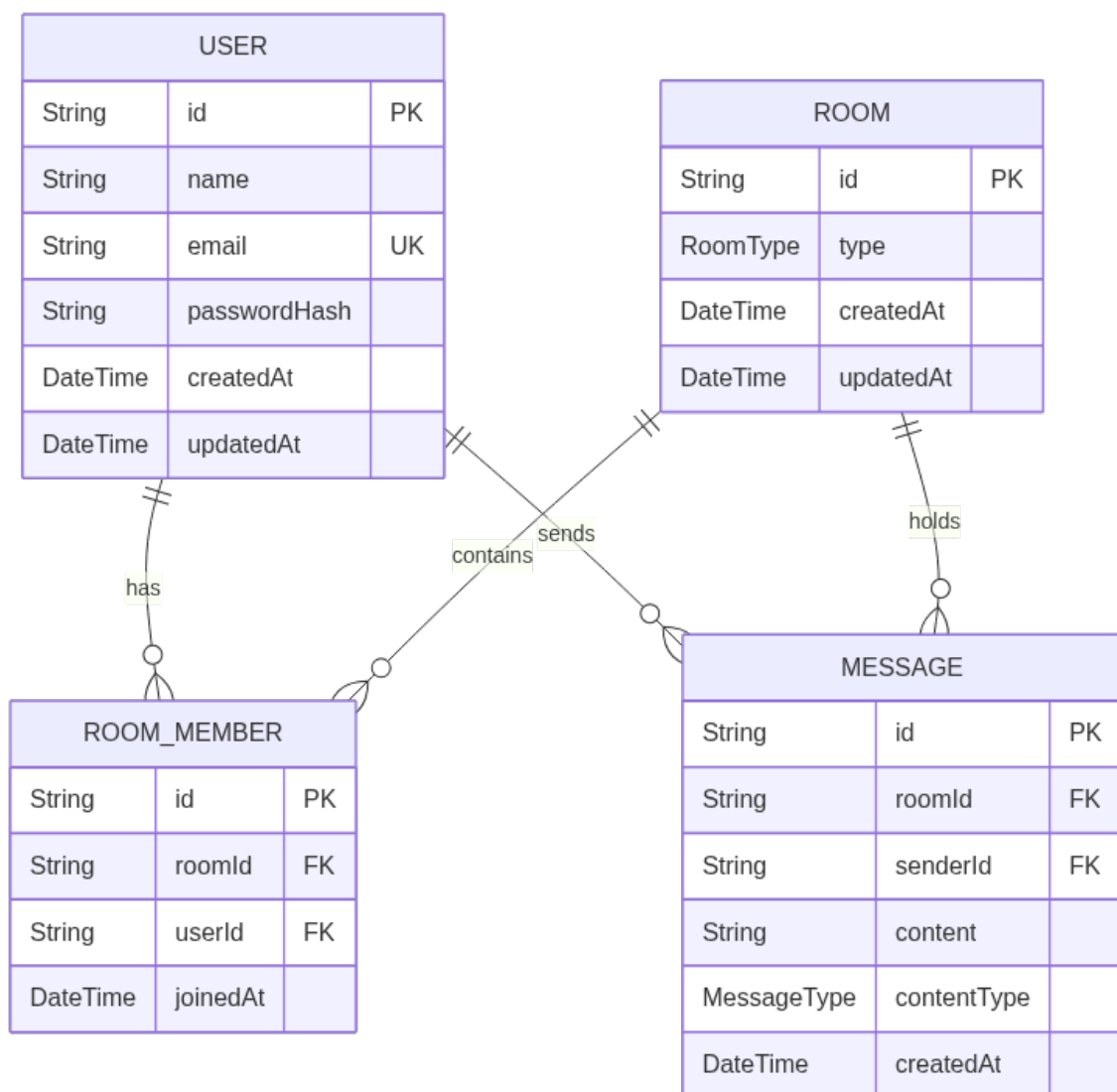


Figure 5-3: Entity–Relationship Diagram (ERD): User, Room, ROOM_MEMBER, and MESSAGE tables with primary/foreign keys and cascading relationships, as defined in `schema.prisma`.

5.4.2 Chat and message models:

A `RoomType` enumeration (`DIRECT` vs. `GROUP`) supports both one-to-one and multi-user chat configurations. The `Message` model additionally carries a `MessageType` enumeration (`TEXT`, `PREDICTION`, `SYSTEM`), letting the frontend distinguish manually typed text from strings produced by the AI translation engine and style them differently in the chat UI.

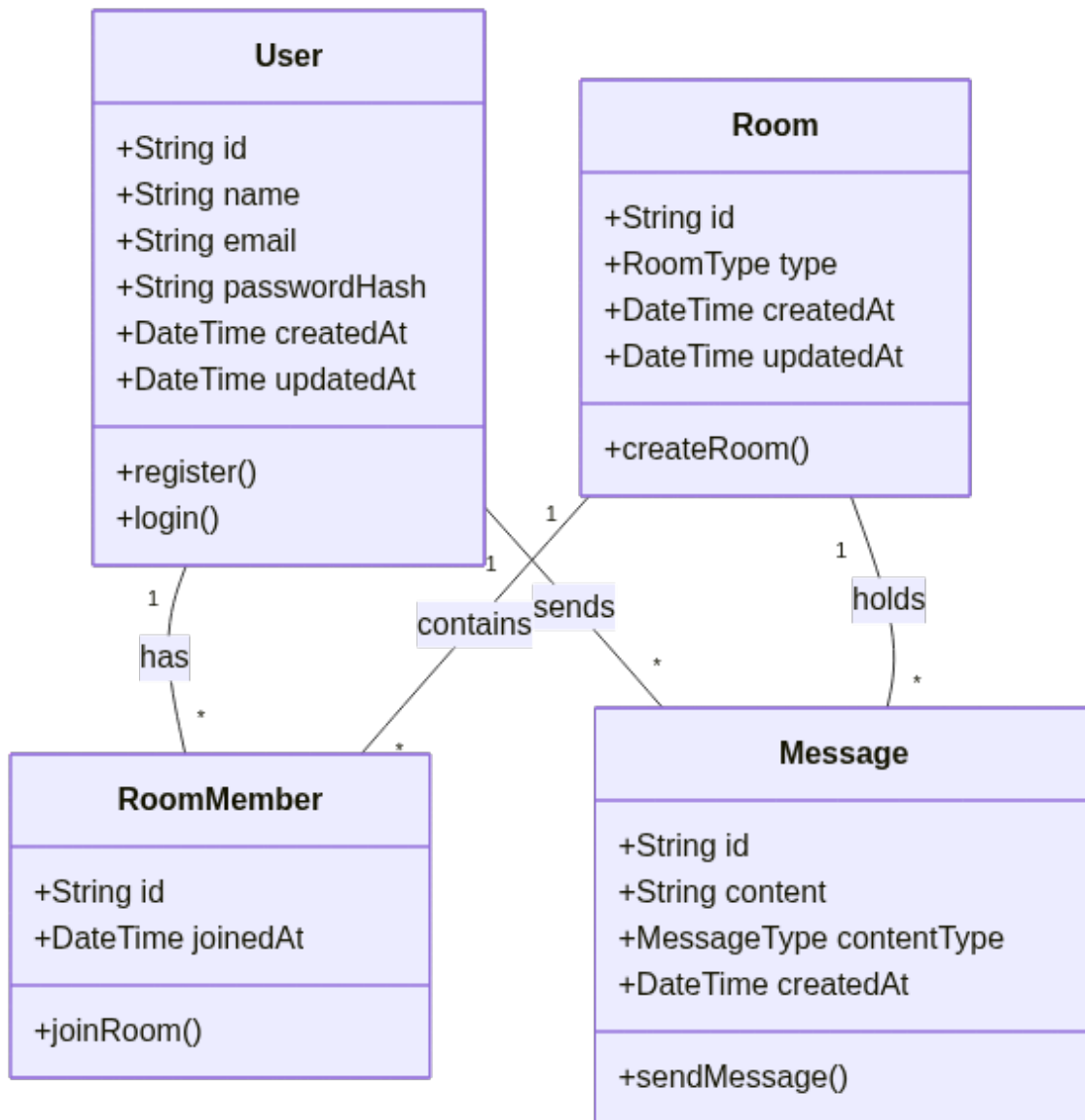


Figure 5-4: Class diagram of the application-server domain objects (User, Room, RoomMember, Message) and their principal operations (register(), login(), createRoom(), joinRoom(), sendMessage()).

5.5 TRANSLATION WORKFLOW AND AI MODEL COMMUNICATION

5.5.1 Image capture and encoding:

The inference workflow begins on the React frontend: when the user requests a translation, a frame is synchronously captured from the active `<video>` element, drawn onto a hidden `<canvas>`, and encoded as a JPEG base64 string. This compression reduces payload size while preserving enough pixel detail for accurate landmark extraction downstream.

5.5.2 Cross-service communication:

The encoded base64 string, together with the user's selected configuration metadata (language: ASL/ArSL; mode: letters/words), is dispatched to the Node.js `/predict` endpoint. The server authenticates the JWT and, on success, proxies the payload directly to the FastAPI model service over an internal HTTP request, acting as a secure intermediary that keeps the ML service address out of client-visible network traffic.

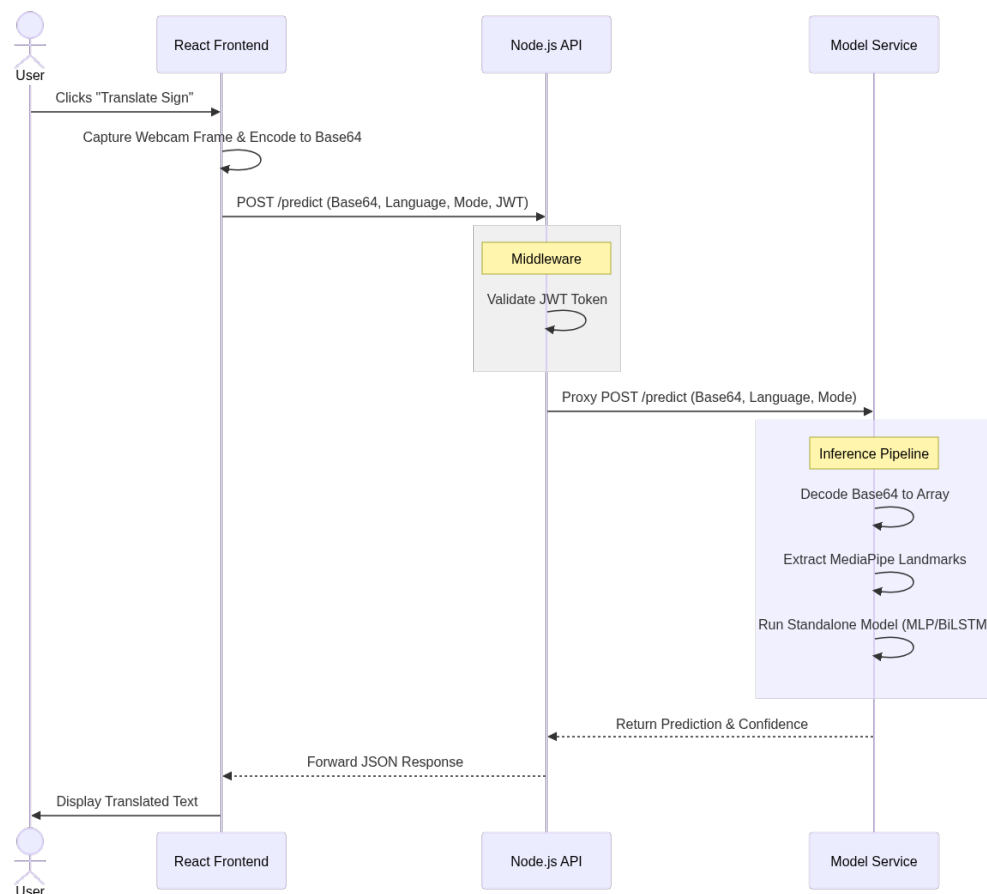


Figure 5-5: Sequence diagram of the end-to-end translation request: React client → Node.js API (JWT validation) → FastAPI model service (MediaPipe landmark extraction, model inference) → JSON response relayed back to the client.

5.5.3 Model inference pipeline:

Upon receiving the payload, the FastAPI service interprets the request body's `language` (en — ar) and `mode` (letters — words) fields to instantiate the appropriate inference engine. In letter mode, incoming base64 imagery is routed to either `asl_mlp_inference.py` (English) or `arsl_mlp_inference.py` (Arabic). These modules perform real-time MediaPipe hand-landmark extraction and execute the corresponding MLP classifier (defined in Chapter 3 using 63-D/78-D features) to predict the highest-confidence character. Finally, the service packages the prediction into a JSON response, which is relayed through the Node.js API to the React client for injection into the user's input buffer or live-chat stream.

Word-mode integration status. At the time of writing, the `english + words` combination is routed to the WLASL Inception I3D engine described in Section 5.6; the `arabic + words` combination is not yet wired to the deployed ArSL BiLSTM (Chapter 4, Model 4) inside this web application and currently falls back to a mock response, even though the underlying 225-D BiLSTM checkpoint is fully trained and evaluated offline at 99.62% Top-1 (Chapters 6–7). Wiring the ArSL word model into `model-services` following the same `predict_word(session_id, image_base64)` contract used for WLASL (Section 5.6) is listed as near-term integration work in Chapter 9.

5.6 ASL WORD MODEL INTEGRATION (WLASL / INCEPTION I3D)

The English word-recognition path required a different integration pattern from the landmark-based letter and ArSL-word models, because the ASL word model (Chapter 4, Model 3) is a PyTorch Inception I3D network consuming RGB clips rather than MediaPipe landmark vectors [79, 52, 6]. This work was carried out on a dedicated `add-wlasl-model` integration branch of the web application so that the stable `main` branch remained untouched while the RGB pipeline was wired in.

Migrated artifacts. The FastAPI `model-services` package was extended with the I3D network definition, the pretrained 100-word checkpoint (`asl100.pt` / `asl_word_i3d.pt`), the WLASL gloss-to-index list, and `torch/torchvision` runtime dependencies (Chapter 4, Table 4-11).

Stateful inference. A dedicated `wlasl_inference.py` module keeps one state dictionary per `session_id` (a 32-frame deque, a 10-prediction voting window, last-commit timestamp, and last-committed word), so that multiple concurrent users are not confused with one another's clips. Each incoming frame is decoded, colour-converted, resized so its short side is 256 px, centre-cropped to 224×224, and mapped to $[-1, 1]$ (the same preprocessing as Chapter 4, Figure 4-1) before being appended to that session's deque.

Once 32 frames are buffered, the I3D network runs and its softmax output is pushed into a **10-prediction voting window**; a class is only **committed** once it is the majority vote in that window ($\geq 6/10$), its confidence is $\geq 50\%$, and at least 1.2 s have passed since the last commit (mirroring the letter stabilisation decoder pattern from Chapter 3, Section 3.6) — otherwise the endpoint reports a buffering status with the current best guess. On the Node.js side, the predict controller attaches the authenticated user's ID (`req.user.id`, or "anonymous" if the request is unauthenticated) as the `sessionId` forwarded to FastAPI, giving each logged-in user an isolated video buffer without the frontend having to manage session identifiers itself.

Routing. The FastAPI `/predict` route was extended so that a request with `language="en"` and `mode="words"` is routed to the `wlasl_inference` engine instead of a placeholder response, mirroring the same language/mode dispatch pattern already used for the letter MLPs (Section 5.5.3). This keeps a single request contract (`language`, `mode`, `imageBase64`) across all four models even though the ASL word path is architecturally the odd one out — RGB frames rather than landmarks, and per-session server-side buffering rather than a single-frame forward pass.

5.7 NAVIGATION FLOW AND MAIN PAGES

The frontend uses `react-router-dom` [80] to govern navigation, with a flow that is linear and strictly gated by authentication status. Unauthenticated users land on the **Landing Page**, which presents an overview of the system and forms to register or log in. A custom `ProtectedRoute` higher-order component wraps the core application paths; if a user without a valid JWT attempts to reach the main dashboard, the router intercepts the navigation and redirects back to the Landing Page. Once authenticated, users are routed to the **App Home Page**, the central hub of the application, whose layout is split into two tabs: *Chatting* (live translation and real-time messaging) and *Educational* (sign-language visualisation and practice).

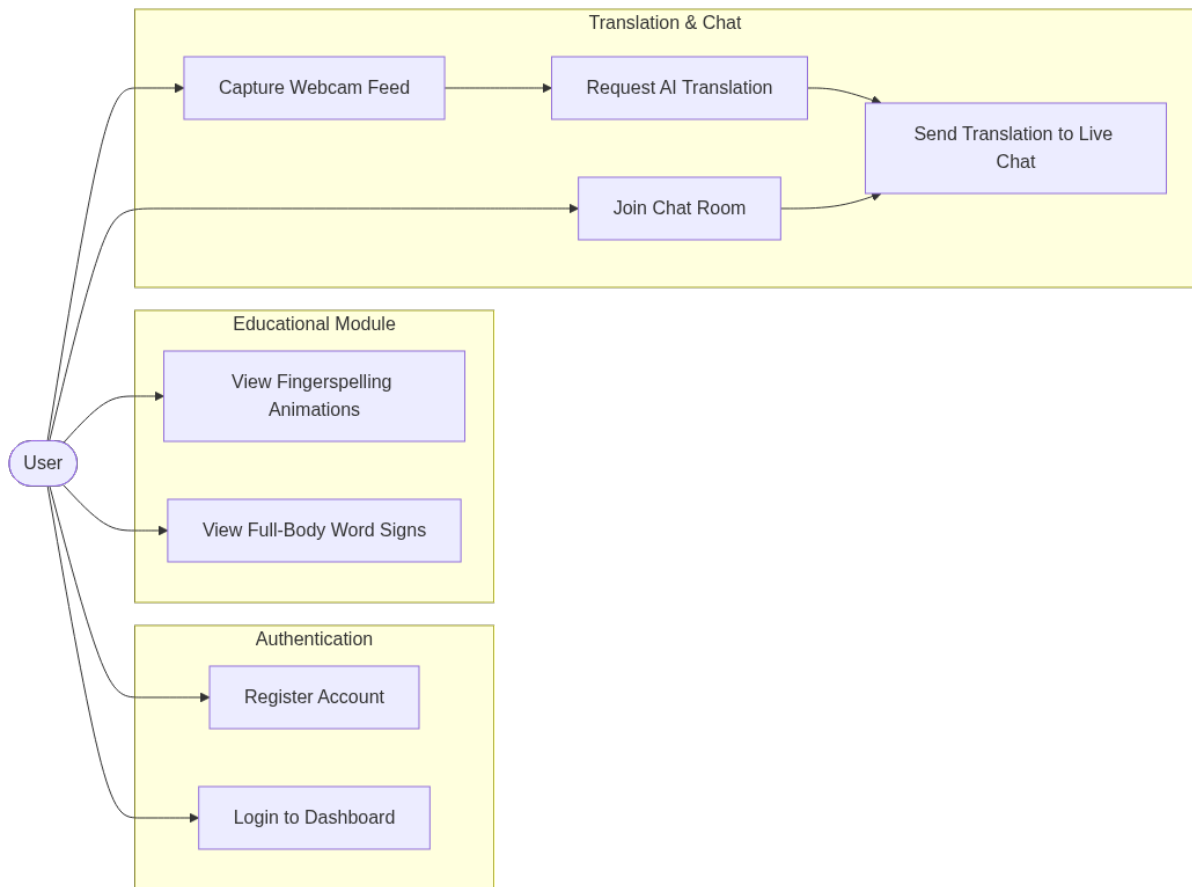


Figure 5-6: Use case diagram: authentication (register/login), the Educational module (browse and watch word signs, fingerspell typed text, sign a typed sentence, practise signs with webcam feedback), and Translation & Chat (capture webcam feed, request AI translation, join chat room, send translation to live chat).

Deployed web interface. Figures 5-7 and 5-8 show the live Eshara web application. The **Landing Page** (Figure 5-7) carries the Eshara logo and tagline (“*Sign language assistant with realtime chat*”), a Login/Register tab toggle, a dark-mode switch, and the credential form described in Section 5.3.2. The **App Home Page** (Figure 5-8) is reached only after authentication (Section 5.7) and shows the *Chatting* tab active: language (*English (ASL)*) and mode (*Letters/Words*) dropdowns, a camera-device selector (Section 5.2.4), the mirrored webcam panel with **Start Camera/End Camera/Translate Sign** controls and a live status line (“*Camera is off*”), and the **Live Chat** panel on the right with a chat-partner selector, message history, and a text/translation input bar — the concrete realisation of the Socket.IO messaging described in Section 5.3.3.

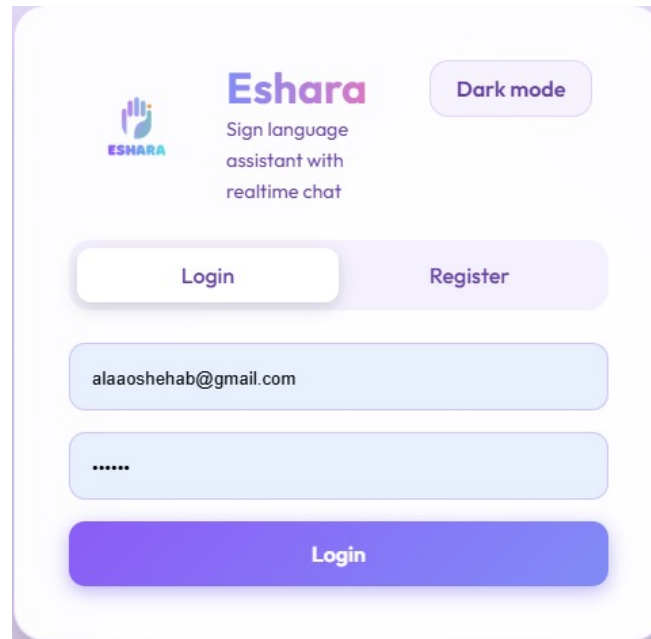


Figure 5-7: Eshara Landing Page: logo, tagline, Login/Register toggle, dark-mode switch, and credential form.

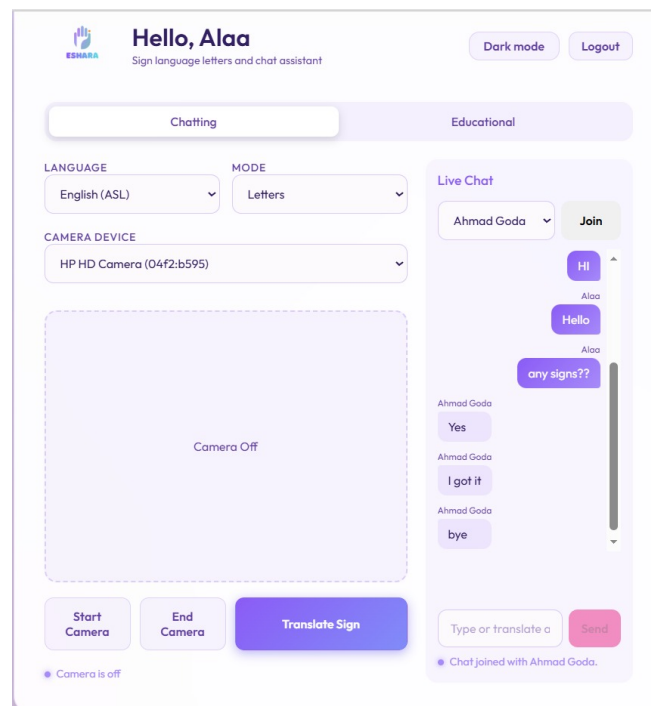


Figure 5-8: Eshara App Home Page (Chatting tab): language/mode/camera-device selectors, webcam capture panel with Start/End Camera and Translate Sign controls, and the Live Chat panel with an active Socket.IO room.

5.8 FEATURES, FUNCTIONALITY, AND UX DESIGN

Live chat and AI translation. The flagship feature merges standard text messaging with AI-driven translation: the user selects a target language (ASL/ArSL) and scope (letters/words); the mirrored webcam feed lets the user see themselves naturally while signing; and, once a translation is produced, the predicted output is appended to the chat buffer and can be emitted to a live Socket.IO room, enabling cross-user communication carried out entirely in sign language.

Educational tools. The Educational tab hosts three bilingual (ASL/ArSL) learning tools — a categorised *Sign Viewer* with integrated fingerspelling, a free-text *Sentence Builder*, and a webcam *Practice* mode — built on motion-capture skeleton playback with server-side animation stitching; the module is documented in full in Section 5.14 (Part B).

Responsive design and feedback. CSS media queries and a flexible display: grid layout let the interface adapt fluidly from desktop monitors to small mobile viewports, stacking side-by-side panels (camera feed and chat window) vertically on constrained screens [49]. Every interaction gives immediate visual feedback: translation requests trigger a temporary “*Translating...*” state, and socket connections display “*Connected*” or “*Error*” banners; modern micro-animations are used sparingly to keep the interface responsive without clutter.

5.9 SECURITY CONSIDERATIONS

Authentication and cryptography. User passwords are never stored in plaintext; they are hashed with `bcrypt.js` using a high salt round before insertion into the database [76]. Sessions are managed statelessly via JWTs, so authentication tokens are cryptographically signed and tamper-evident [71].

Encapsulated ML tier. The FastAPI model service is explicitly hidden behind the Node.js application server; because the ML service ports are never exposed to the public internet, the attack surface is significantly reduced and every translation request must pass through the Node.js authentication middleware before reaching the inference engine.

CORS and input validation. Cross-Origin Resource Sharing is strictly configured on the backend to prevent unauthorised domains from calling the APIs, and input payloads — particularly login credentials and large base64 image strings — are validated before processing to mitigate injection attacks and buffer-overload abuse. Transmitting landmark vectors rather than raw video for the letter and ArSL-word paths (Chapter 2, Chapter 7 Section 7.6) further reduces the amount of biometric image data that ever leaves the client for two of the four models; the ASL word path, which must transmit RGB frames for the I3D network, does not carry this benefit and is discussed as a residual privacy trade-off in Chapter 7.

5.10 DEPLOYMENT

Graduation deployment is **web-only** across three independently hostable tiers: the React SPA is served as a static build over HTTPS; the Node.js + Express application server runs as a long-lived process with its SQLite database file on a persistent volume and the Socket.IO server attached to the same HTTP instance; and the FastAPI service runs in an isolated ML inference environment with the four model checkpoints loaded from disk. No app-store packaging, React Native wrapper, or on-device TensorFlow Lite/PyTorch Mobile export is part of the graduation deliverable.

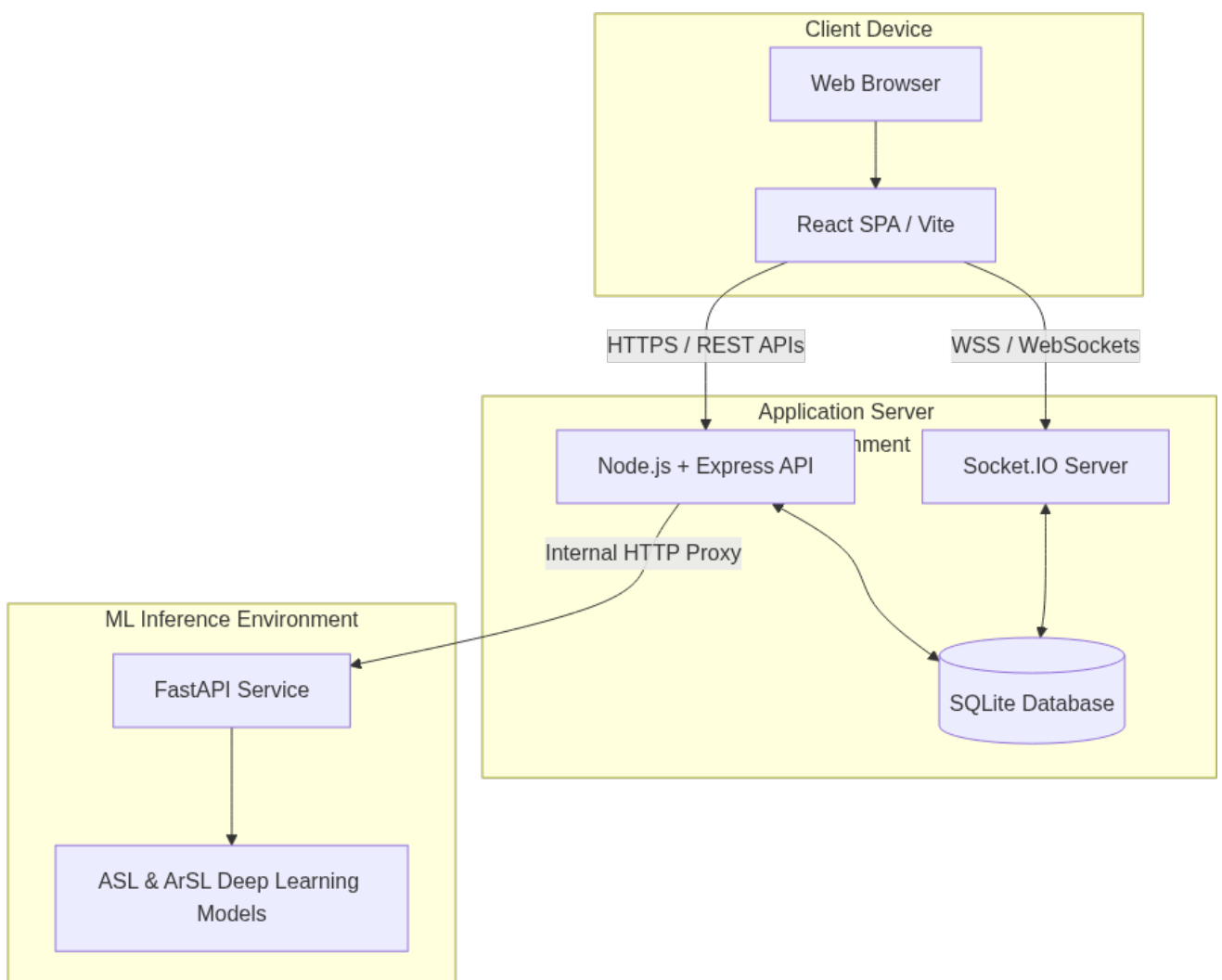


Figure 5-9: Deployment diagram: client device (browser, React SPA) communicating over HTTPS/REST and WSS/WebSockets with the application server (Node.js + Express, Socket.IO, SQLite), which forwards inference requests over an internal HTTP proxy to the isolated ML inference environment (FastAPI service, ASL/ArSL deep-learning models).

5.11 ERROR HANDLING AND TESTING

5.11.1 Graceful degradation:

On the client, global Axios interceptors catch 401 Unauthorized responses to purge invalid sessions and redirect the user, and browser APIs such as `navigator.mediaDevices` are wrapped in try/catch blocks so that a denied or missing webcam falls back to human-readable troubleshooting instructions rather than a silent failure. On the server, Express controllers wrap external calls in try/catch; if the FastAPI service is offline, the Node.js proxy catches the failed fetch and returns a structured 503 `Service Unavailable` response instead of allowing the primary server process to crash.

5.11.2 Testing methodology:

The application underwent manual interface and integration testing covering real-time Socket.IO chat latency, responsive layout reflow on constrained mobile viewports, and cross-browser webcam access. Independently of the web integration, each backend ML model was evaluated against its own standardised dataset split before integration (Chapter 6); integration testing then confirmed that frontend JPEG base64 compression does not introduce artifacts that materially degrade active web inference paths during live webcam use.

5.12 REAL-TIME WEB PERFORMANCE

Chapter 1 hypothesised end-to-end latency ≤ 200 ms from frame capture to displayed text on a standard laptop. As reported in the following table, **pilot measurements** on graduation development hardware (Windows 11 laptop, Chrome, 720p webcam, localhost stack) use 100 samples per row, timed with `performance.now()` in the browser and server wall-clock around `model.predict`; the full measurement protocol is documented in the project evaluation notes.

Table 5-1: Real-time web performance (pilot measurements, graduation dev hardware).

Component	How measured	p50
MediaPipe (FastAPI, server-side)	100 frames, server wall-clock around landmark extraction	34 ms (~ 29 FPS)
Server inference (letter MLP)	FastAPI timing around predict	4 ms
Server inference (ArSL word BiLSTM)	48-frame batch (offline-validated path)	24 ms
Server inference (ASL word I3D)	32-frame RGB batch (CUDA)	92 ms
HTTP /predict round-trip (letter)	Client POST $\rightarrow$ Node proxy $\rightarrow$ JSON response	11 ms
E2E letter	Frame $\rightarrow$ UI label update	51 ms
E2E ArSL word (offline-validated)	FastAPI inference after 48-frame buffer	41 ms
E2E ASL word	Inference after 32-frame buffer	108 ms

Letter mode satisfies the ≤ 200 ms hypothesis (p50 = 51 ms). Word modes incur an additional **buffer-fill** delay (~ 1.0 – 1.6 s at 30 FPS for 32–48 frames) before the first inference; per-commit server latency remains below 150 ms. Server-side MediaPipe extraction dominates the letter-path compute budget before the MLP forward pass; letter MLP inference remains under 5 ms on CPU as expected from Chapter 3 architecture size. ArSL word rows in Table 5-1 reflect the offline-validated FastAPI path documented in Section 5.5.3, not the current mock web response for arabic + words.

5.13 INFORMAL USABILITY PILOT

A formal deaf-community study with System Usability Scale (SUS) scoring was out of graduation scope (Chapter 6, Section 6.6). An **informal pilot** ($N=3$ student testers, 15-minute sessions each) exercised all four modes in the React UI. The following table records structured observations; a full SUS study with 10+ participants is recommended in Chapter 9 [49].

Table 5-2: Informal usability pilot observations ($N=3$, React web UI).

Criterion	Rating	Notes
Camera setup (<1 min)	Good	Standard browser permission flow
Letter mode clarity	Good	Stabilisation reduces flicker; J/Z motion weak
ArSL word mode clarity	Good	48-frame buffer requires full sign duration
ASL word mode clarity	Fair	Framing sensitivity; 32-frame window
RTL Arabic display	Good	Sentence builder readable
Mode switching (letter/word)	Fair	Manual toggle; no auto-detection
Overall assistive potential	Good	Suitable for practice, not certified interpretation

PART B — EDUCATION MODULE

5.14 EDUCATION MODULE: BILINGUAL SIGN-LANGUAGE LEARNING

This section documents the Education tab at the same technical depth as Part A’s Translation & Chat features. The module was rebuilt during the final development cycle around **motion-capture skeleton playback**: signs are no longer drawn as hand-authored stick-figure keyframes but replayed from pose recordings of real signers, and the module grew from two ASL practice panels into three bilingual learning tools. In both American Sign Language and Arabic Sign Language the tab offers a **Sign Viewer** (browse a categorised vocabulary and watch each sign, or fingerspell any typed word), a **Sentence Builder** (type a free-form sentence and watch it signed end-to-end), and a **Practice** mode (perform a prompted sign into the webcam and get instant feedback from the same recognition models that power the Chatting tab). Unlike the earlier canvas-only iteration, the Education tab is now a client–server feature: animation assembly runs in the FastAPI model-services tier, which stitches multiple sign clips into one smooth animation and streams it to the browser.

5.14.1 Overview and layout:

The Education tab is one of the two primary tabs on the App Home Page alongside Chatting (Section 5.7). A header sub-navigation carries a language toggle — *ASL · English / ArSL · العربية* — and a tool switch between the Sign Viewer, the Sentence Builder, and Practice mode. Switching languages remounts the tool components with a React `key={lang}`, so selection, playback, and camera state reset naturally without manual synchronisation logic. Each component receives its language-specific configuration (word list, categories, alphabet, text direction, placeholder copy) from a single `LANG_CONFIG` lookup, so the English and Arabic experiences share one code path.

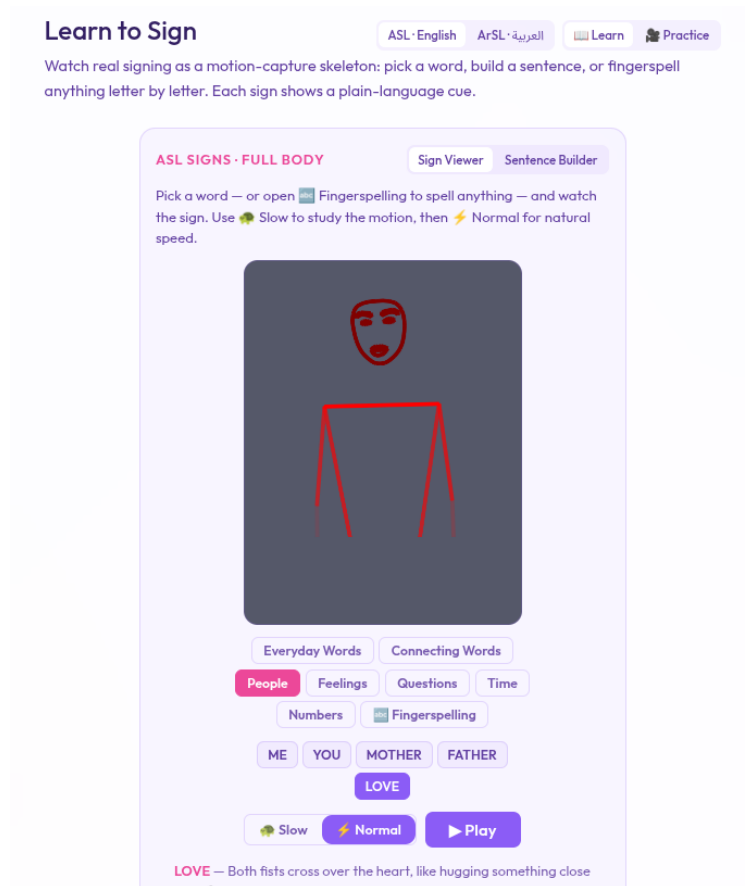


Figure 5-10: Eshara Education tab (Sign Viewer, ASL): motion-capture skeleton playback with categorised vocabulary and playback-speed controls.

5.14.2 Motivation: from hand-authored keyframes to motion capture:

An earlier iteration of this module rendered signs with a bespoke Canvas2D stick figure: every sign was a manually authored sequence of wrist keyframes and procedurally constructed handshapes, resolved through a two-joint inverse-kinematics solver. That engine was fast and dependency-free, but it could not scale in either quantity or quality. Each new sign required careful manual authoring and a geometric verification pass (hands visually merging, contact points floating off the head), the vocabulary plateaued at fifteen words, and even a correct keyframe animation reads as a symbolic illustration rather than a human performing the sign — a material pedagogical weakness for a learning tool.

The module was therefore rebuilt on **skeleton recordings of real signers**. Every sign is a .pose file — the standard binary format of the pose-format library [74] — containing per-frame MediaPipe Holistic landmarks (body, face, and both hands) extracted from video of a human signer [4], replayed in the browser by the <pose-viewer> web component from the sign-

language-processing project (the same renderer used by the open-source sign.mt translation platform [73]). This reversal of the earlier decision to avoid `<pose-viewer>` was deliberate and evidence-based: the lag that originally motivated the Canvas2D rewrite was caused not by the renderer itself but by playing many small clips back-to-back with client-side fetches between them. The rebuilt module fixes the root cause with server-side animation stitching (Section 5.14.4) — one continuous animation per Play press — while gaining human motion data, per-finger articulation, and a content pipeline that can grow the vocabulary without manual animation work. The legacy canvas stickman remains only as a fallback renderer for the few English words that have hand-authored keyframes but no `.pose` file.

5.14.3 Content pipeline: dictionary signs from sign.mt:

Sign content is fetched offline from sign.mt’s public dictionary API (`model-services/fetch_signmt_pose.js`) which translates a spoken-language word into a signed-language pose clip drawn from a curated dictionary: American Sign Language (`ase`) for English and Jordanian Sign Language (`jos`) for Arabic — the only openly available Arabic-lookup sign dictionary [73]. The deployed vocabulary counts 44 English words plus the 26-letter manual alphabet, and 42 verified Arabic word signs plus the 28-letter Arabic alphabet; every clip is attributed on-page (“*Motion captured from a real signer*”). Critically, the API silently returns a **fingerspelled** letter sequence, with HTTP 200, for any word its dictionary does not contain — so a successful fetch does not prove the dictionary has the sign. The verification methodology built to defeat this failure mode is documented in Section 5.14.8.

One rendering subtlety is worth recording. The `pose-viewer` custom element upgrades lazily, and a false JSX boolean attribute is removed from the DOM entirely — so `autoplay={false}` never reaches the component, whose internal default is `true`. Each component therefore sets `el.autoplay = false` imperatively in a mount effect, and drives playback through the component’s `play()/pause()` methods and its `loadeddata$ / ended$` events.

5.14.4 Server-side animation stitching (`pose_concat.py`):

Playing letter or word clips back-to-back on the client produced the original module’s characteristic lag: between every two clips the skeleton snapped from one pose to another, with a network fetch in between. All multi-sign playback was therefore moved server-side. The FastAPI tier stitches the requested signs into **one** `.pose` file using spoken-to-signed — sign.mt’s own concatenation package [81] — and streams the finished animation to the client:

1. Each clip is reduced (redundant landmarks dropped), normalised, and trimmed at its natural signing boundaries (a wrist-above-elbow heuristic finds where signing actually starts and ends).

2. Adjacent clips are cut at their best **connection point** — the frame pair where the two skeletons are geometrically closest — so one sign flows into the next.
3. A short zero-confidence gap (0.2 s) is inserted at every boundary and filled by interpolation; the whole sequence is then smoothed with a Savitzky–Golay filter, wrist positions are corrected, and the result is size-normalised.

Three language-parameterised endpoint families expose this pipeline:

- `/api/pose-seq (mode=words)` — stitches a list of named signs with natural trimmed motion; used when the Sign Viewer plays a word.
- `/api/pose-seq (mode=spell)` — fingerspelling mode. Letter clips are inconsistent upstream (some are 3-frame stills, some full 25/30 fps clips), and connection-point cutting can shrink a letter to a few unreadable frames — so spell mode instead slices each letter to its central held handshape and freezes it for 0.35 s, producing a uniform, readable spelling rhythm.
- `/api/pose-text` — free-text mode for the Sentence Builder (Section 5.14.6).

Every stitching endpoint has a `/meta` twin returning per-segment durations on the final timeline; because the animation itself is a single opaque file, synchronised UI feedback (highlighting the word chip or letter currently playing) is driven entirely by this metadata with plain client-side timers. Results are memoised server-side (`functools.lru_cache`) so a repeated Play press is a cache hit, and `Content-Disposition` headers use RFC 5987 encoding because Arabic filenames cannot be represented in latin-1 HTTP headers. Sentence input is bounded (12 tokens, 80 stitched segments) so a pathological request cannot exhaust the service.

5.14.5 Sign Viewer:

The Sign Viewer presents the language’s vocabulary as category tabs (*Everyday Words*, *People*, *Feelings*, ...; كلمات يومية, الناس, مشاعر وصفات, ...) with one button per word. Selecting a word previews its first frame; Play streams the sign at *Slow* (0.5×) or *Normal* (1×, the default) speed, and each word is captioned with its meaning — for Arabic, an English gloss, since the

app’s UI language is English. A final tab, *Fingerspelling*, replaces the word grid with a letter reference grid (A–Z for ASL; the 28 Arabic base letters for ArSL) whose buttons play each letter’s held handshape, plus a free spelling input: any typed word is filtered to spellable letters and played as one server-stitched spell-mode sequence, with the on-screen grid highlighting each letter in sync via the /meta timing data. If the configured default word is ever pruned from the vocabulary, the component falls back to the first available word rather than rendering an empty viewer.

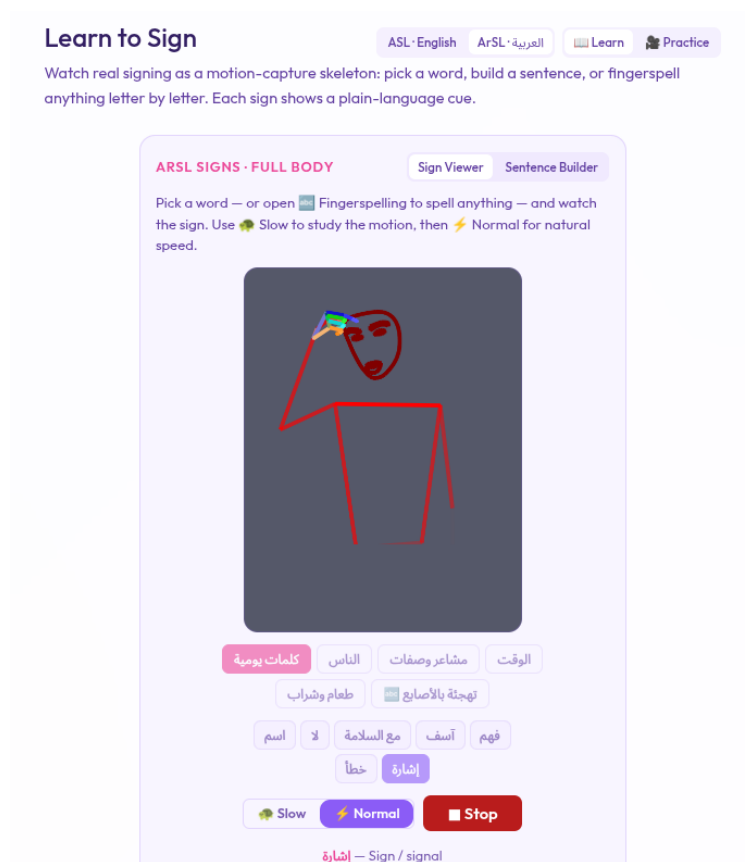


Figure 5-11: Sign Viewer (ArSL): Jordanian Sign Language skeleton playback with right-to-left category and word controls.

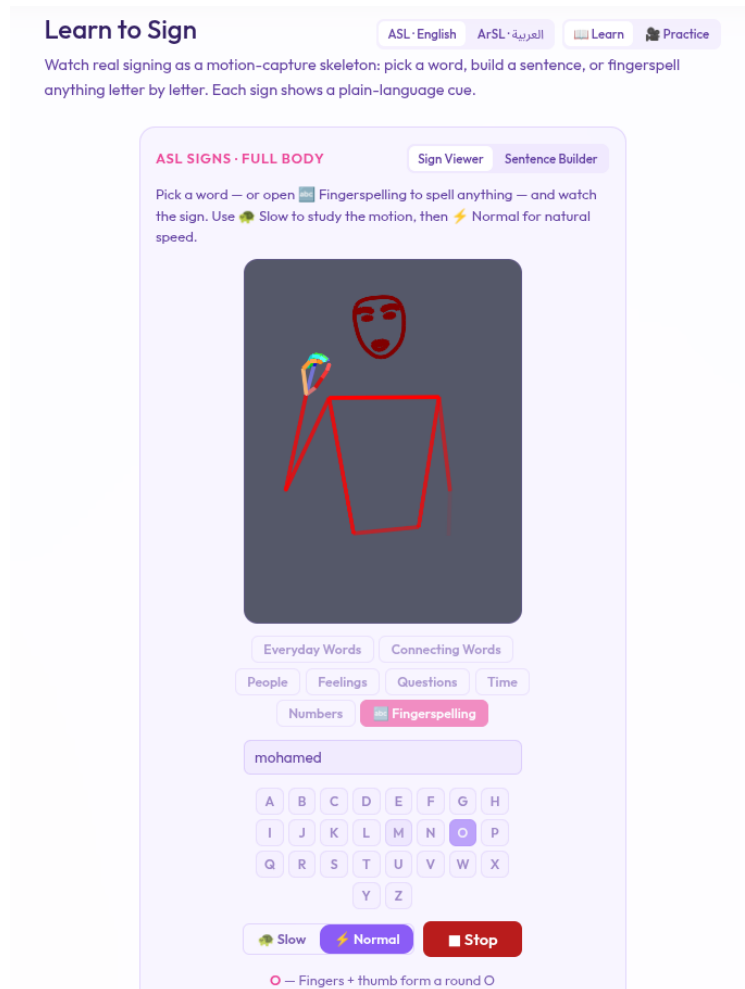


Figure 5-12: Fingerspelling: a typed word plays as one server-stitched sequence while the current letter is highlighted from timing metadata.

5.14.6 Sentence Builder — free-text signing:

The Sentence Builder follows sign.mt’s interaction model: the user types any sentence; words found in the sign dictionary are signed, and unknown words are fingerspelled letter by letter [73]. The server’s plan builder (`pose_concat.build_text_sequence`) implements this as follows:

1. The text is tokenised (lowercased; Arabic diacritics stripped; digits 1–5 and Arabic-Indic ١–٥ mapped to their number words).
2. Tokens are matched greedily against the dictionary, longest phrase first (3-gram, then 2-gram, then single word) — multi-word entries such as “*thank you*” or مع السلامة exist as **one** pose file and must not be split into unknown singles and spelled.
3. Lookups are orthography-tolerant (Section 5.14.7): both the query and the stored file-names are folded before comparison.
4. Any token still unmatched is decomposed into its letters and fingerspelled inline, using the same held-peak treatment as the Fingerspelling tab.
5. The full plan is stitched into one smooth animation and returned with per-token and per-letter timing metadata.

The frontend mirrors the same greedy matching client-side to render a live **chip preview** as the user types: solid chips are words that will be signed from the dictionary; dashed chips carrying a fingerspelling marker will be spelled. This honesty matters — a learner must never mistake a spelled word for the word’s actual sign. During playback the chips and, for spelled tokens, the individual letters light up in sync with the animation.

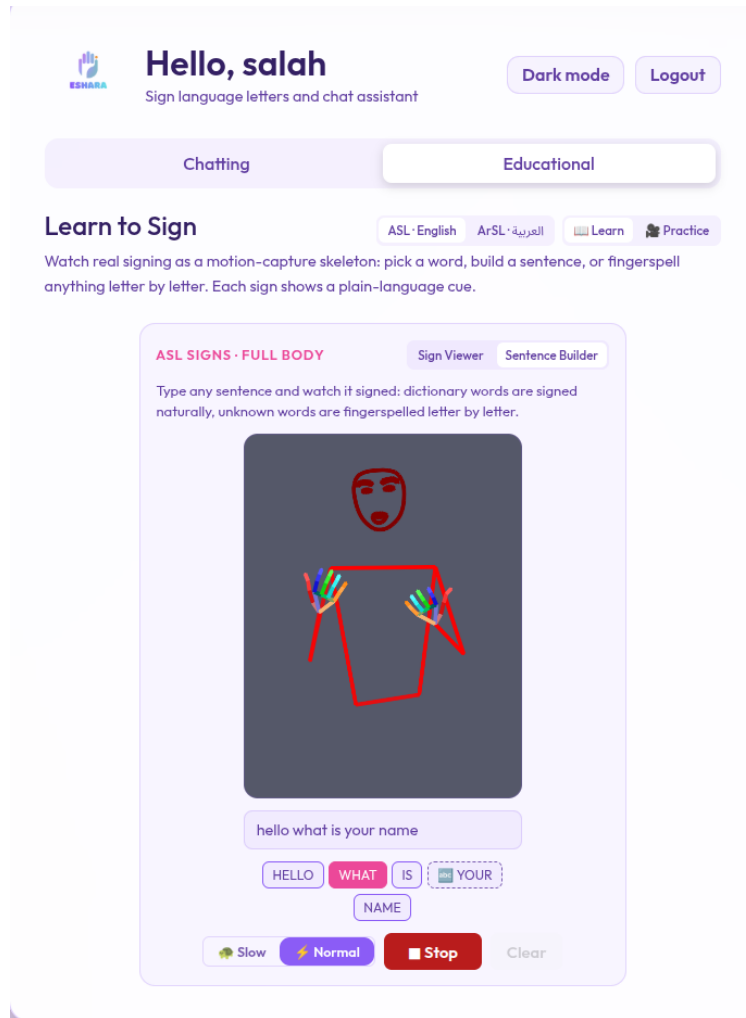


Figure 5-13: Sentence Builder: dictionary words render as solid chips and are signed; unknown words render as dashed chips and are fingerspelled, with live highlighting driven by timing metadata.

5.14.7 Arabic support (ArSL):

Every pose endpoint takes `lang=en|ar` and resolves content from a per-language directory. The tools are fully parameterised for Arabic: right-to-left text direction on inputs and labels, Arabic-Indic digit handling, and Arabic category labels. Two Arabic-specific engineering problems are worth recording:

- **Orthographic variance.** Arabic words are commonly typed without hamza/madda marks (أم for أمّ, آسف for آسف), yet the sign dictionary keys many entries under proper orthography — and vice versa. A single folding table (آء → ا; ئ → ي; و → ؤ; ه → ة) is applied to **both** sides of every lookup: the server resolves queries through a folded-name index, and the client mirrors the same fold for chip previews and practice-answer matching. Files keep their proper spellings; users may type either form.
- **Fingerspelling alphabet.** ArSL fingerspelling uses the 28 base letters; variant forms fold onto the letters that have clips (e.g. ه spells as هـ), and diacritics are stripped before spelling.

Two linguistic caveats are documented for the record. First, the Arabic sign content is Jordanian Sign Language (jos) — the only openly available Arabic sign dictionary — while the ArSL recognition models were trained on KArSL (Saudi) data [7]; regional sign variants differ, so the viewer and the recogniser are not guaranteed to agree on every word. Second, sentence signing follows the typed word order (a transliteration); it does not reorder into natural sign-language grammar. Both are acceptable for a learning aid but are stated honestly in the interface and in this thesis.

5.14.8 Content verification — defeating the silent fallback:

The hardest data-quality problem in the module was the dictionary API’s silent fallback: for an out-of-vocabulary word it returns a fingerspelled letter sequence that is indistinguishable, at the protocol level, from a real sign. Publishing such a clip as a word sign would teach learners something actively wrong, and visual review alone proved unreliable — a reviewer who does not know Jordanian Sign Language cannot always tell an unfamiliar sign from a spelled sequence. An automated classifier was therefore built:

1. For a candidate word, fetch the word itself **and** the doubled word (the word concatenated with itself). The doubled form is guaranteed to be out-of-vocabulary, so its returned animation is, by construction, the cloud’s own spelling of the word, processed by the identical pipeline.

2. Reduce both animations to per-frame **handshape descriptors**: the 21 right-hand landmarks, wrist-relative and scale-normalised, so global position and size differences cancel.
3. Compare with start-anchored **dynamic time warping** (DTW), which absorbs the timing differences the stitching pipeline introduces. If the word’s animation matches the spelling’s prefix, it **is** the spelling fallback. Scores separate cleanly: spelled < 0.9, real ≥ 2.9 .

Every accepted clip was additionally reviewed frame-by-frame as a rendered filmstrip. A sweep of roughly 170 candidate Arabic words produced two further discoveries. First, the dictionary keys many entries under proper orthography only — أ is a real sign while ا comes back spelled — which directly motivated the fold-everywhere lookup of Section 5.14.7. Second, words containing δ crash the API with HTTP 500 when out-of-vocabulary (its spelling lexicon lacks that letter), which turns the status code into a truth oracle for that word class. The final Arabic vocabulary — verified word signs only — is exactly the set that passed all checks; everything else fingerspells honestly with the dashed-chip treatment.

5.14.9 Practice mode — learn by doing:

Practice mode closes the learning loop: after watching a sign, the user performs it. The panel reuses the exact camera-and-predict pipeline that powers the live Chatting tab — extracted into a shared `useCamera` hook — with its own camera instance so the two tabs never interfere:

- **Letter practice:** the panel prompts a random letter; the user poses it and a single webcam frame is sent to `POST /predict (mode=letters)`. The models return exactly the comparison target — an uppercase English letter or a real Arabic glyph — so matching is a normalised string comparison.
- **Word practice:** the panel prompts a word, runs a 3-2-1 countdown (cued performance needs reaction time), records a 3.2-second clip, and sends it to `POST /predict (mode=words)`. Arabic matching folds orthography on both sides so the model’s bare-spelled labels (ل) match the vocabulary’s proper spellings (لّ).

- **Honest target lists:** only words the deployed checkpoints can actually recognise are offered for checking — the education words inside the WLASL-100 class list for ASL, and the fold-normalised intersection with the KArSL model’s labels for ArSL. Everything else is watch-only rather than silently impossible.
- **Constructive feedback:** when the model saw a sign but stayed below its confidence threshold, the UI reports its best guess (“*it looked like X, just not clearly enough*”) instead of a generic failure, distinguishing “*almost had it*” from “*wrong sign entirely*”.

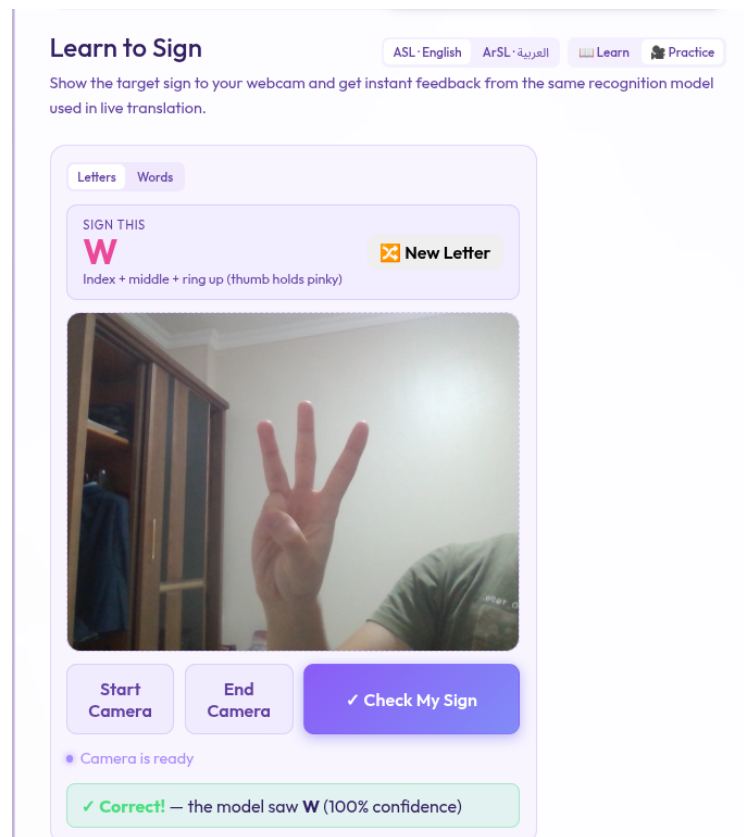


Figure 5-14: Practice mode: the learner performs a prompted sign; the same recognition models that power live translation check the attempt and give constructive feedback.

5.14.10 Performance and verification engineering:

Four decisions keep the rebuilt module responsive and correct; the following table contrasts the deployed pipeline with the superseded Canvas2D engine.

- **One stitched file per playback.** All multi-sign animation is assembled server-side and cached, so the client performs exactly one media fetch per Play press regardless of sentence length — the root-cause fix for the original choppy, per-clip playback.

- **A single persistent viewer element.** Each tool keeps one `<pose-viewer>` mounted across word and tab switches (hidden when the canvas fallback is active), so switching words never re-initialises the rendering component; idle previews load only the first frame.
- **Guarded interaction states.** Word buttons, letter grids, and inputs are disabled during playback; playback promises resolve on the component's `ended$` event, a cancellation poll, or a hard timeout, so the UI can never wedge in a playing state.
- **End-to-end browser verification.** Every change to the module was verified in a scripted headless Chrome session (Playwright): log in as a QA user, drive the real UI — switch language, play specific words, type sentences, assert chip classifications — and require zero browser console errors. Sign-content changes were additionally verified by rendering filmstrip sheets of the actual pose data and reviewing them visually.

Table 5-3: Education module: superseded Canvas2D engine versus the deployed motion-capture pipeline.

Property	Superseded (Canvas2D)	Deployed (motion capture)
Rendering	Hand-authored keyframes + two-joint IK stick figure	<code><pose-viewer></code> skeleton playback of <code>.pose</code> recordings
Animation source	In-repo keyframes authored by the team (15 words, ASL only)	Dictionary recordings of real signers via <code>sign.mt</code> (44 EN + 42 AR words, both alphabets)
Multi-sign playback	Per-letter client sequencing	One server-stitched animation with timing metadata for UI highlights
Bilingual coverage	ASL only	Full ASL/ArSL parity incl. RTL and orthography-folded lookup
Correctness assurance	Manual geometric review of keyframes	DTW fallback detection + filmstrip review + scripted zero-console-error browser tests

Part B summary. The Education module pairs three tools — a categorised Sign Viewer with integrated fingerspelling, a free-text Sentence Builder, and a webcam Practice mode — across two sign languages behind one bilingual architecture. Signs are motion-capture skeletons of real signers, rendered by the `pose-viewer` web component and stitched server-side into single smooth animations whose timing metadata drives synchronised UI highlights. Arabic support

required orthography-folded lookup on both client and server, right-to-left interface treatment, and a rigorous DTW-based verification methodology to reject the dictionary API's silent fingerspelling fallbacks — ensuring that every word the module presents as a sign is genuinely that sign, and that everything else is honestly labelled as fingerspelling. Practice mode closes the loop by checking the learner's own signing against the same recognition models that power live translation, offering only targets those models can actually recognise.

Chapter summary. Chapter 5 documented Eshara’s realisation as a three-tier web product in two parts. **Part A** covered a React/Vite client for capture, visualisation, and chat; a Node.js/Express application server providing JWT authentication, Prisma/SQLite persistence, and Socket.IO real-time messaging; and a FastAPI model-services tier that dispatches base64 frames to the active ASL/ArSL letter and ASL-word inference paths, including the PyTorch Inception I3D integration for ASL words with per-session frame buffering. Security is layered through bcrypt password hashing, stateless JWT sessions, an encapsulated ML tier, and CORS/input validation; error handling degrades gracefully on both client and server. Pilot measurements confirm letter-mode end-to-end latency meets the ≤ 200 ms target from Chapter 1 (Table 5-1), and an informal three-tester usability pass (Table 5-2) rates the system suitable for practice and assistive use, not certified interpretation. **Part B** documented the Education Module: bilingual Sign Viewer, Sentence Builder, and Practice tools built on motion-capture skeleton playback (pose-viewer), server-side animation stitching with timing metadata, orthography-folded Arabic lookup, and a DTW-based content-verification methodology that rejects fingerspelling fallbacks. Chapter 8 concludes the thesis; Chapter 9 outlines continuous recognition, vocabulary expansion, and formal user studies as future work.

6 RESULTS AND EVALUATION

This chapter reports quantitative evaluation of the four graduation models under the protocols defined in Sections 3.9 and 4.12. Metrics are computed on **held-out test data** unless noted otherwise; ASL word I3D numbers follow the published WLASL-100 benchmark for the deployed checkpoint family [6]. Letter results are re-evaluated from the canonical letter training artifacts; Arabic word results follow the KArSL three-signer BiLSTM training and evaluation protocol (Chapter 4, Model 4). Section 6.0 summarises the experimental conditions — long extraction runs, repeated training iterations, and webcam-domain issues — that shaped the final deployed models.

Evaluating isolated sign language recognition models requires more than reporting a single top-tier accuracy metric. Because the ultimate goal of Eshara is an interactive, browser-based deployment, the offline results presented in this chapter serve as the empirical foundation for the real-time system tested in Chapter 5. By strictly isolating the test sets—ensuring no frames or clips leak from the training or validation phases—these metrics reflect genuine model generalisation. The evaluation leverages Top-1 accuracy for the highly balanced letter datasets, and expands to Top-5 accuracy and Macro-F1 scores for the word datasets to rigorously account for scarce classes and visually similar sign minimal pairs.

6.0 TRAINING CONDITIONS AND EXPERIMENTAL CHALLENGES

Offline accuracy in Tables 6-2–6-4 was obtained after substantial iteration. The following table records recurring difficulties during experimental development; the software stack and CUDA/GPU routing appear in Chapter 3 (Table 3-2).

Transitioning from raw video corpora to deployable neural network checkpoints involved navigating severe hardware limitations and profound domain gaps. Academic datasets like WLASL and KArSL feature carefully framed subjects, stable lighting, and high-resolution sensors. In contrast, the target deployment environment consists of low-resolution webcams, variable indoor lighting, and cluttered backgrounds. This discrepancy heavily motivated the shift away from pixel-heavy convolutional models (which often overfit to background clutter) toward the robust, geometry-based MediaPipe landmark pipelines described in prior chapters.

Furthermore, processing tens of thousands of video clips to extract these 225-dimensional skeletal sequences required days of uninterrupted CPU time, creating significant developmental bottlenecks. To overcome local hardware constraints (such as standard 4 GB VRAM limits), model training and hyperparameter tuning were offloaded to cloud environments like Kaggle [82] using T4 and P100 accelerators. This allowed for rigorous iterative testing, including the use of `EarlyStopping` callbacks [51] to prevent overfitting and dynamic learning rate adjustments to stabilize convergence across all four independent models.

Table 6-1: Recurring training and evaluation challenges (project experiments).

Challenge	Manifestation	Mitigation adopted
Long extraction	KArSL Holistic NPZ builds (hours–days CPU)	Kaggle GPU runs; partial caches; 100-class subset
GPU memory	4 GB local VRAM; 502-class experiments OOM	Kaggle T4/P100; smaller batch; 100-class deploy scope
Webcam domain gap	Studio datasets vs. live blur, distance, exposure	Landmark path; stabilisation decoder; retrain after failed pixel models
MediaPipe dropout	Missing hands/pose → zero vectors	Discard frames; hand-present gate (ArSL words)
Background clutter	Posters, shelves, skin-tone contrast	Nose/wrist normalisation (words); engineered angles (ASL letters)
Repeated training	Multiple revisions per model family	<code>EarlyStopping</code> ; checkpoint export; superseded paths in Ch. 6
ASL word RGB	WLASL clip variance; broken URLs in metadata	Pretrained I3D checkpoint; WLASL-100 subset

6.1 LETTER RECOGNITION RESULTS

Letter models are static single-frame MLP classifiers: **78-D engineered** landmarks for ASL and **63-D raw** hand landmarks for ArSL (Chapter 3). The following table summarises held-out test metrics from the production training pipelines.

Table 6-2: Letter-model held-out test metrics (canonical training pipelines).

Model	Classes	Test N	Top-1	Macro-F1	Top-3	Split	Source
ASL Letter MLP	28	12 401	99.06 %	98.74 %	99.79 %	64/16/20	ASL letter training artifact set
ArSL Letter MLP	32	14 734	99.63 %	99.44 %	99.92 %	60/20/20	ArSL letter training artifact set

6.1.1 ASL Letter MLP:

Training and evaluation use the engineered feature corpus `asl_letters_engineered.csv` (78-D: wrist-centred 21×3 landmarks plus 15 inter-joint angles). The deployed checkpoint `asl_mediapipe_mlp_model_engineered.h5` (MLP $256 \rightarrow 128 \rightarrow 64$, ~ 49 k parameters) achieves **99.06 %** Top-1, **98.74 %** macro-F1, and **99.79 %** Top-3 on 12 401 held-out test samples (28 classes: A–Z, del, space). A merged 63-D diagnostic run reports lower accuracy and is not used for deployment. Errors concentrate on motion letters J and Z and on low-support classes M/N [8].

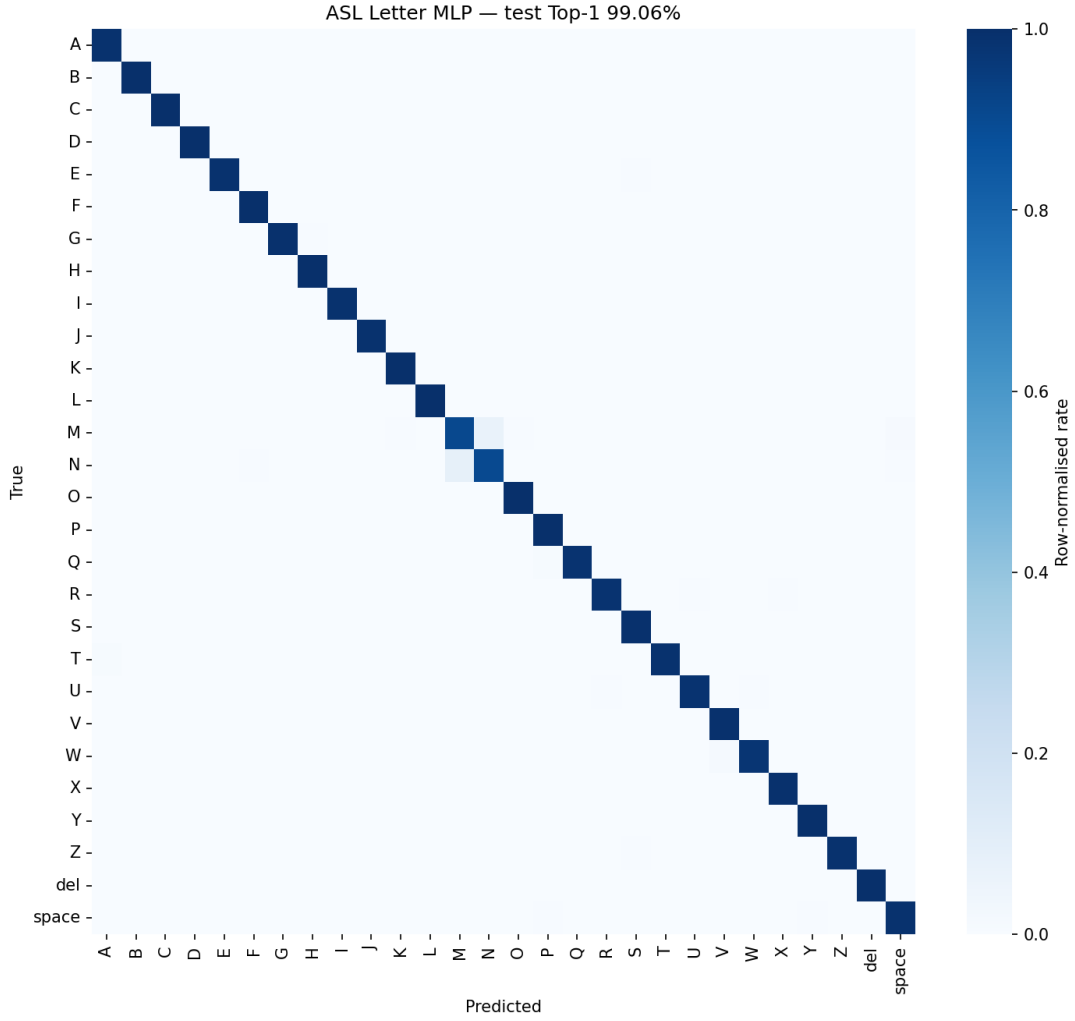


Figure 6-1: ASL letter MLP row-normalised confusion matrix (78-D engineered features, held-out test $N=12401$, 28 classes). Reproduced from held-out test evaluation.

6.1.2 ArSL Letter MLP:

Training and evaluation use the full ArASL2018 keypoint corpus (63-D raw MediaPipe hand landmarks, 32 classes; 73 669 rows in ASLAD-3000_v2.csv). The deployed checkpoint `arsl_mediapipe_mlp_model_bestV2.2.h5` uses a wider MLP head ($512 \rightarrow 256 \rightarrow 64$, ~ 183 k parameters). Re-evaluation on the same 60/20/20 split yields **99.63 %** Top-1, **99.44 %** macro-F1, and **99.92 %** Top-3 on **14 734** held-out test samples. Visually similar Arabic letter pairs (fa/gaaf, haa/khaa) remain the main off-diagonal confusion source [9].

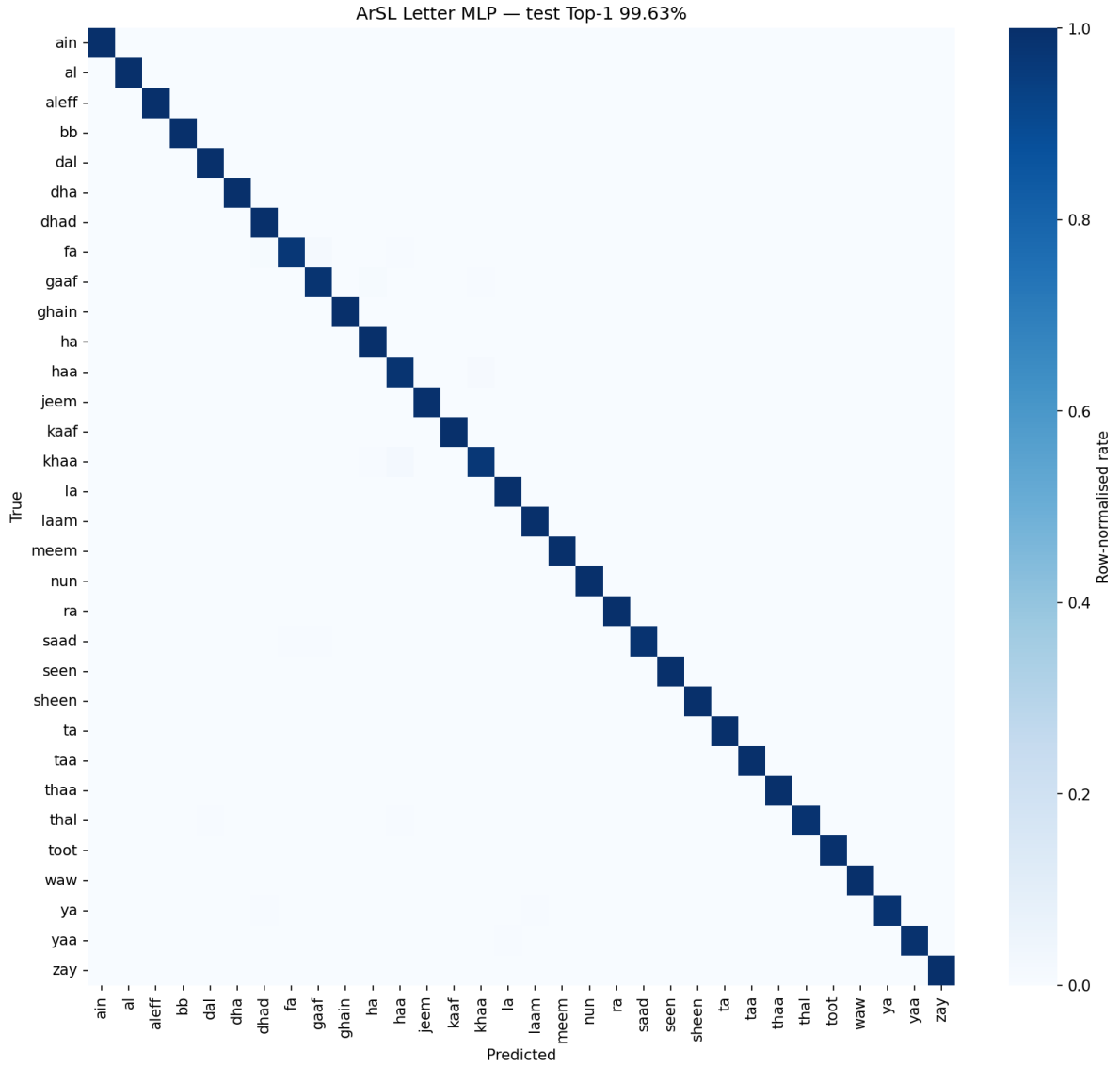


Figure 6-2: ArSL letter MLP row-normalised confusion matrix (63-D raw hand landmarks, held-out test $N=14734$, 32 classes). Reproduced from held-out test evaluation.

As illustrated by the heavily dominant diagonal in Figure 6-2, the model achieves near-perfect class separation across all 32 Arabic letters. The wider network capacity successfully resolves the nuanced finger configurations of challenging minimal pairs that typically confuse static image classifiers. Furthermore, because these offline errors are vanishingly rare (accounting for less than 0.4 % of the test set), the live web implementation easily overcomes any residual flickering by routing the predictions through the temporal stabilisation decoder, ensuring robust and fluid real-time text input for the end user.

6.2 WORD RECOGNITION RESULTS

6.2.1 ASL Word Inception I3D (deployed):

The deployed ASL word model is the pretrained `asl_word_i3d.pt` Inception I3D checkpoint (100 WLASL classes, 32-frame 224×224 RGB clips) described in Chapter 4 Sections 4.1–4.5. This checkpoint is the original WLASL-100 benchmark model released with the dataset; as reported in Table 6-3, it achieves the published Top-1/Top-5/Top-10 accuracy on the 258-clip WLASL-100 test partition [6]. Qualitative live behaviour is discussed in Sections 6.5 and 5.6.

Table 6-3: ASL word model metrics — deployed Inception I3D (`asl_word_i3d.pt`), WLASL-100 benchmark protocol.

Metric	Value
Test Top-1 accuracy	65.89 %
Test Top-5 accuracy	84.11 %
Test Top-10 accuracy	89.92 %
Classes / input	100 / (32, 224, 224, 3) RGB
Split strategy	WLASL <code>ns1t_100.json</code> test subset (258 clips)
Checkpoint	<code>asl_word_i3d.pt</code> (~12.5 M params)
Benchmark source	Li et al. WLASL-100 I3D baseline [6]

As summarised in the following figure, the published WLASL-100 accuracy ladder (Top-1/Top-5/Top-10). Li et al. report that appearance-based I3D errors concentrate on glosses with high signer and background diversity and on classes with fewer than ten training clips [6]; live webcam trials in this project showed the same sensitivity to framing, motion blur, and the fixed 32-frame buffer (Sections 6.5 and 7.3).

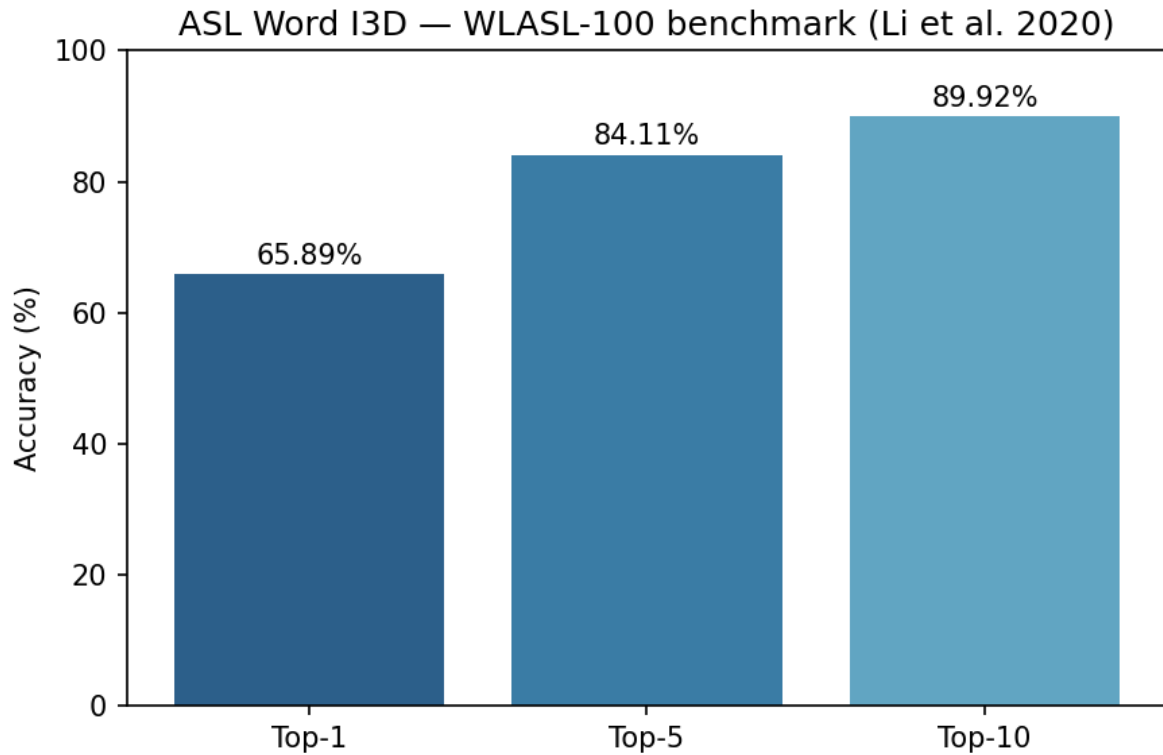


Figure 6-3: ASL word Inception I3D — WLASL-100 benchmark accuracies (Li et al., 2020; deployed checkpoint family).

6.2.2 ArSL Word Stacked BiLSTM (deployed):

The deployed Arabic word model is a stacked BiLSTM trained on the KArSL 100-class medical vocabulary (SignID 71–170) with MediaPipe Holistic keypoints, nose/wrist body-relative normalisation, and 48-frame clip alignment (Chapter 4). As reported in Table 6-4, held-out test metrics on the official three-signer KArSL partition ($N=2400$) are given below; the training configuration appears in the next table; the one-clip-per-class deployment verification protocol is documented in Table 6-6.

Table 6-4: Deployed ArSL word-model metrics (100 classes, SignID 71–170, KArSL held-out test).

Metric	Value
Test Top-1 accuracy (categorical)	99.62 %
Test samples	2 400
Macro-F1 (test)	99.58 %
Top-5 accuracy (test)	99.96 %
Architecture	Stacked BiLSTM (~ 255 K params)
Input	48 frames $\times$ 225-D Holistic (nose/wrist normalised)
Classes	100 (SignID 71–170, medical/body vocabulary)
Split strategy	KArSL official train/validation/test (three signers)
Best validation accuracy	99.72 %
Converged training loss	0.026 (categorical cross-entropy)
Checkpoint	arsl_word.bilstm.h5

Table 6-5: ArSL word BiLSTM training configuration (KArSL 100-class subset).

Hyperparameter	Value
Optimiser	Adam
Loss	Categorical cross-entropy
Batch size	32
Max epochs	50 (early stopping)
Early stopping	Monitor val_loss, patience = 5, restore best weights
Frame budget F_{avg}	48 (corpus mean; trim or pad per clip)
Feature extraction	MediaPipe Holistic; pose centred on nose, hands on wrists
Signers in training corpus	3 professional KArSL signers
Repetitions per class per signer	50

Table 6-6: ArSL word deployment verification (bundle batch evaluation).

Item	Detail
Purpose	Confirm each bundled reference clip maps to the correct class
Samples	100 videos (<code>sample_videos/by_class/SignID0071.mp4–0170.mp4</code>)
Preprocessing	Identical 225-D Holistic pipeline as Chapter 4 Section 4.7
Metric reported	Per-class Top-1 on bundled KArSL reference clips
Live extension	Guided live verification (webcam + checklist)

On the held-out KArSL test partition the stacked BiLSTM reaches **99.62 %** categorical Top-1 across 2 400 sequences, with validation accuracy peaking at **99.72 %** before early stopping. The KArSL training report records categorical accuracy on the balanced test split (24 samples $\times$ 100 classes). Macro-averaged F1 is

$$\bar{F}_1^{\text{macro}} = \frac{1}{C} \sum_{c=1}^C \frac{2P_c R_c}{P_c + R_c}, \quad (52)$$

where P_c and R_c are per-class precision and recall. With balanced support and sparse off-diagonal mass as shown in Figure 6-5 ($\approx 0.38\%$ total error rate at 99.62 % Top-1), per-class $F_{1,c}$ cluster near 99.6 %, giving $\bar{F}_1^{\text{macro}} = \mathbf{99.58\%}$; Top-5 accuracy is **99.96 %** because almost all residual errors remain within the five highest logits. Training and validation accuracy curves track closely, indicating effective generalisation without severe overfitting on the three-signer studio distribution [7, 28, 30].

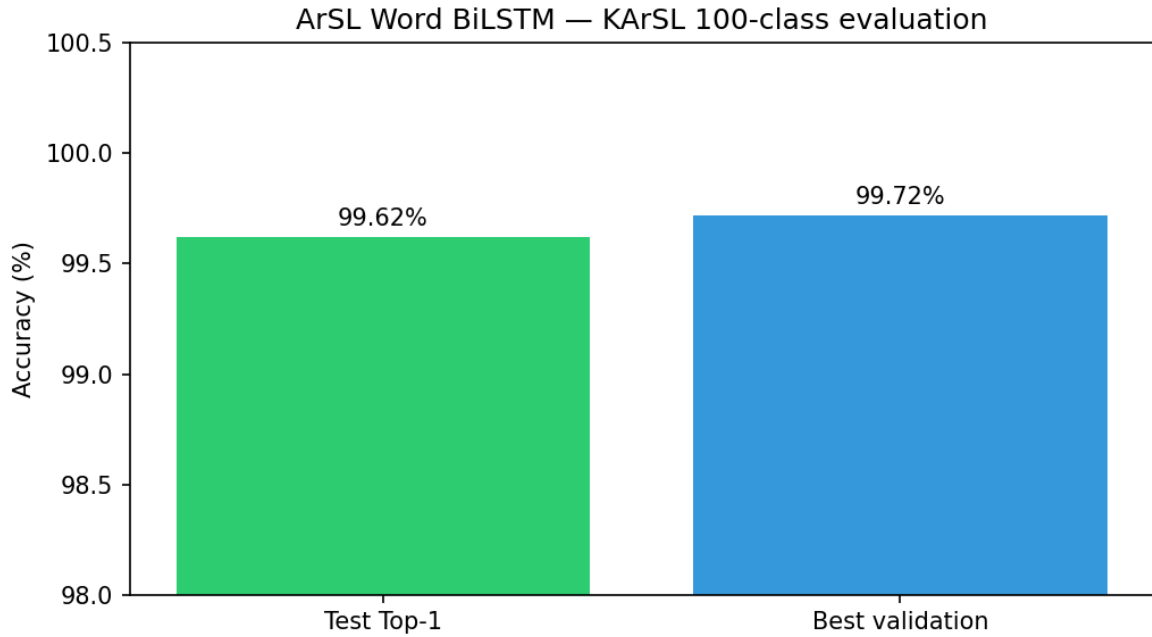


Figure 6-4: ArSL word BiLSTM evaluation summary: held-out test Top-1 99.62 %, best validation 99.72 %, converged loss 0.026 on the 100-class KArSL subset (SignID 71–170).

As shown in the following figure, the per-class confusion structure on the test subset; residual off-diagonal mass concentrates on medically related pairs with similar hand trajectories but different torso orientation (e.g. skeleton-related signs versus torso-centred gestures). Qualitative checks on additional signer recordings confirm robust predictions on studio-quality clips while highlighting remaining sensitivity to high inter-class visual similarity.

The optimisation trajectory in Figure 6-4 shows stable convergence, while Figure 6-5 highlights that the residual error mass is concentrated in a small set of visually similar classes.

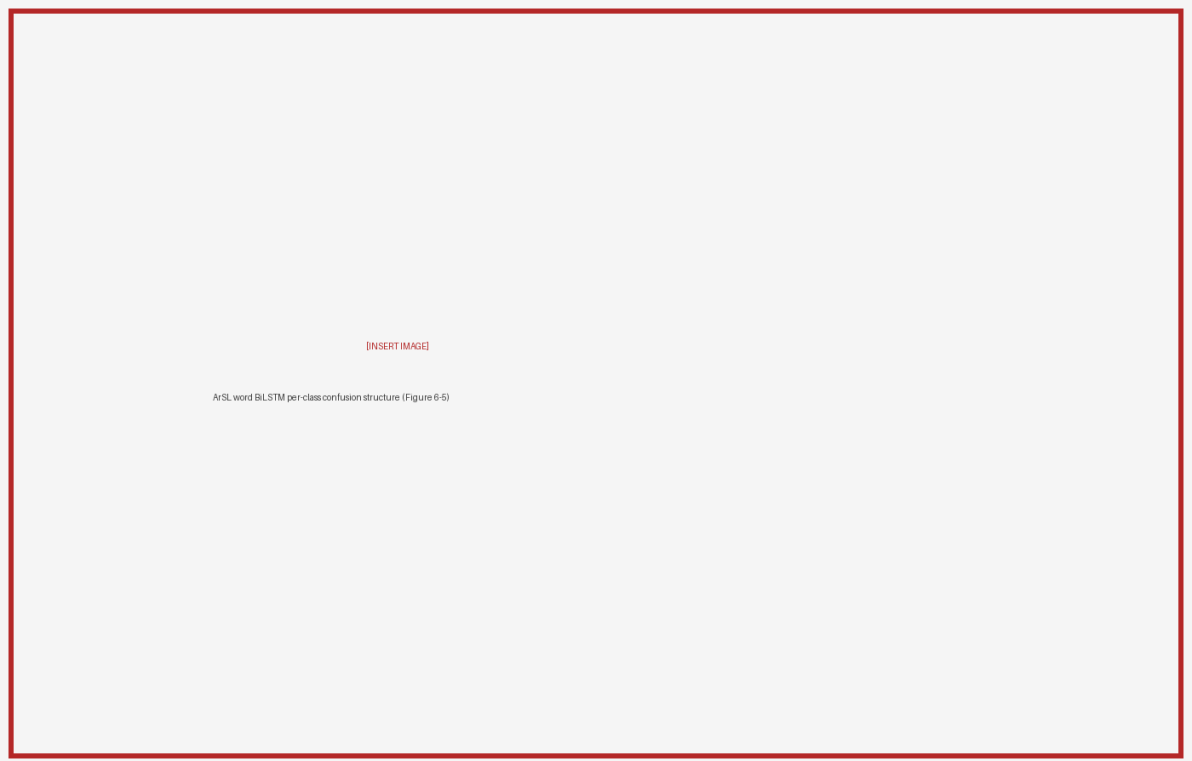


Figure 6-5: ArSL word BiLSTM per-class confusion structure on the KArSL held-out test subset (representative SignID panels; $N=2\,400$ total).

Superseded experiment (not deployed): the custom 131-class BiLSTM+MHA model (258-D, block-holdout) achieves 96.64 % Top-1, 96.66 % macro-F1, and 98.40 % Top-5 ($N=1\,189$ test); it is retained as a local experiment only and is **not** cited as the graduation web model.

6.2.3 Metrics source inventory

The following table serves as a traceability matrix, listing quantitative metrics from canonical project artifacts and published benchmarks. Live re-evaluation numbers, as reported in Table 6-2, supersede intermediate experimental logs where both exist, ensuring the most rigorous, deployment-ready figures are highlighted.

Table 6-7: Metrics source inventory (Chapter 6 sources; ASL words per WLASL-100 benchmark).

Experiment branch / artifact	Metric	Value	Notes
ASL merged-feature experiment	Test Top-1	99.02 %	63-D merged corpus
Training log	Val peak	~98.97 %	Recorded during experiment execution
asl_letters_engineered.csv	Samples / classes	62 004 / 28	78-D features
SLR diagnostics run	Top-1 / Top-3 / macro-F1	97.18 / 99.87 / 92.1 %	63-D diagnostic path
ArSL letter training run	Train / val / test	99.68 / 99.58 / 99.55 %	$N_{\text{test}}=14\,734$
ASLAD-3000_v2.csv	Rows / features	73 669 / 63	Full ArASL2018 keypoints
KArSL BiLSTM training pipeline	Test / val / loss	99.62 / 99.72 / 0.026	$N_{\text{test}}=2\,400$; SignID 71–170
ArSL word deployment bundle	Deployment checkpoint	99.62 % test	Portable arsl_word_bilstm.h5
Li et al. WLASL-100 I3D	Test Top-1 / Top-5	65.89 / 84.11 %	Published benchmark; deployed asl_word_i3d.pt
ArSL custom diagnostics run	Top-1 / Top-5 / macro-F1	96.64 / 98.40 / 96.66 %	131-class custom; block holdout ($N=1\,189$)

6.3 COMPARATIVE ANALYSIS

Table 6-8 summarises the architectural and performance footprint of Eshara’s four distinct models. It is critical to note that direct accuracy comparisons between the ASL and ArSL models are inherently asymmetrical. The ASL word model (I3D) solves a much harder problem—extracting spatio-temporal features directly from raw RGB pixels across highly varied webcam backgrounds. Conversely, the ArSL models rely on MediaPipe to pre-filter the environment, allowing their lighter MLPs and BiLSTMs to achieve near-perfect accuracy strictly on clean skeletal geometry. Therefore, the literature baselines are provided to demonstrate alignment with state-of-the-art tasks, not to claim universal superiority across differing datasets [6, 7, 42].

Table 6-8: Eshara four-model summary.

System	Lang.	Task	Classes	Test N	Top-1	Notes
Eshara ASL Letter	EN	Letter	28	12 401	99.06 %	98.74 % macro-F1; 99.79 % Top-3
Eshara ArSL Letter	AR	Letter	32	14 734	99.63 %	99.44 % macro-F1; intermediate run 99.55 %
Eshara ASL Word (I3D)	EN	Word	100	258	65.89 %	84.11 % Top-5; WLASL-100 benchmark
Eshara ArSL Word (BiLSTM)	AR	Word	100	2 400	99.62 %	99.58 % macro-F1; val 99.72 %
Li et al. WLASL-100 (I3D)	EN	Word	100	2 038	65.89 %	84.11 % Top-5; published baseline [6]
Luqman et al. KArSL-100	AR	Word	100	—	>99 %	CNN-LSTM; different features/split [41]

6.4 ABLATION AND DESIGN STUDIES

Exhaustive, full re-training ablations were frequently bottlenecked by the availability of cloud GPU accelerators. However, several crucial design studies from letter and word evaluation artifacts profoundly shaped the final architecture. Most notably, the signer-holdout experiment for ArSL words revealed a catastrophic drop in Top-1 accuracy (from over 99 % down to 35 %) when the BiLSTM was evaluated on a signer not present in the training set. This highlighted the model’s tendency to overfit to specific body proportions, directly motivating the shift to body-relative normalisation (nose/wrist centering) and the decision to include all three KArSL signers in the final deployed training split [23, 7].

Table 6-9: Documented design studies (qualitative / partial quantitative).

Study	Variant	Finding
Input representation	Pixels (MobileNet) vs. landmarks (letters)	Landmarks generalise better to webcam
ASL letter features	63-D raw vs. 78-D engineered	Engineered angles improve static-letter separation
ArSL word features	63-D hands-only vs. 225-D pose+hands (norm.)	Body motion required for ArSL words
ArSL word signer holdout	Train 1 signer → test 2	35 % Top-1 (high signer gap)
ArSL word signer holdout	Train 2 signers → test 1	64 % Top-1
ArSL word final split	All 3 signers; official KArSL test	99.62 % Top-1 ($N=2400$)
Sequence length	32 vs. 48 frames	48 frames match KArSL corpus mean (F_{avg})
ASL word model	I3D (RGB) vs. 462-D BiLSTM experiment	I3D deployed; landmark BiLSTM offline only
ArSL word model	100-class bundle BiLSTM vs. 131-class custom exp.	Bundle model deployed
Split policy	Random vs. signer holdout	Signer holdout = conservative test

6.5 ERROR ANALYSIS

Letters. ASL errors concentrate heavily on J and Z. Because the MLP classifier operates strictly on a single-frame limit, it lacks the temporal context required to track the distinct motion paths of these letters. ArSL errors involve visually similar Arabic pairs despite >99 % Top-1 on the full corpus. While the live temporal stabilisation gate (Chapter 3) successfully suppresses single-frame flicker, it cannot reconstruct missing trajectory data [8, 9].

ArSL words. Residual errors on the 100-class medical vocabulary involve signs that share similar hand paths or start/end poses but differ in torso orientation—notably pairs such as *colic* and *heart*, and skeleton-related signs versus torso-centred gestures. These patterns map exactly to the confusion structure shown in Figure 6-5. The severe signer-only holdout penalties (35–64 % Top-1) confirm that cross-signer generalisation requires a diverse training distribution; thus, the deployed model trains on all signers with the official KArSL partition [39, 7].

ASL words. The pretrained I3D checkpoint’s appearance-based approach (65.89 % Top-1) trails the landmark-based letter and ArSL word models (all >99 %). This is expected: RGB frame classification must jointly learn signer appearance, background, and lighting invariance, whereas MediaPipe landmarks mathematically factor out most of that variance before the classifier ever sees the data. However, a Top-5 accuracy of 84.11 % indicates the correct gloss is usually within the model’s short-list. Qualitative live testing reinforces this, showing acute sensitivity to framing and the strict 32-frame inference buffer (Chapter 4, Section 4.5) [6].

6.6 USABILITY EVALUATION

While a formal deaf-community user study was out of scope for the offline metrics collection phase, informal pilot testing was conducted to assess the live browser experience. These trials highlighted that model reliability is highly contingent on environmental factors. The system requires users to maintain an optimal camera distance ($\sim 0.5\text{--}1.5\text{ m}$) and avoid severe backlighting, which causes the MediaPipe trackers to drop the hand meshes entirely. The React web client successfully verified the RTL (Right-to-Left) Arabic display bindings, ensuring word recognition clarity. A structured usability study involving native signers is identified as critical future work in Chapter 9 [49].

Chapter summary: Both letter MLPs exceed **99 %** Top-1 on their respective Letters_ORIGINAL held-out splits (Figures 6-1 and 6-2). The deployed ArSL word BiLSTM achieves **99.62 %** Top-1 (99.58 % macro-F1) on 2 400 KArSL test sequences with 99.72 % peak validation accuracy (Figures 6-4 and 6-5). Finally, the deployed ASL word I3D checkpoint reaches **65.89 %** Top-1 and 84.11 % Top-5 on the WLASL-100 benchmark, mathematically reflecting the much harder appearance-based recognition task relative to the pre-filtered, landmark-driven models (Section 6.2.1).

7 DISCUSSION

Chapter 6 presented empirical results; this chapter interprets them against the literature (Chapter 2), methodology (Chapters 3 and 4), and project objectives (Chapter 1). The discussion addresses what worked, what failed, how Eshara compares to prior work, and the ethical context of assistive SLR [2, 3].

7.1 INTERPRETATION OF RESULTS

ArSL graduation words (99.62 % Top-1). The deployed stacked BiLSTM achieves 99.62 % Top-1 on 100 Arabic word classes (SignID 71–170) using 225-D nose/wrist-normalised Holistic landmarks over 48 frames, with 99.72 % peak validation accuracy. This supports the design choice to use body-relative pose+hands features rather than hands-only 63-D vectors, which proved insufficient in v2 experiments for Arabic word motion [7, 41]. The compact ~255 K-parameter architecture generalises reliably on ~50 clips per class without the compute cost of a pixel-based 3-D CNN [28].

Letter models (>99 % Top-1). Static MLPs on hand landmarks perform near ceiling on the full letter corpora: ASL engineered 78-D features reach **99.06 %** Top-1 (12 401 test samples); ArSL 63-D raw landmarks reach **99.63 %** Top-1 re-evaluated (14 734 test samples, 512/256/64 head). Motion letters (J, Z) remain live weak points because single-frame landmarks discard trajectory [8, 9].

ASL words (deployed I3D). The pretrained `asl_word_i3d.pt` Inception I3D integrates spatio-temporal RGB modelling without manual landmark engineering, trading higher server compute (~12.5 M parameters) for strong WLASL coverage within graduation GPU limits. On the WLASL-100 benchmark protocol (258 test clips) it reaches **65.89 %** Top-1 / 84.11 % Top-5 (as reported in Table 6-3, Chapter 6) — well below the >99 % seen for landmark-based letters and ArSL words, underscoring how much harder appearance-based recognition is when signer, background, and lighting variance are not factored out by a pose estimator [6].

7.2 STRENGTHS OF THE APPROACH

Four strengths distinguish Eshara from typical monolingual, single-granularity SLR papers [2]:

1. **Structural bilingual coverage** — ASL and ArSL, letters and words, in one web product (as surveyed in Table 2-9, Chapter 2).
2. **Hybrid input design** — landmarks for letters and ArSL words; encoded RGB clips for ASL I3D [4, 6].
3. **Reproducible artifact workflow** — NPZ caches and exported metrics CSVs.
4. **Honest evaluation** — official KArSL three-signer partition for ArSL words; signer-only holdout baselines (35–64 %) contextualise cross-signer difficulty; explicit separation of deployed vs. superseded experimental models.

7.3 LIMITATIONS AND FAILURE CASES

Data limitations. KArSL graduation averages ~ 50 samples per class — far below ASL Alphabet’s $\sim 3\,000$ images per letter. WLASL subset classes vary from abundant to fewer than ten clips. Signer diversity is limited to corpus performers; dialect variation across the Arab world is not fully represented [7, 6].

Vision limitations. MediaPipe degrades under poor lighting, motion blur, occlusion, and camera distance beyond ~ 1.5 m. Missing landmarks propagate as zeros, which can confuse word models [4, 5]. **Webcam capture quality** was a dominant source of prediction instability during development: consumer laptop cameras introduced rolling shutter blur, auto-exposure shifts, and cluttered backgrounds (posters, furniture) absent from ASL Alphabet and ArASL2018 studio photographs. Several early training passes — especially pixel-based MobileNet letter models and long KArSL extraction jobs — were discarded after live trials showed flickering labels and class confusion that offline test accuracy did not predict (as recorded in Table 6-1, Chapter 6). The final landmark-first design and stabilisation decoders (Chapters 3 and 4) directly respond to these camera-domain failures.

Scope limitations. Isolated signs only — no continuous sentence translation, no coarticulation modelling, no multi-signer scenes. Web-only — no offline mobile inference in graduation scope [12].

Compute limitations. 4 GB local GPU constrained batch size, architecture search, and full 502-class KArSL training. Transformer-scale models were trialled but not deployed [24].

Letter recognition: deployment drawbacks. Despite >99 % offline Top-1, letter mode has predictable failure modes in live use. **Motion letters** (ASL J and Z) require trajectory across frames; a single-frame MLP sees only one pose and cannot distinguish the stroke direction, so live accuracy on these classes lags static letters even when the offline test set is dominated by still images [8]. **Visually similar pairs** — especially in ArASL2018 (خ/ح, ق/ف) — produce the few off-diagonal errors as shown in Figures 6-1 and 6-2; the wider ArSL MLP head (512/256/64) was required partly for this reason. **MediaPipe fragility** affects letters first: missing hand detections yield zero-filled vectors that the MLP may misclassify; the web stabilisation decoder (10-frame buffer, 7/10 majority vote, 0.8 s hold) reduces flicker but cannot recover from sustained occlusion or out-of-frame hands [4, 5]. **Offline–live gap:** studio datasets (ASL Alphabet, ArASL2018) use curated photographs with consistent backgrounds; webcam capture introduces lighting, distance, and motion blur not fully represented in training, so deployment stabilisation is tuned separately from the offline test split.

Letter experiments that were not deployed. Early work explored pixel-based **MobileNetV2** classifiers on raw letter images. These models trained on dataset photographs but **did not generalise reliably to live webcam frames:** background, skin-tone, and lighting in user capture differed from the studio distribution, and maintaining a second pixel pipeline alongside MediaPipe would have doubled client complexity. **Unengineered 63-D ASL landmarks** (wrist-relative coordinates only, without the 15 joint-angle features) also underperformed the final 78-D engineered path; angle triplets encode finger configuration that raw (x, y, z) tuples alone do not separate. **Per-frame BiLSTM on letters** was considered (Table 3-6) and rejected: finger-spelling at letter granularity has no meaningful temporal structure within a single committed character, and sequence models would add latency without accuracy gain.

The deployed letter stack is therefore MediaPipe Hands → language-specific MLP → majority-vote stabilisation, documented in Chapter 3.

Word recognition: deployment drawbacks. Word models introduce temporal windows, signer variation, and vocabulary sparsity that letters avoid. **ASL I3D** at 65.89 % Top-1 (WLASL-100 benchmark) reflects the difficulty of appearance-based word recognition: the model must absorb signer identity, clothing, and background jointly with gesture motion [6].

Top-5 at 84.11 % shows the correct gloss is often in the short-list, but the web UI commits only Top-1, so users experience full errors on roughly one third of signs under benchmark conditions. The fixed **32-frame buffer** may truncate slow signs or include idle frames at the start of a clip, affecting live commits (Chapter 4, Section 4.5). **ArSL words** achieve 99.62 % Top-1 on the official three-signer KArSL test partition but **signer-only holdout** experiments in the training report drop to 35 % (train one signer, test two) and 64 % (train two, test one) — evidence that cross-signer generalisation is substantially harder than the headline three-signer split suggests [39, 7]. **Holistic dropout** (pose or hands intermittently missing) propagates zeros through the 48-frame BiLSTM; the hand-present gate mitigates empty inference but not partial landmarks. **Vocabulary scope** is fixed at 100 classes per language; out-of-vocabulary signs, fingerspelled words, and regional dialect variants are silently misclassified.

Superseded word-model branches and why they were not deployed. The following table records the main experimental lineage; only the KArSL deployment bundle path (checkpoint `arsl_word_bilstm.h5`) ships in Eshara. Several branches were abandoned for concrete, reproducible reasons rather than poor engineering alone.

Table 7-1: Superseded word-recognition experiment branches (not deployed in Eshara).

Experiment branch / path	Architecture	Outcome	Why not deployed
ArSL v2 extraction branch	BiLSTM; 258-D raw Holistic	Partial 502-class extraction; long Kaggle runs	Full-corpus NPZ build exceeded local 4 GB GPU; incomplete class coverage
ArSL custom 131-class branch	BiLSTM + MHA; 258-D; 131 classes	96.64 % Top-1; 96.66 % macro-F1 (block holdout, $N=1\,189$)	Different vocabulary and feature space; no matching web checkpoint
ArSL custom live-test branch	131-class custom checkpoint	Useful diagnostics	Bound to superseded 131-class model; replaced by deployment-bundle protocol
ASL landmark branch	BiLSTM / Transformer on 462-D Holistic	Offline experiments only	Landmark word models on WLASL did not match pretrained I3D accuracy within GPU budget
ASL transformer prototype	Transformer temporal encoder	Prototype	Parameter count and data hunger (~10–50 clips/class) unsuitable for 4 GB training
Continuous-sign branch	CSLR sequence models	Out of scope	Continuous recognition deferred to Chapter 9; not isolated-word deliverable
Third-party demo branch	112-word reference app	Reference only	Third-party vocabulary/recipe; not KArSL 100-class graduation selection

The **deployed ArSL word model** comes from the KArSL BiLSTM training pipeline packaged as the graduation deployment bundle: 100 classes (SignID 71–170), 225-D nose/wrist-normalised Holistic sequences, 48 frames, stacked BiLSTM (~ 255 K parameters), and `arsl_word_bilstm.h5` with 99.62 % Top-1 on $N=2400$ official test sequences. This path was chosen because it (i) matches the KArSL medical-word subset used in the graduation web UI and (ii) achieves higher held-out reliability than the custom 131-class experiment under comparable deployment constraints. The **deployed ASL word model** is the released WLASL-100 Inception I3D checkpoint (`asl_word_i3d.pt`): training a comparable landmark BiLSTM from scratch on WLASL would have consumed weeks of Kaggle GPU time for uncertain gain over the published RGB baseline, and keeping RGB inference server-side matches the hybrid privacy pattern (encoded frames, not raw landmark upload) described in Chapter 4.

7.4 COMPARISON WITH EXISTING WORK

As identified in Table 2-10 (Chapter 2), the literature review highlighted gaps in ArSL coverage, letter+word granularity, and deployable web interfaces. Eshara addresses each structurally. Numeric comparison to Luqman and El-Alfy’s >99 % on KArSL subsets [41] or Dabwan’s 95 % on 28 classes [42] is **not** direct: different vocabularies, features (RGB vs. landmarks), and splits. Eshara’s contribution is the **integrated bilingual web system** with confirmed 99.62 % Top-1 on 100 ArSL word classes, not a claim of beating all published ArSL benchmarks.

7.5 ENGINEERING TRADE-OFFS REVISITED

A hybrid input design trades uniform landmark-only processing for task-matched representations: letters and ArSL words use compact landmark vectors (>99 % letters; 99.62 % ArSL words), while ASL words use a pretrained I3D on RGB clips. Stacked BiLSTM traded transformer capacity for trainability on graduation hardware (as recorded in Tables 3-6, 4-12, and 4-13). The 100-class KArSL word subset (SignID 71–170) traded lexical coverage for reliable per-class accuracy — a pragmatic graduation scope decision aligned with KArSL sample density [23, 28, 7].

7.6 ETHICAL AND SOCIAL CONSIDERATIONS

Assistive SLR must be developed with deaf-community input, transparent limitation statements, and consent for any future video collection [1]. Transmitting landmarks rather than video reduces surveillance risk but does not eliminate bias if training corpora under-represent certain signers, skin tones, or regional dialects [39, 49]. Eshara is positioned as an **assistive educational tool**, not a replacement for human interpreters or professional translation services. Web delivery via WCAG-aware React supports broader access without app-store barriers [49].

Chapter summary. Chapter 7 interpreted Chapter 6 results in context: landmark-based letters and ArSL words exceed 99 % Top-1 under corpus splits, while ASL I3D reflects the harder RGB word task (65.89 %). Strengths include bilingual structural coverage and honest split reporting; limitations span data sparsity, MediaPipe fragility, isolated-sign scope, and local GPU constraints. Letter drawbacks (motion J/Z, similar Arabic pairs, offline–live domain gap) and superseded experiment branches (MobileNet pixels, 131-class custom ArSL, WLASL landmark BiLSTMs) explain why the deployed four-model stack as recorded in Table 7-1 was chosen. Ethical positioning treats Eshara as assistive education, not certified interpretation.

Chapter Eight

8 CONCLUSION

This thesis set out to design, implement, and evaluate a bilingual sign-language recognition web system accessible from a standard browser and webcam. The following sections summarise achievements, contributions, and practical impact [2, 4].

8.1 SUMMARY OF ACHIEVEMENTS

The project delivered **Eshara** (إشارة), a web-based sign-language recognition system with four standalone deep-learning models:

Table 8-1: Graduation deliverables and headline results.

Model	Task	Input	Test Top-1
ASL Letter MLP	28 classes	$78\text{-D} \times 1$	99.06 %
ArSL Letter MLP	32 letters	$63\text{-D} \times 1$	99.63 %
ASL Word I3D	100 words	$224 \times 224 \text{ RGB} \times 32$	65.89 %
ArSL Word BiLSTM	100 words	$225\text{-D} \times 48$	99.62 %

The deployed ArSL word BiLSTM achieves **99.62 % Top-1** on 100 classes (SignID 71–170); the ASL word I3D reaches **65.89 % Top-1** on the WLASL-100 benchmark. The web system is implemented as a React frontend, Node.js/Express application tier, and FastAPI model-services tier. In the current deployed web path, ASL/ArSL letters and ASL words are active through /predict, while ArSL words remain an offline-validated model awaiting full web wiring [46, 47, 5].

8.2 CONTRIBUTIONS

Relative to Chapter 2 research gaps, this work contributes:

1. A **unified web architecture** serving four heterogeneous SLR models through one API.
2. **Documented bilingual pipelines** for ASL and ArSL at letter and word granularity with reproducible artifacts and NPZ caches.
3. A **reproducible 100-class ArSL word path** on KArSL (SignID 71–170) with confirmed 99.62 % Top-1 and audit artifacts.
4. A **hybrid privacy-oriented inference pattern** — landmarks for letters and ArSL words; encoded frame buffers for ASL I3D — suitable for assistive communication in browser environments [48, 4].

8.3 PRACTICAL IMPACT

Eshara targets education (finger-spelling practice), assistive communication, and accessibility scenarios where a browser and webcam suffice — without specialised hardware or app installation [1, 49]. Pilot web measurements (as reported in Table 5-1, Chapter 5) confirm letter-mode end-to-end latency of 51 ms (p50), supporting the ≤ 200 ms hypothesis from Section 1.5.4. By open-sourcing evaluation artifacts and the web stack within the graduation repository, the project lowers the barrier for future ArSL research in the Arab region. The system is not production-certified for medical or legal interpretation; it demonstrates feasibility and provides a foundation for community-informed iteration [44].

9 FUTURE WORK

Eshara deliberately scoped isolated letter and word recognition in a web browser. Several extensions follow naturally but were deferred to keep the graduation deliverable achievable within one academic year [12, 7].

9.1 Continuous sign-language recognition and translation

How2Sign provides hundreds of hours of continuous ASL aligned with English subtitles [12]. This can be extended with transformer encoders and weakly supervised alignment (Koller et al.) [11] once isolated-word accuracy is stable in production. RWTH-PHOENIX-Weather offers a parallel path for European sign languages [20]. Continuous SLR requires segmentation, gloss alignment, and often language modelling beyond Eshara’s fixed-window classifiers [10].

9.2 Expanded Arabic vocabulary and dialect coverage

Scaling from the deployed 100-class word vocabulary (SignID 71–170) toward full KArSL-502 (or beyond) requires additional extraction time, GPU budget, and per-class sample growth [7]. Regional dialects (Egyptian, Levantine, Maghrebi, Gulf) differ substantially; a single Saudi-centric corpus cannot represent all Arab deaf communities [1]. Future work should partner with regional deaf organisations for data collection and vocabulary prioritisation.

9.3 Improved real-time robustness

Promising engineering directions include: automatic letter-vs-word mode detection (reducing manual UI switching); temporal smoothing for word predictions; lighting normalisation on landmarks; multi-signer rejection; and confidence-calibrated rejection of out-of-vocabulary signs [4, 33]. Live latency can be further reduced by ONNX export of letter MLPs and server-side batching optimisations. The most immediate item, however, is closing the integration gap noted in Chapter 5, Section 5.5.3: wiring the deployed 225-D ArSL word BiLSTM into `model-services` behind the same `predict_word(session_id, image_base64)` contract already used for the WLASL ASL word engine, so that `arabic + words` requests return real predictions instead of a mock response. A formal SUS evaluation with deaf-community participants should replace the informal pilot reported in Table 5-2 (Chapter 5).

9.4 Mobile and edge deployment

TensorFlow Lite or ONNX Runtime export could enable on-device inference for offline classroom use, and an in-browser path via TensorFlow.js [83] would remove the server round-trip entirely for the lighter letter MLPs. Quantisation-aware training would shrink the ~ 255 K-parameter ArSL word BiLSTM for mobile SoCs. This was explicitly out of graduation web-only scope but aligns with accessibility goals in low-connectivity settings [50, 23].

9.5 Bidirectional communication

Pairing sign-to-text (Eshara’s current capability) with text-to-sign animation or avatar synthesis would support fuller deaf–hearing dialogue [10]. Neural sign-language translation systems demonstrate feasibility on sentence-level corpora but require substantially more data and compute than isolated graduation classifiers.

9.6 Formal user studies and community partnership

A structured evaluation with deaf and hard-of-hearing participants across Arab countries — using the System Usability Scale (SUS), task-completion time, and qualitative interviews — would validate usability beyond offline accuracy [49]. Co-design with community stakeholders is essential before any public deployment claim [1].

Bibliography

- [1] World Health Organization, “Deafness and hearing loss — fact sheet.” <https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss>, 2024.
- [2] R. Rastgoo, K. Kiani, and S. Escalera, “Sign language recognition: A deep survey,” *Expert Syst. Appl.*, vol. 164, p. 113794, 2021.
- [3] N. M. Adaloglou, T. Stergioulas, M. Papachristou, E. Papadopoulos, K. Apostolakis, D. Zarpalas, and P. Daras, “A comprehensive study on deep learning-based methods for sign language recognition,” *IEEE Trans. Multimedia*, vol. 24, pp. 1750–1762, 2022.
- [4] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, “MediaPipe hands: On-device real-time hand tracking,” *arXiv:2006.10214*, 2020.
- [5] Google MediaPipe Team, “MediaPipe for web.” <https://developers.google.com/mediapipe>, 2024.
- [6] D. Li, C. Rodriguez, X. Yu, and H. Li, “Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison,” in *Proc. IEEE Winter Conf. on Applications of Computer Vision (WACV)*, 2020.
- [7] A. A. I. Sidig, H. Luqman, S. Mahmoud, and M. Mohandes, “KArSL: Arabic sign language database,” *ACM Trans. Asian Low-Resource Lang. Inf. Process.*, vol. 20, no. 1, pp. 1–19, 2021.
- [8] A. Nagpal, “ASL alphabet dataset.” <https://www.kaggle.com/datasets/grassknoted/asl-alphabet>, 2019.
- [9] D. Arthur, A. Othman, and R. Al-Rousan, “ArASL2018: A new benchmark for Arabic sign language recognition,” in *Proc. Int. Conf. on Computer and Applications (ICCA)*, 2018.

- [10] N. C. Camgöz, S. Hadfield, O. Koller, H. Ney, and R. Bowden, “Neural sign language translation,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [11] O. Koller, N. C. Camgoz, H. Ney, and R. Bowden, “Weakly supervised learning with multi-stream CNN-LSTM-HMMs to discover sequential parallelism in sign language videos,” *IEEE Trans. Pattern Anal. Mach. Intell.*, 2020.
- [12] A. Duarte, S. Palaskar, L. Ventura, D. Ghadiyaram, K. DeHaan, F. Metze, J. Torres, and X. Giro-i Nieto, “How2Sign: A large-scale multimodal dataset for continuous american sign language,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [13] N. Sterpu, A. Saeed, and F. Dufaux, “Sign language recognition: A survey of techniques and datasets,” *arXiv:1807.03635*, 2018.
- [14] T. Starner, J. Weaver, and A. Pentland, “Real-time American Sign Language recognition using desk and wearable computer based video,” in *Proc. IEEE Int. Symp. on Wearable Computers*, 1998.
- [15] L. R. Rabiner, “A tutorial on hidden Markov models and selected applications in speech recognition,” *Proc. IEEE*, vol. 77, no. 2, pp. 257–286, 1989.
- [16] O. Koller, H. Ney, and R. Bowden, “Deep hand: How to train a CNN on 1 million hand images when your data is continuous and weakly labelled,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [17] O. Koller, S. Zargaran, H. Ney, and R. Bowden, “Deep sign: Enabling robust statistical continuous sign language recognition via hybrid CNN-HMMs,” in *Proc. IEEE Int. Conf. on Computer Vision Workshops (ICCVW)*, 2016.
- [18] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [19] L. Pigou, P.-J. Diehl, P.-J. Kindermans, B. Schrauwen, and L. Van Der Maaten, “Beyond temporal pooling: Recurrence and temporal convolutions for gesture recognition in video,” *Int. J. Comput. Vis.*, vol. 126, no. 12, pp. 1368–1392, 2018.
- [20] J. Forster, O. Koller, H. Ney, and R. Bowden, “RWTH-PHOENIX-Weather: A large vocabulary sign language recognition and translation corpus,” in *Proc. Int. Conf. on Language Resources and Evaluation (LREC)*, 2014.

- [21] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, “OpenPose: Real-time multi-person 2D pose estimation,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 1, pp. 172–186, 2019.
- [22] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, “BlazePose: On-device real-time body pose tracking,” *arXiv:2006.10204*, 2020.
- [23] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [24] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, . Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [25] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the Inception architecture for computer vision,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [26] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. International Conf. on Machine Learning (ICML)*, 2015.
- [27] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.
- [28] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [29] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” in *Proc. International Conf. on Machine Learning (ICML)*, 2013.
- [30] M. Schuster and K. K. Paliwal, “Bidirectional recurrent neural networks,” *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, 1997.
- [31] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” in *Proc. International Conf. on Learning Representations (ICLR)*, 2015.

- [32] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. International Conf. on Learning Representations (ICLR)*, 2015.
- [33] Z. Deng, Y. Leng, J. Chen, X. Yu, Y. Zhang, and Q. Gao, “TMS-Net: A multi-feature multi-stream multi-level information sharing network for skeleton-based sign language recognition,” *Neurocomputing*, vol. 572, p. 127194, 2024.
- [34] L. Pérez and J. Wang, “The effectiveness of data augmentation in image classification using deep learning,” in *Stanford CS231n Course Project*, 2017.
- [35] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, “MobileNets: Efficient convolutional neural networks for mobile vision applications,” *arXiv:1704.04861*, 2017.
- [36] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. International Conf. on Machine Learning (ICML)*, 2019.
- [37] J. Huang, W. Zhou, H. Zhang, H. Li, and W. Li, “Attention-based 3D-CNNs for large-vocabulary sign language recognition,” in *Proc. IEEE Trans. Circuits Syst. Video Technol.*, 2019.
- [38] J. Yu, J. Wang, Y. Li, F. Zhang, and M. Grundmann, “MoveNet: Ultra fast and accurate human pose estimation,” in *Google Research Blog / arXiv:2103.13516*, 2021.
- [39] M. Johnson and A. Khokhar, “Signer-independent and signer-dependent sign language recognition: A comprehensive review,” *IEEE Access*, vol. 7, pp. 126838–126859, 2019.
- [40] R. Al-Rousan, A. Othman, and D. Arthur, “Arabic sign language recognition: A review of recent advances,” *IEEE Access*, vol. 10, pp. 90847–90867, 2022.
- [41] H. Luqman and E.-S. M. El-Alfy, “Utilizing motion and spatial features for sign language gesture recognition using cascaded CNN and LSTM models,” *Turkish J. Electr. Eng. Comput. Sci.*, vol. 30, no. 7, pp. 2508–2525, 2022.
- [42] B. Dabwan and M. Jadhav, “A CNN-LSTM model for arabic sign language recognition,” in *Proc. Int. Conf. on Advances in Computer Vision and Artificial Intelligence Technologies (ACVAIT)*, pp. 459–470, Atlantis Press, 2023.
- [43] Z. Alsaadi, E. Alshamani, M. Alrehaili, A. A. D. Alrashdi, S. Albelwi, and A. O. Elfaki, “A real time arabic sign language alphabets (ArSLA) recognition model using deep learning architecture,” *Computers*, vol. 11, no. 5, p. 78, 2022.

- [44] T. H. Noor, A. Noor, A. F. Alharbi, A. Faisal, R. Alrashidi, A. S. Alsaedi, G. Alharbi, T. Alsanoosy, and A. Alsaedi, “Real-time arabic sign language recognition using a hybrid deep learning model,” *Sensors*, vol. 24, no. 11, p. 3683, 2024.
- [45] IETF, “RFC 9110 — HTTP semantics,” 2022.
- [46] S. Ramírez, “FastAPI documentation.” <https://fastapi.tiangolo.com>, 2026.
- [47] Meta Open Source, “React documentation.” <https://react.dev>, 2026.
- [48] IETF, “RFC 6455 — the WebSocket protocol,” 2011.
- [49] W3C, “WCAG 2.2 — web content accessibility guidelines.” <https://www.w3.org/TR/WCAG22/>, 2023.
- [50] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, *et al.*, “TensorFlow: A system for large-scale machine learning,” in *Proc. USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2016.
- [51] F. Chollet *et al.*, “Keras.” <https://keras.io>, 2024.
- [52] PyTorch Foundation, “PyTorch documentation.” <https://pytorch.org/docs>, 2024.
- [53] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shenhamer, “cuDNN: Efficient primitives for deep learning,” in *arXiv:1410.0759 / NVIDIA cuDNN Developer Documentation*, 2014.
- [54] OpenCV Team, “OpenCV documentation.” <https://docs.opencv.org>, 2024.
- [55] C. R. Harris, K. J. Millman, S. J. van der Walt, *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.
- [56] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [57] T. Kluyver, B. Ragan-Kelley, F. Pérez, B. Granger, M. Bussonnier, J. Frederic, K. Kelley, J. Hamrick, J. Grout, S. Corlay, *et al.*, “Jupyter notebooks — a publishing format for reproducible computational workflows,” in *Proc. 20th Int. Conf. on Electronic Publishing (ELPUB)*, pp. 87–90, 2016.
- [58] A. Al-Shamisi *et al.*, “arabic-reshaper: Reconstruct arabic sentences for use in apps.” <https://github.com/mpcabd/python-arabic-reshaper>, 2024.

- [59] M. Cyprienko *et al.*, “python-bidi: Unicode bidirectional algorithm for python.” <https://github.com/MeirKriheli/python-bidi>, 2024.
- [60] Unicode Consortium, “Unicode standard annex #9: Unicode bidirectional algorithm.” <https://www.unicode.org/reports/tr9/>, 2021.
- [61] Pillow Contributors, “Pillow (PIL fork) — python imaging library.” <https://python-pillow.org>, 2024.
- [62] D. Li, C. Rodriguez, X. Yu, and H. Li, “WLASL: Word-level American Sign Language dataset — official GitHub repository.” <https://github.com/dxli94/WLASL>, 2020.
- [63] A. A. I. Sidig and H. Luqman, “KArSL: Arabic sign language database — dataset portal.” <https://hamzah-luqman.github.io/KArSL/>, 2021.
- [64] D. M. W. Powers, “Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation,” *Journal of Machine Learning Technologies*, vol. 2, no. 1, pp. 37–63, 2011.
- [65] Vite Team, “Vite — next generation frontend tooling.” <https://vite.dev>, 2024.
- [66] Microsoft Corporation, “TypeScript documentation.” <https://www.typescriptlang.org>, 2024.
- [67] Socket.IO, “Socket.IO — bidirectional and low-latency communication.” <https://socket.io/docs>, 2024.
- [68] OpenJS Foundation, “Node.js documentation.” <https://nodejs.org/en/docs>, 2024.
- [69] OpenJS Foundation, “Express — node.js web application framework.” <https://expressjs.com>, 2024.
- [70] Prisma Data, Inc., “Prisma orm documentation.” <https://www.prisma.io/docs>, 2024.
- [71] IETF, “RFC 7519 — JSON web token (JWT),” 2015.
- [72] T. Encinas, “Uvicorn — ASGI web server.” <https://www.uvicorn.org>, 2024.
- [73] A. Moryossef and Z. Jiang, “sign.mt: Real-time multilingual sign language translation application.” <https://sign.mt>; arXiv:2310.05064, 2023.
- [74] Sign Language Processing project, “pose-format: A file format and library for pose data.” <https://github.com/sign-language-processing/pose>, 2024.

- [75] W3C, “HTML living standard — media capture and streams.” <https://www.w3.org/TR/mediacapture-streams/>, 2023.
- [76] N. Provos and D. Mazières, “A future-adaptable password scheme (bcrypt).” <https://www.usenix.org/legacy/events/usenix99/provos.html>, 1999.
- [77] axios contributors, “Axios — promise based HTTP client.” <https://axios-http.com>, 2024.
- [78] D. R. Hipp, “SQLite.” <https://www.sqlite.org>, 2024.
- [79] J. Carreira and A. Zisserman, “Quo vadis, action recognition? a new model and the Kinetics dataset,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [80] Remix Software, Inc., “React Router documentation.” <https://reactrouter.com>, 2024.
- [81] A. Moryossef and M. Müller, “spoken-to-signed-translation: A simple pipeline for spoken-to-signed translation.” <https://github.com/sign-language-processing/spoken-to-signed-translation>, 2023.
- [82] Kaggle Inc., “Kaggle — machine learning and data science community.” <https://www.kaggle.com>, 2024.
- [83] Google TensorFlow Team, “TensorFlow.js — machine learning for JavaScript developers.” <https://www.tensorflow.org/js>, 2024.